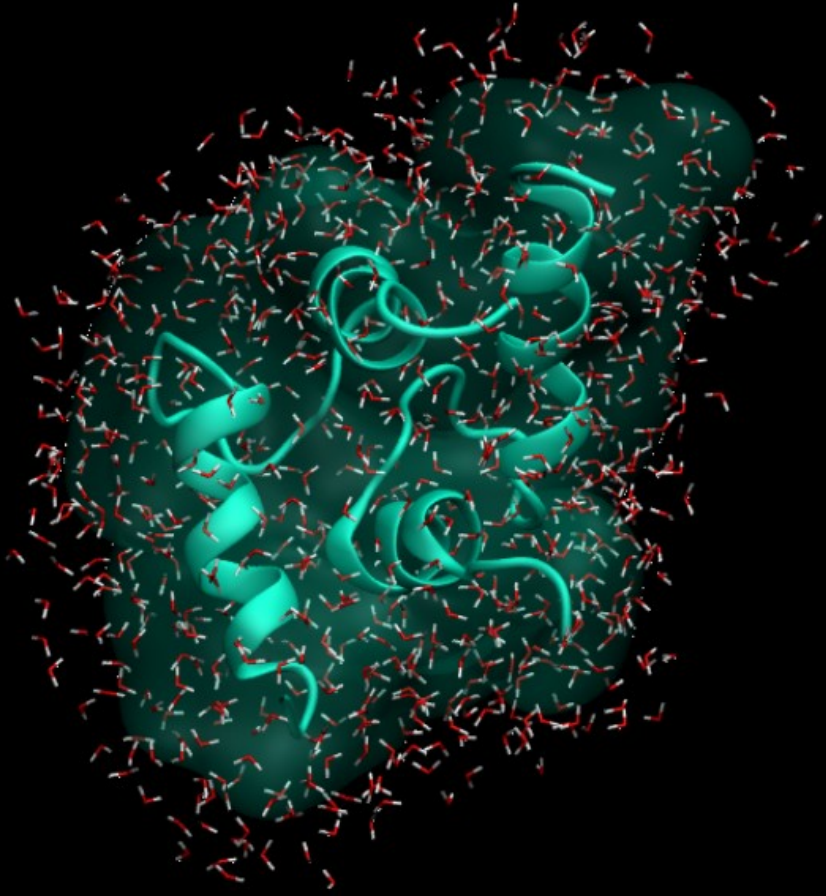




Molecular dynamics

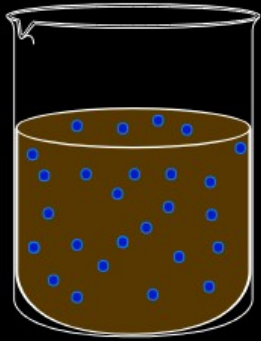
General concepts

Simon Gravelle
LIPhy (Grenoble France), CNRS

Why use molecular dynamics (MD) ?

Why use molecular dynamics (MD) ?

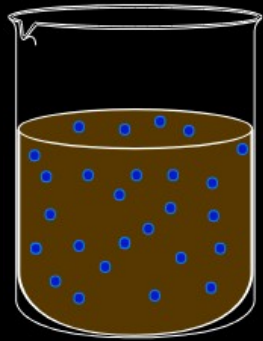
Experiment



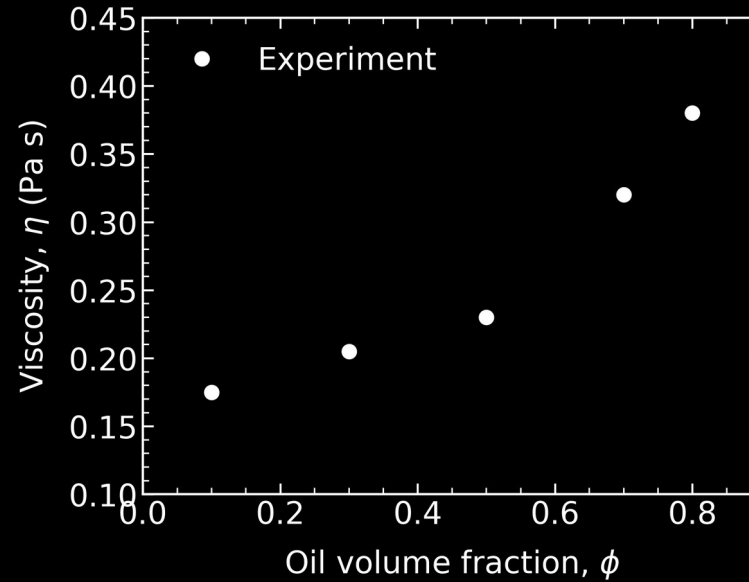
Oil-in-water
Emulsion

Why use molecular dynamics (MD) ?

Experiment

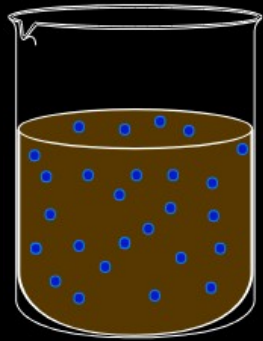


Oil-in-water
Emulsion

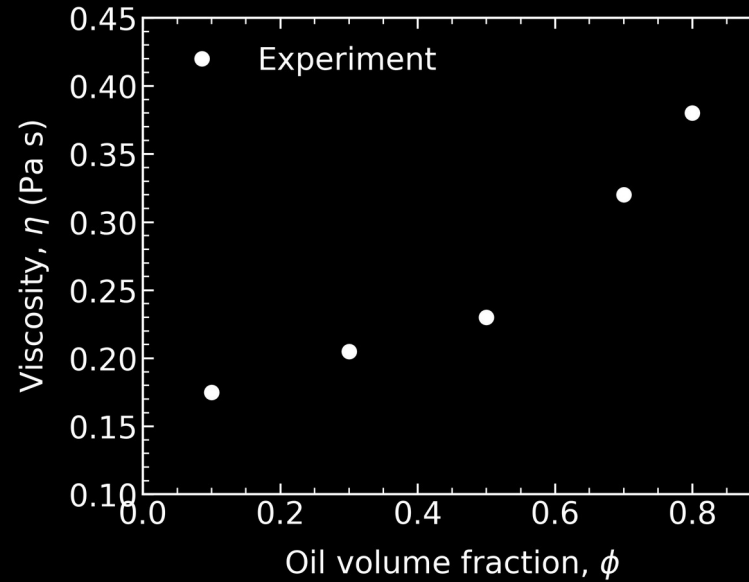


Why use molecular dynamics (MD) ?

Experiment



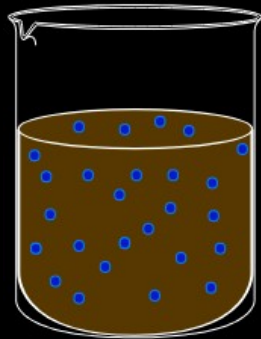
Oil-in-water
Emulsion



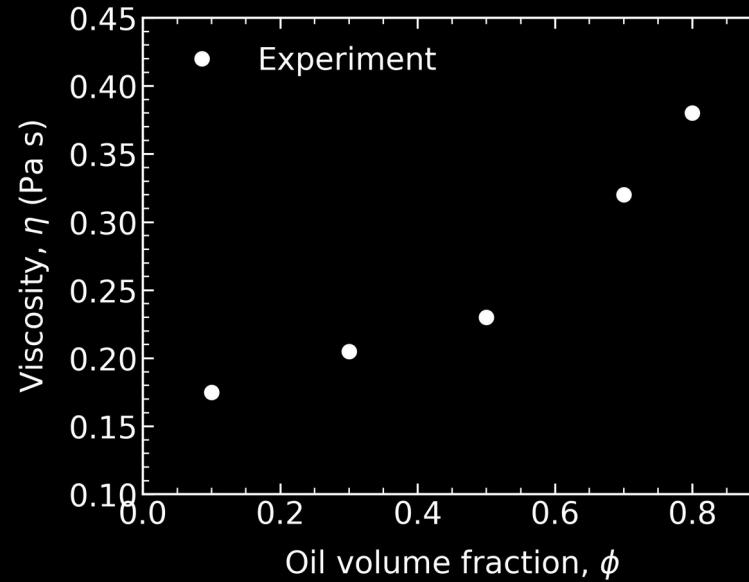
Experimental results can
be difficult to interpret

Why use molecular dynamics (MD) ?

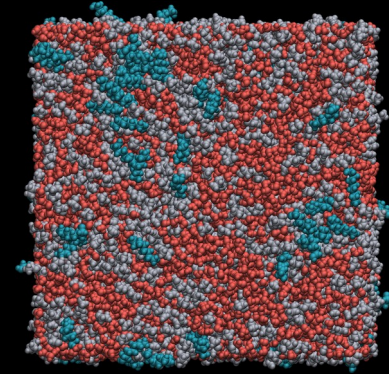
Experiment



Oil-in-water
Emulsion



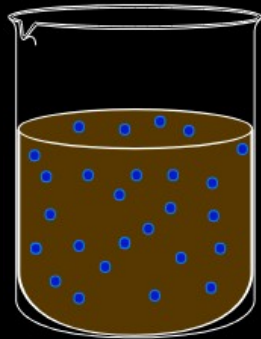
MD Simulation



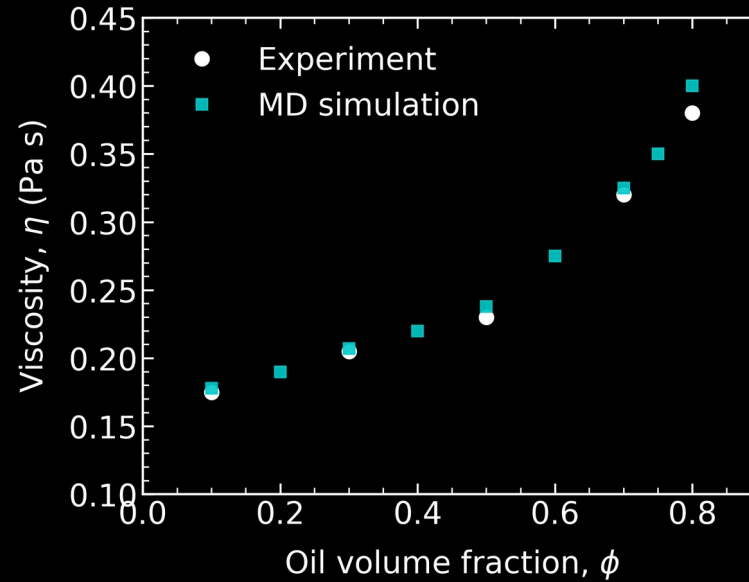
Experimental results can
be difficult to interpret

Why use molecular dynamics (MD) ?

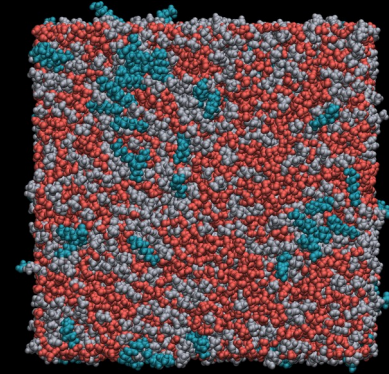
Experiment



Oil-in-water
Emulsion



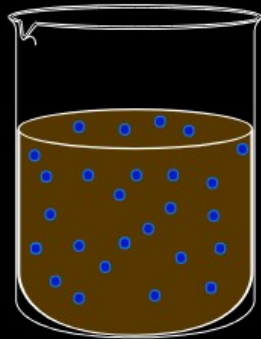
MD Simulation



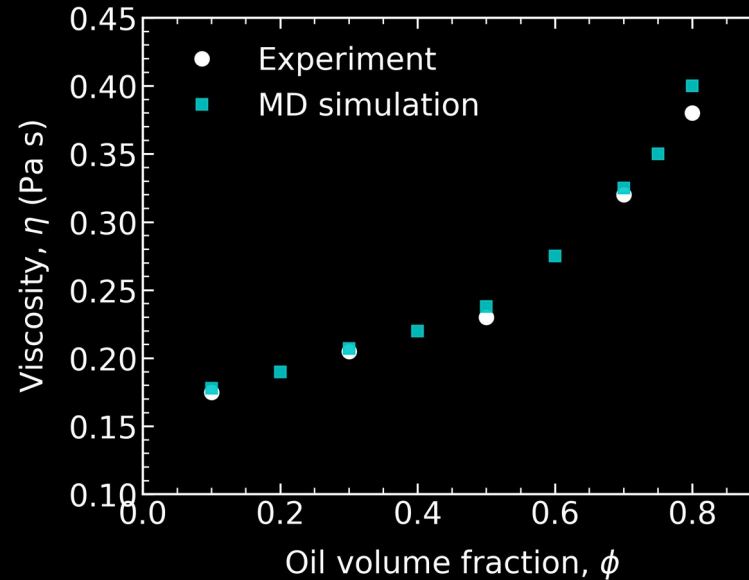
Experimental results can
be difficult to interpret

Why use molecular dynamics (MD) ?

Experiment

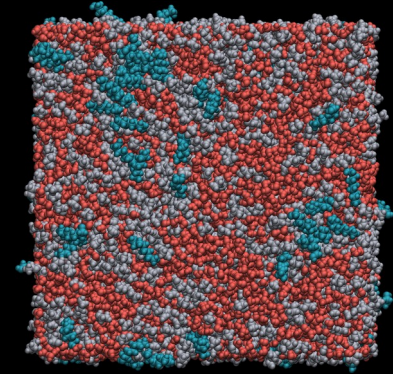


Oil-in-water
Emulsion



Experimental results can
be difficult to interpret

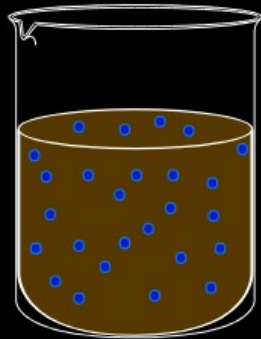
MD Simulation



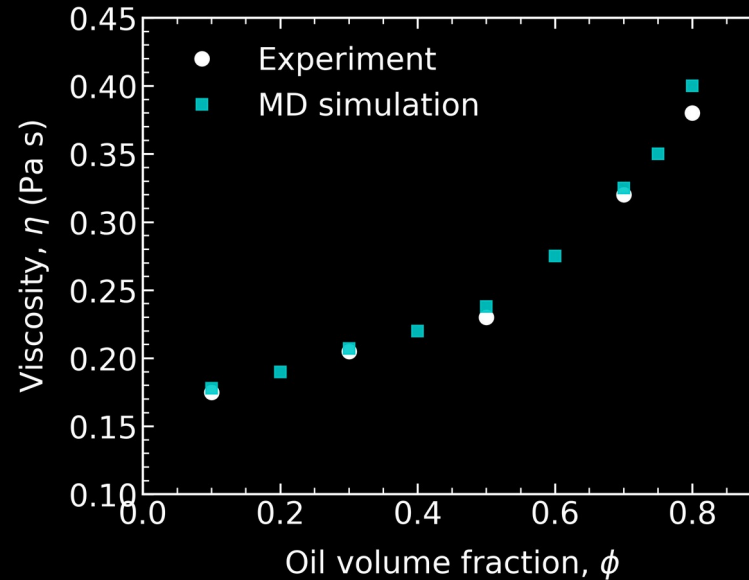
MD simulations
provide trajectories

Why use molecular dynamics (MD) ?

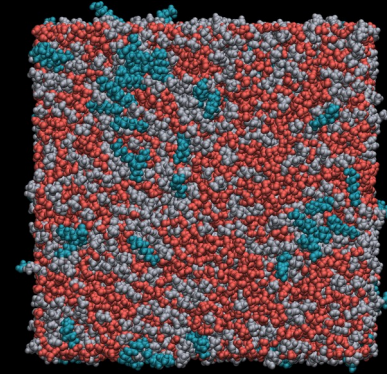
Experiment



Oil-in-water
Emulsion



MD Simulation

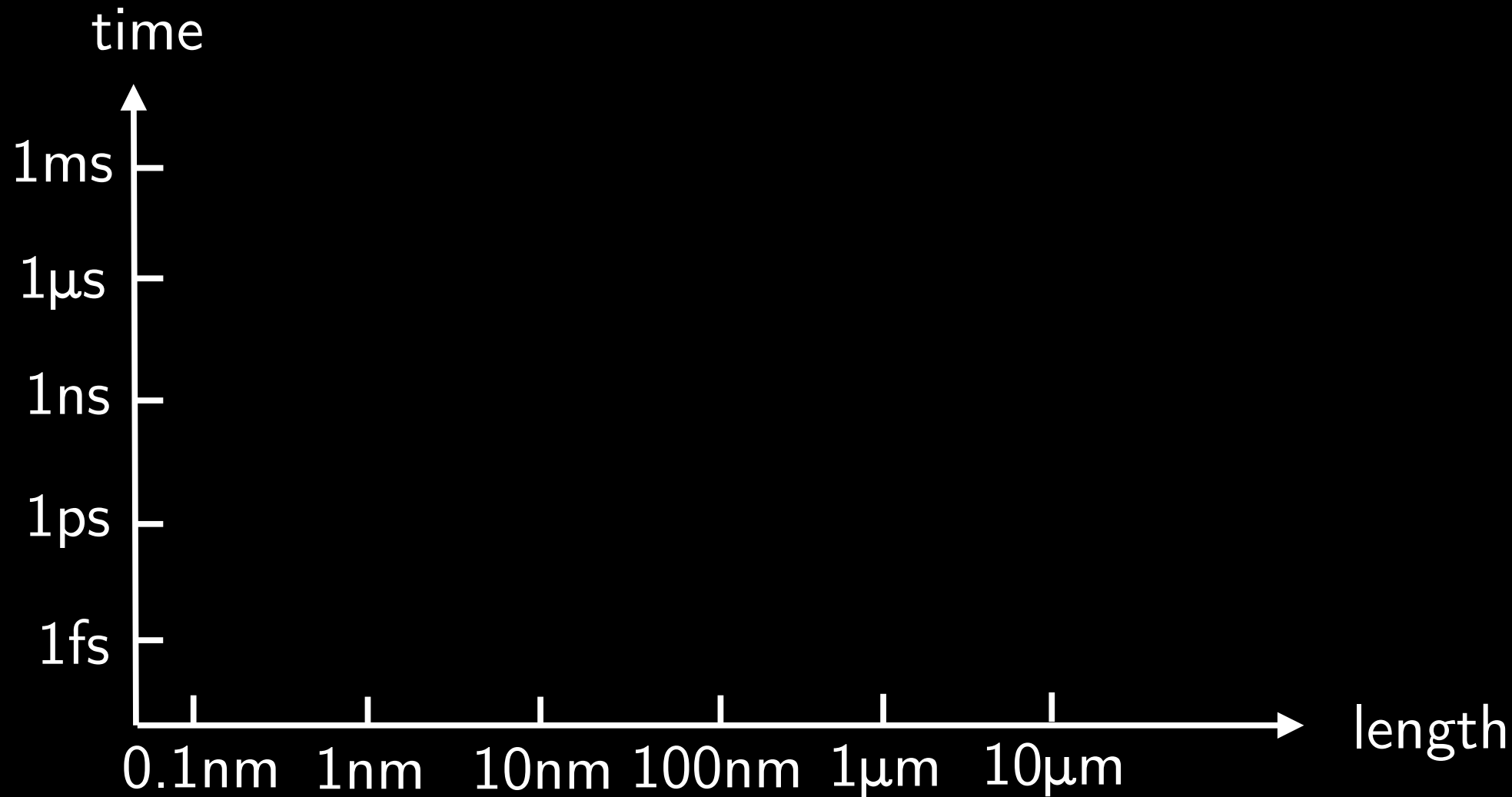


Experimental results can
be difficult to interpret

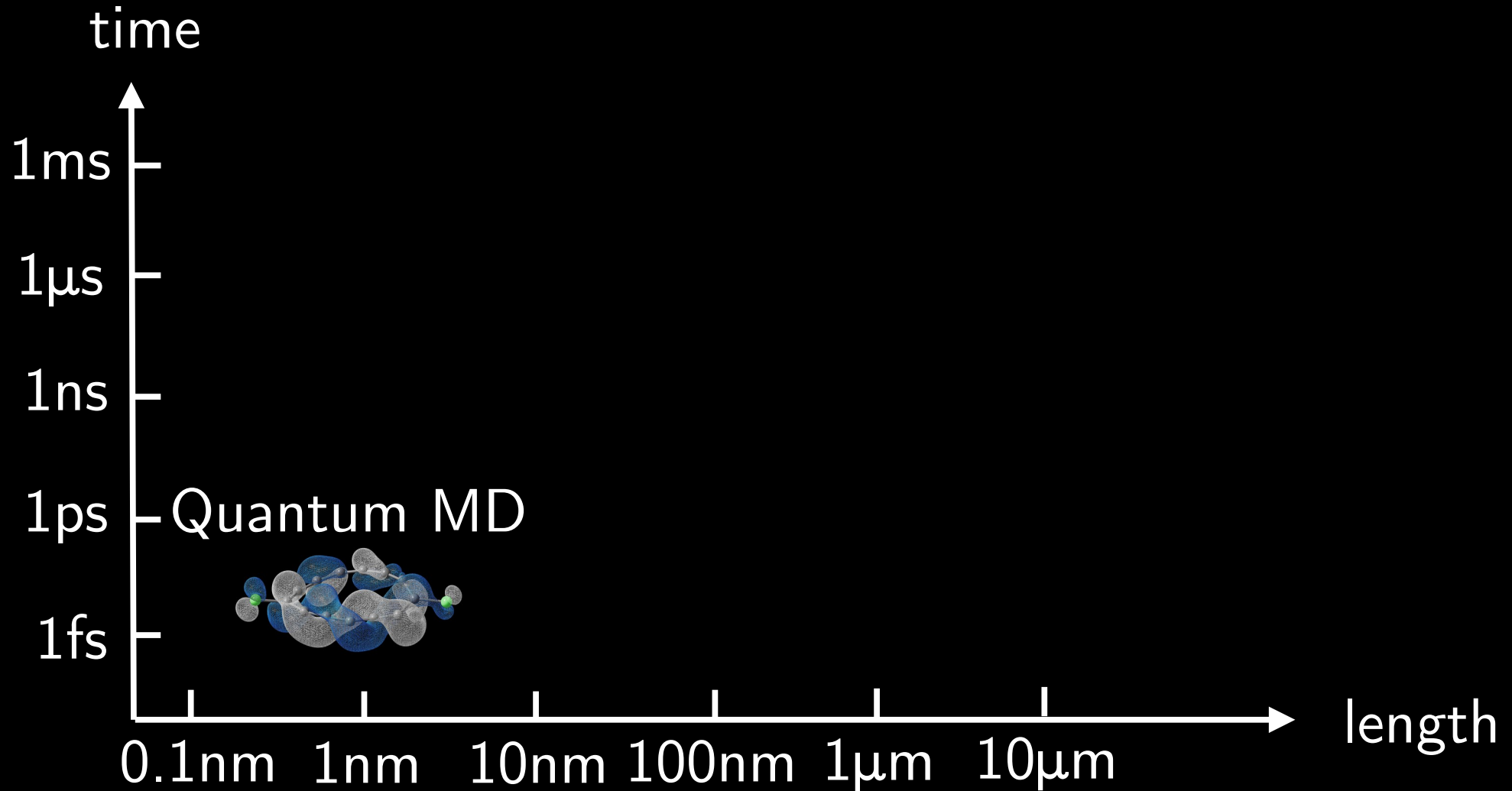
MD simulations
provide trajectories

Experiments are necessary to validate simulations.
Simulations can help interpret the results.

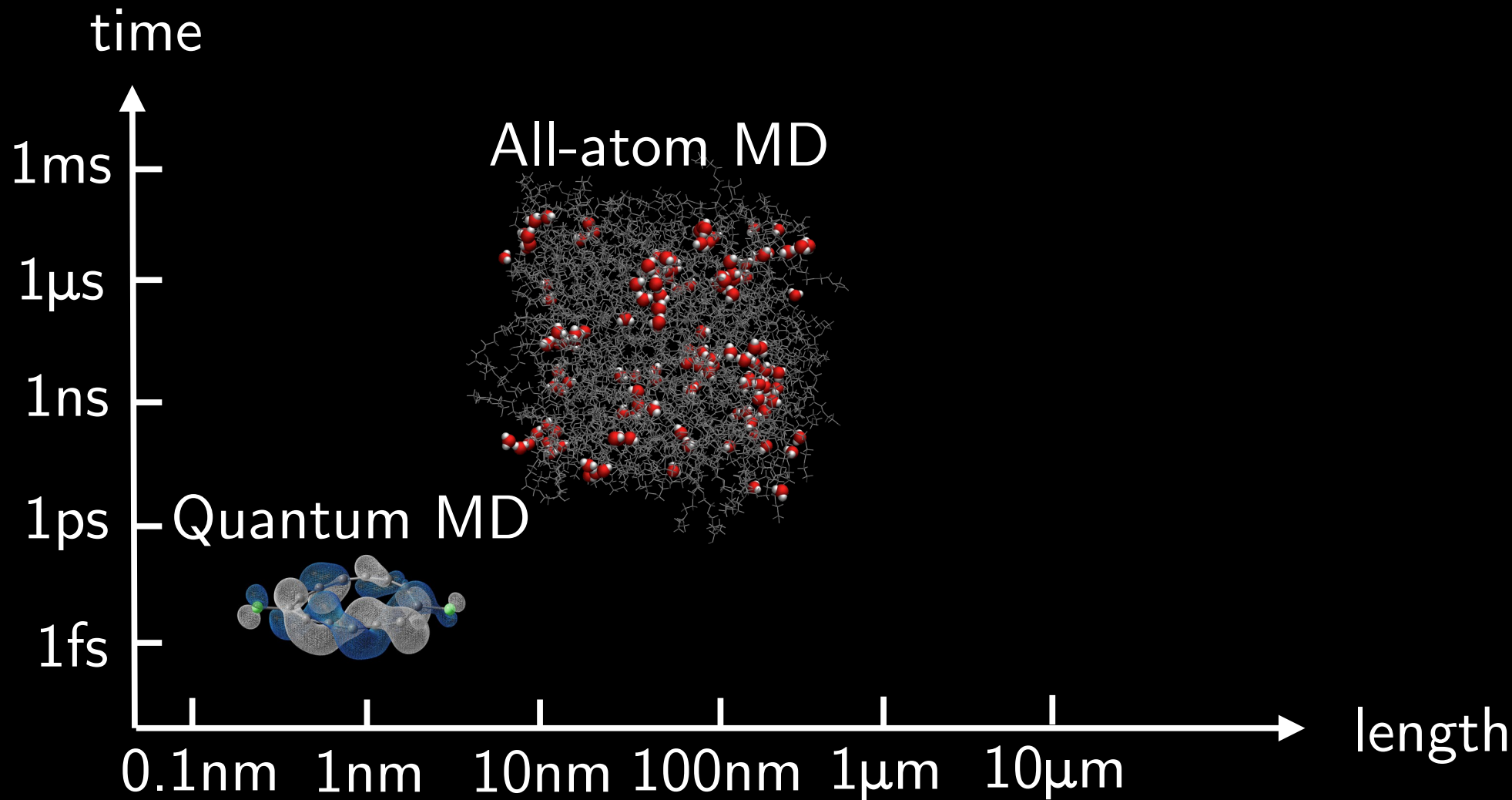
Time and length scales



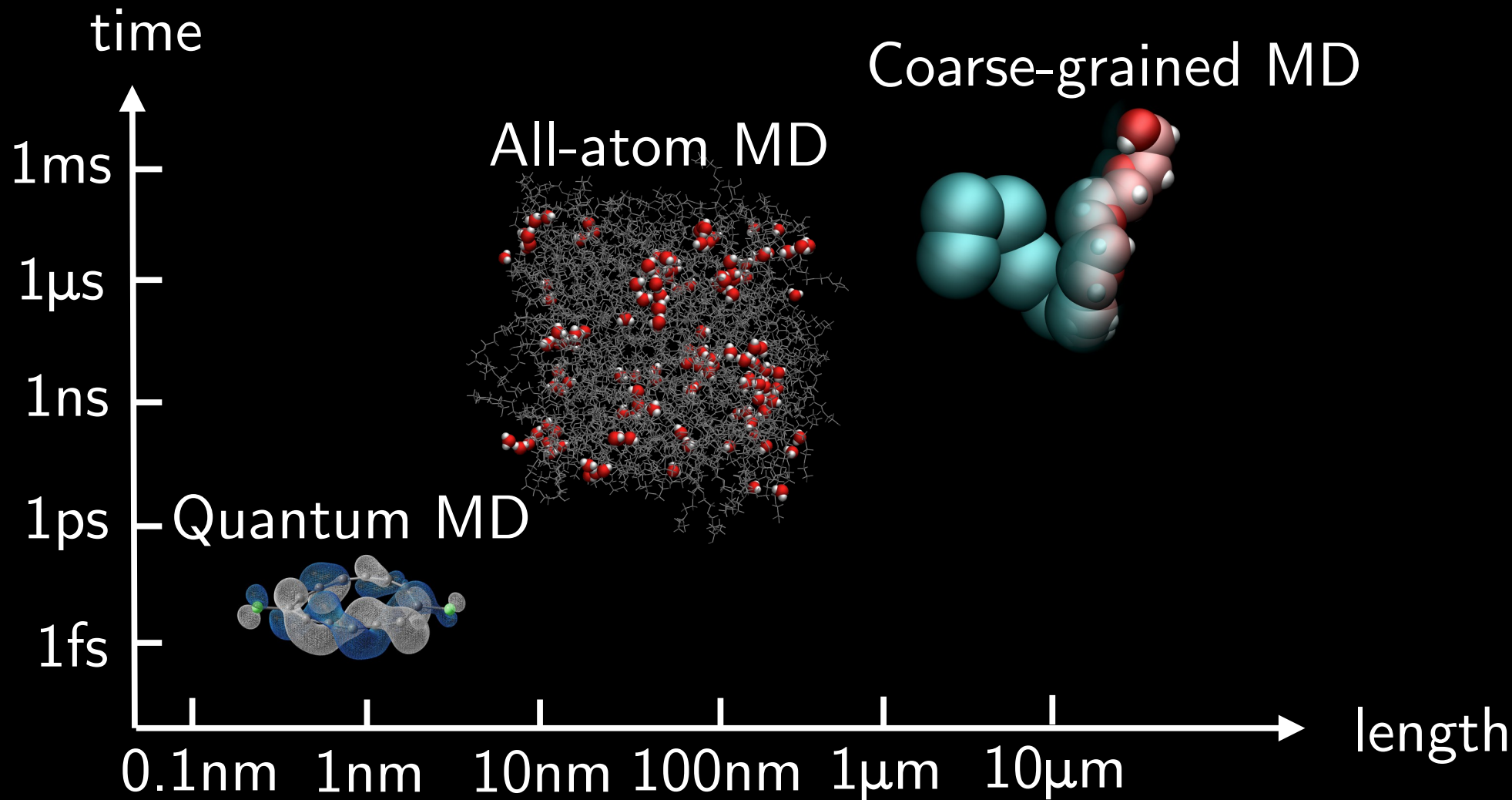
Time and length scales



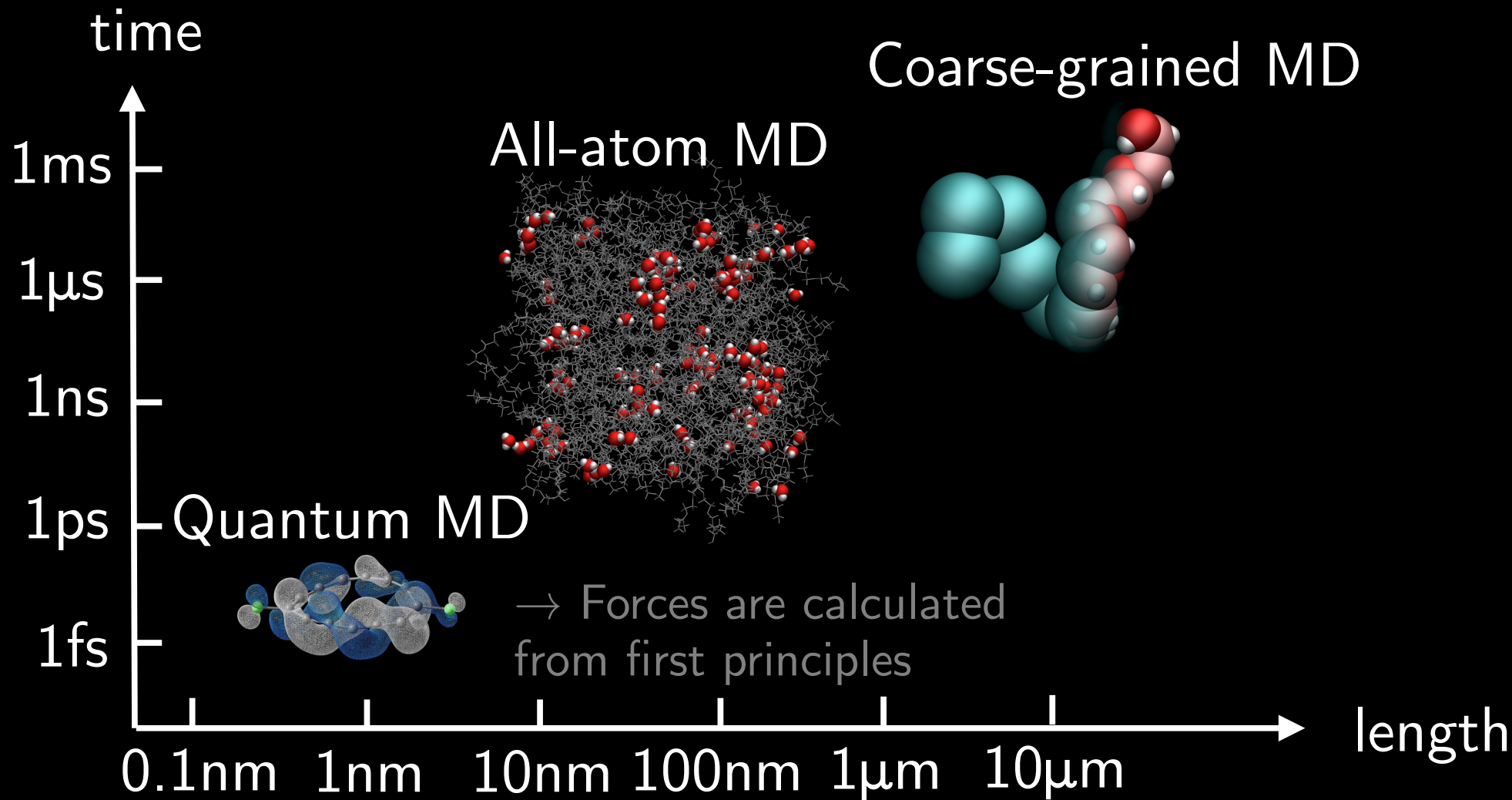
Time and length scales



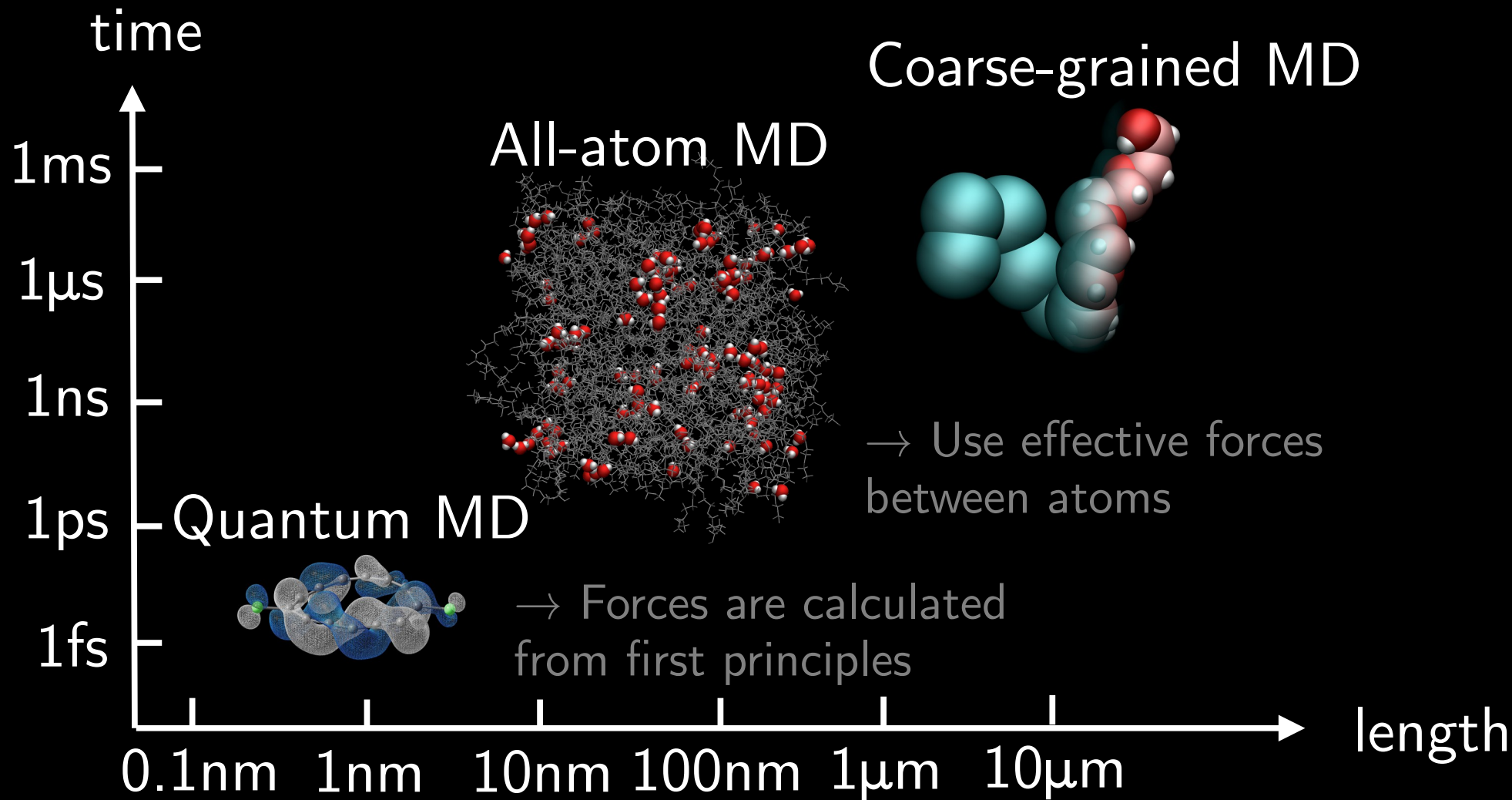
Time and length scales



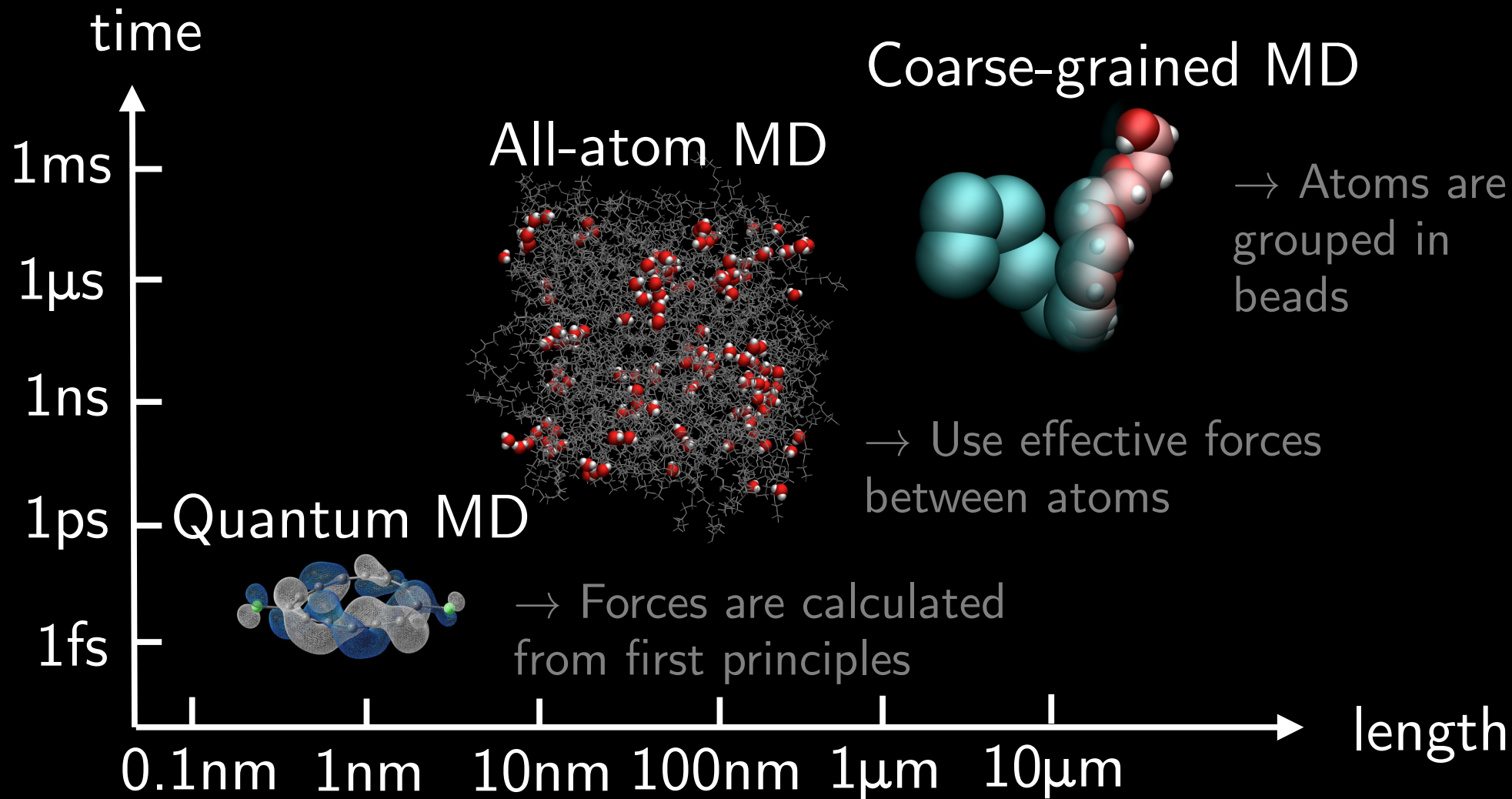
Time and length scales



Time and length scales

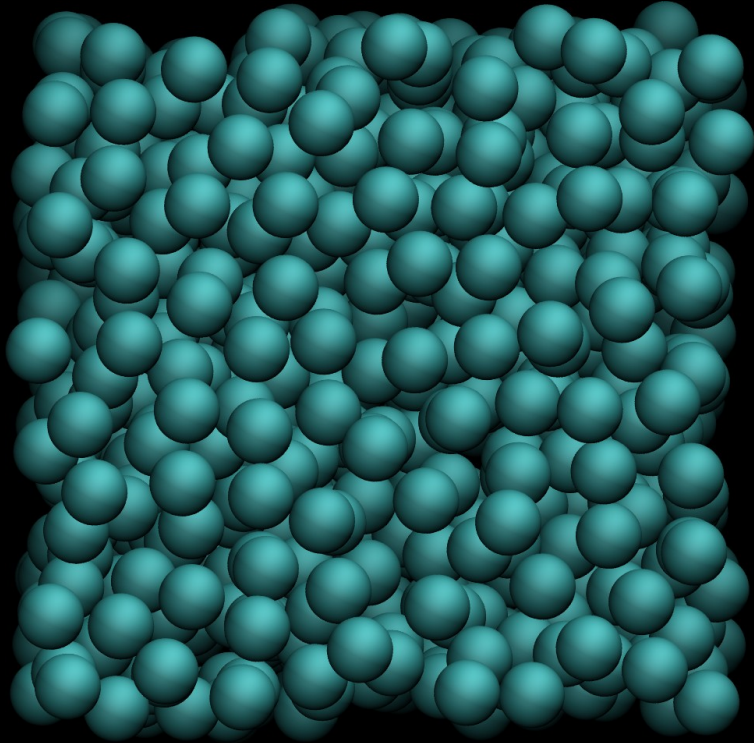


Time and length scales



Molecular dynamics

1)



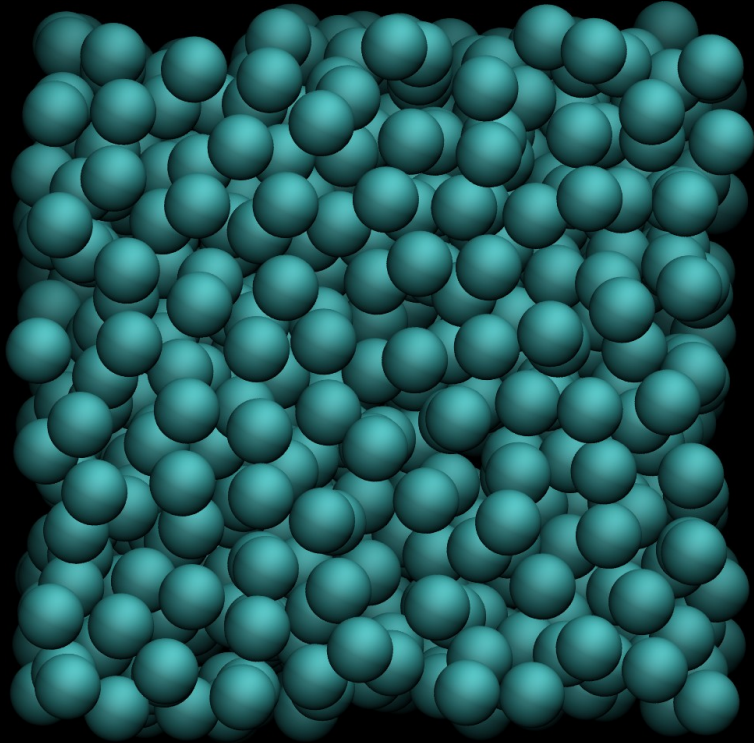
2)

3)

4)

Molecular dynamics

1) Define the system



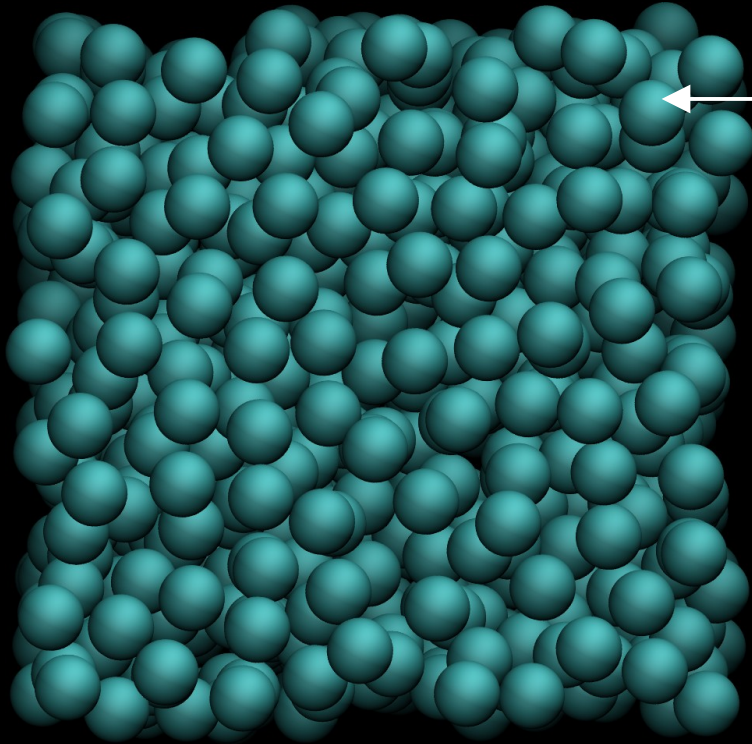
2)

3)

4)

Molecular dynamics

1) Define the system



Atom i (position $\mathbf{r}_i$, velocity $\dot{\mathbf{r}}_i$)

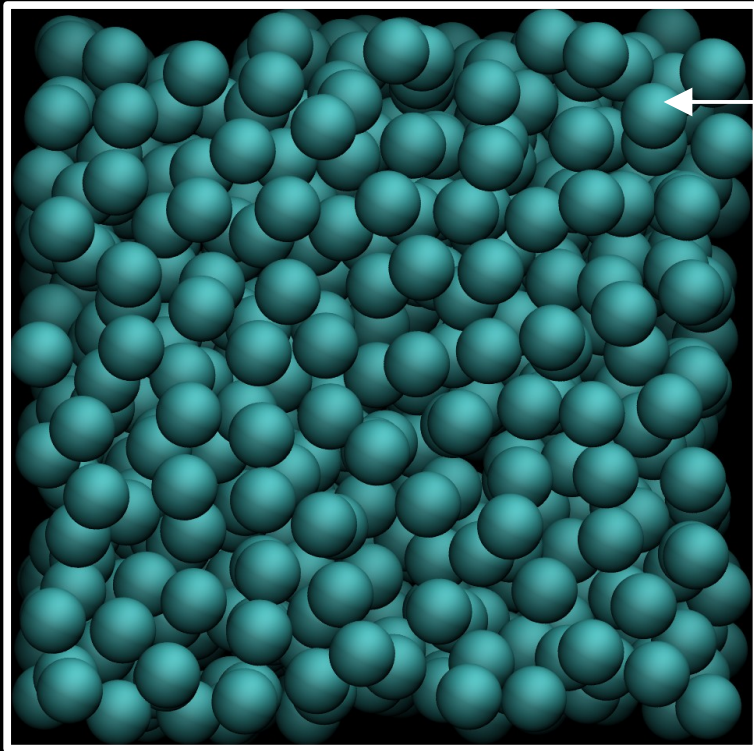
2)

3)

4)

Molecular dynamics

1) Define the system



Atom i (position $\mathbf{r}_i$, velocity $\dot{\mathbf{r}}_i$)

Box boundary (generally periodic)

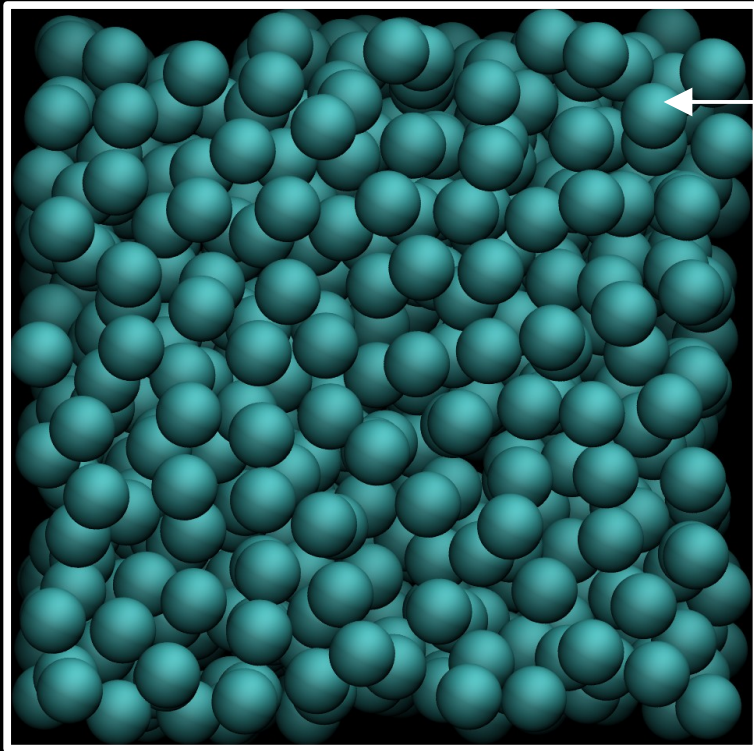
2)

3)

4)

Molecular dynamics

1) Define the system



Atom i (position $\mathbf{r}_i$, velocity $\dot{\mathbf{r}}_i$)

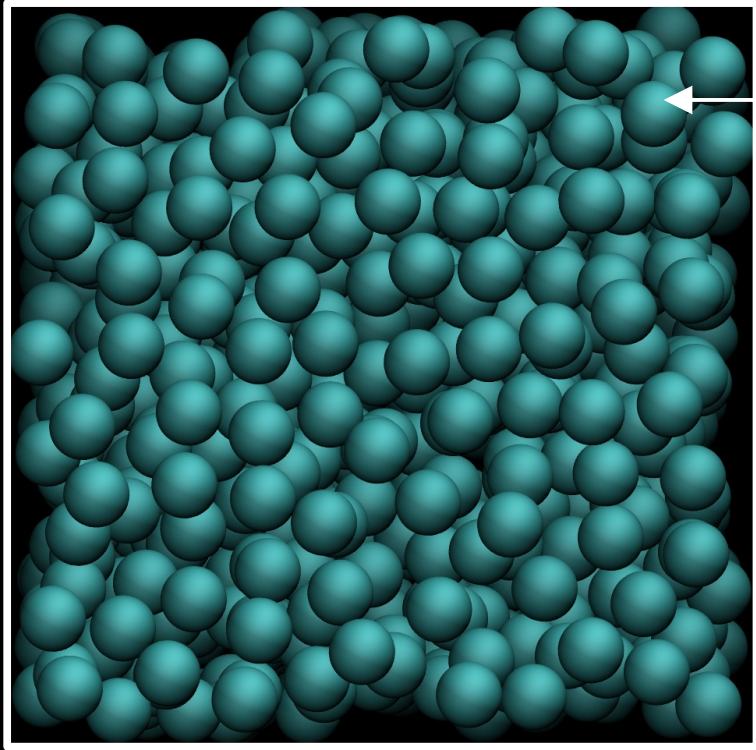
Box boundary (generally periodic)

2) Measure forces between atoms $\mathbf{F}_{ij}$

3)

4)

Molecular dynamics



1) Define the system

Atom i (position $\mathbf{r}_i$, velocity $\dot{\mathbf{r}}_i$)

Box boundary (generally periodic)

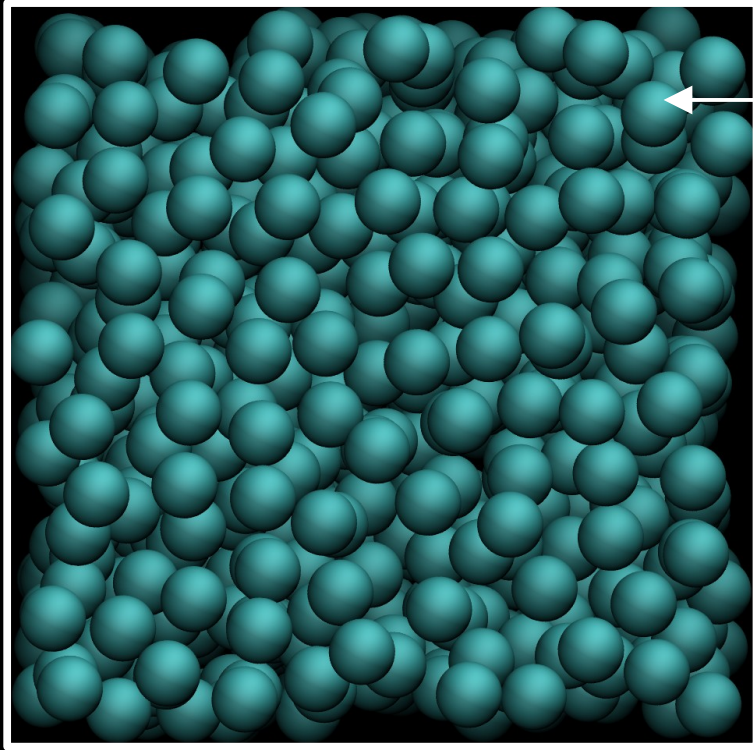
2) Measure forces between atoms $\mathbf{F}_{ij}$

3) Solve Newton's equations of motion

$$\sum_{j \neq i} \mathbf{F}_{ij} = m_i \ddot{\mathbf{r}}_i$$

4)

Molecular dynamics



1) Define the system

Atom i (position $\mathbf{r}_i$, velocity $\dot{\mathbf{r}}_i$)

Box boundary (generally periodic)

2) Measure forces between atoms $\mathbf{F}_{ij}$

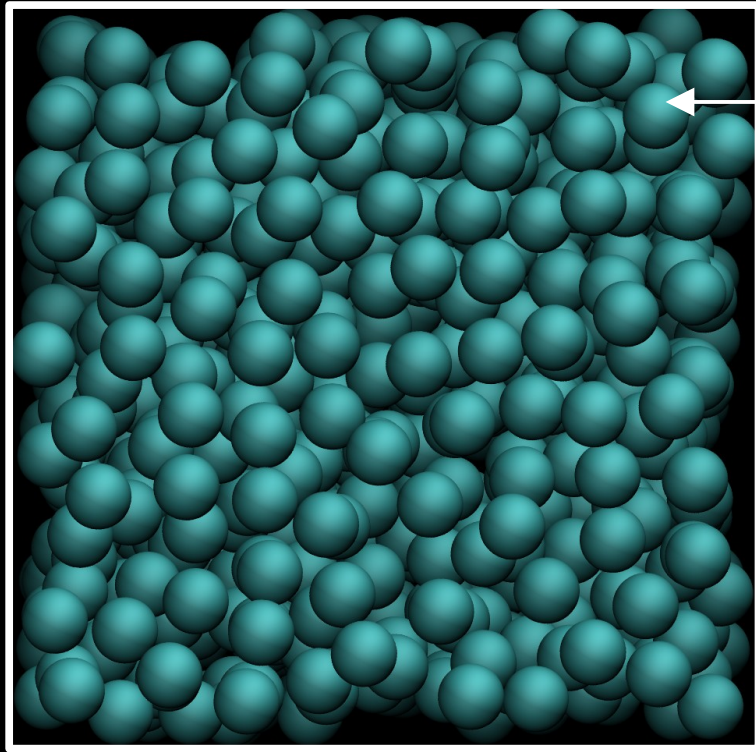
3) Solve Newton's equations of motion

$$\sum_{j \neq i} \mathbf{F}_{ij} = m_i \ddot{\mathbf{r}}_i$$

4) Move atoms

Molecular dynamics

1) Define the system



Atom i (position $\mathbf{r}_i$, velocity $\dot{\mathbf{r}}_i$)

Box boundary (generally periodic)

2) Measure forces between atoms $\mathbf{F}_{ij}$

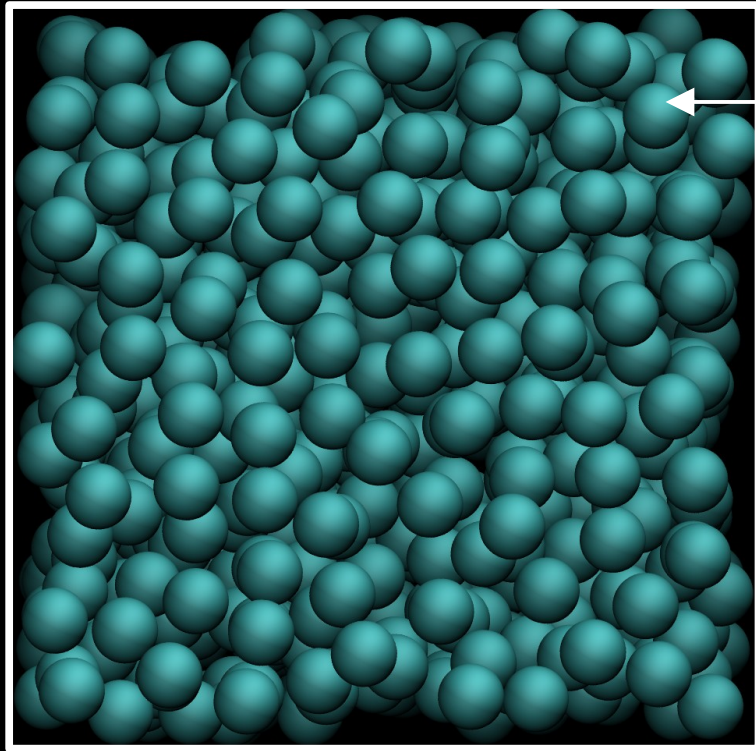
3) Solve Newton's equations of motion

$$\sum_{j \neq i} \mathbf{F}_{ij} = m_i \ddot{\mathbf{r}}_i$$

4) Move atoms

Molecular dynamics

1) Define the system



Atom i (position $\mathbf{r}_i$, velocity $\dot{\mathbf{r}}_i$)

Box boundary (generally periodic)

2) Measure forces between atoms $\mathbf{F}_{ij}$

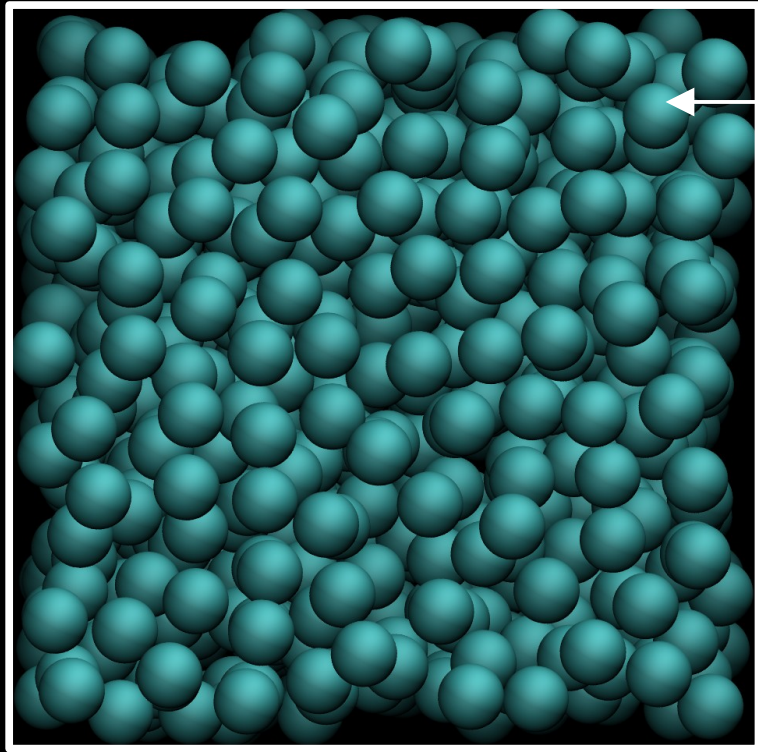
3) Solve Newton's equations of motion

$$\sum_{j \neq i} \mathbf{F}_{ij} = m_i \ddot{\mathbf{r}}_i$$

4) Move atoms

Molecular dynamics

1) Define the system



Atom i (position $\mathbf{r}_i$, velocity $\dot{\mathbf{r}}_i$)

Box boundary (generally periodic)

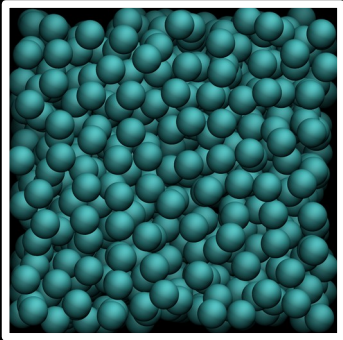
2) Measure forces between atoms $\mathbf{F}_{ij}$

3) Solve Newton's equations of motion

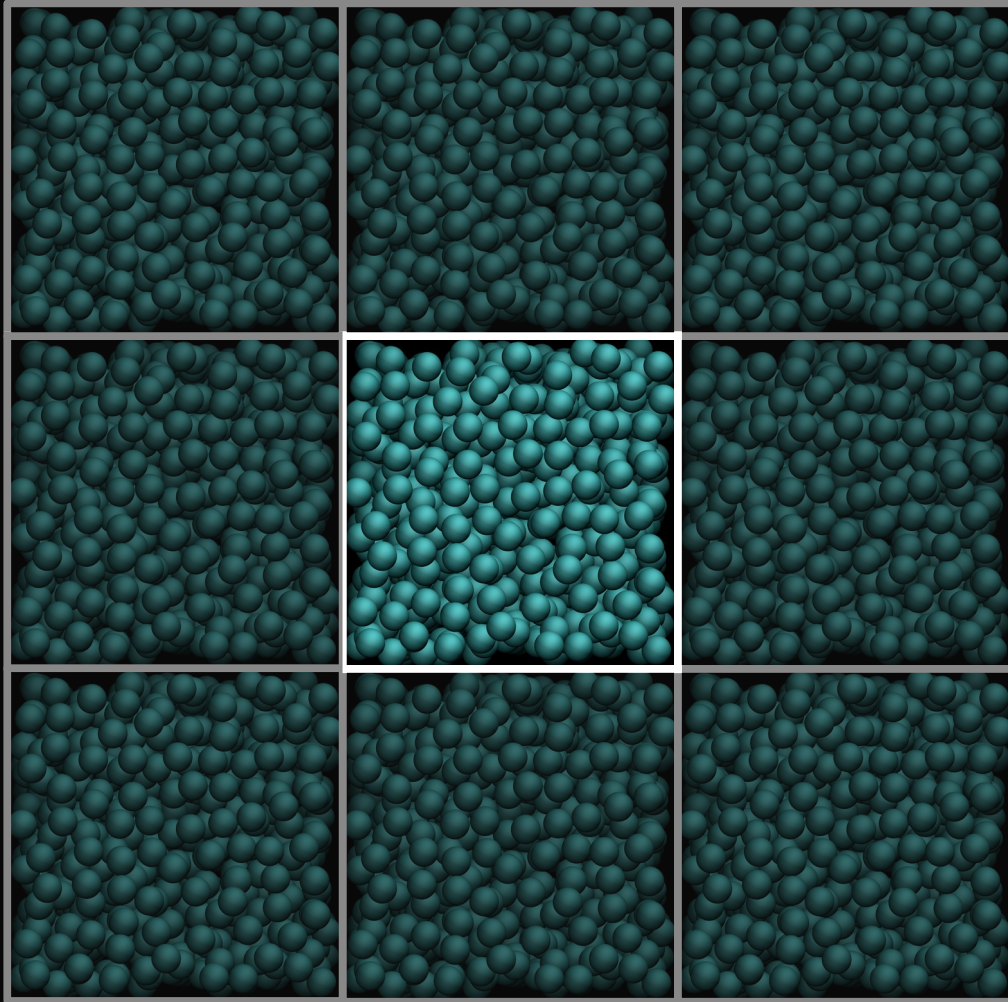
$$\sum_{j \neq i} \mathbf{F}_{ij} = m_i \ddot{\mathbf{r}}_i$$

4) Move atoms

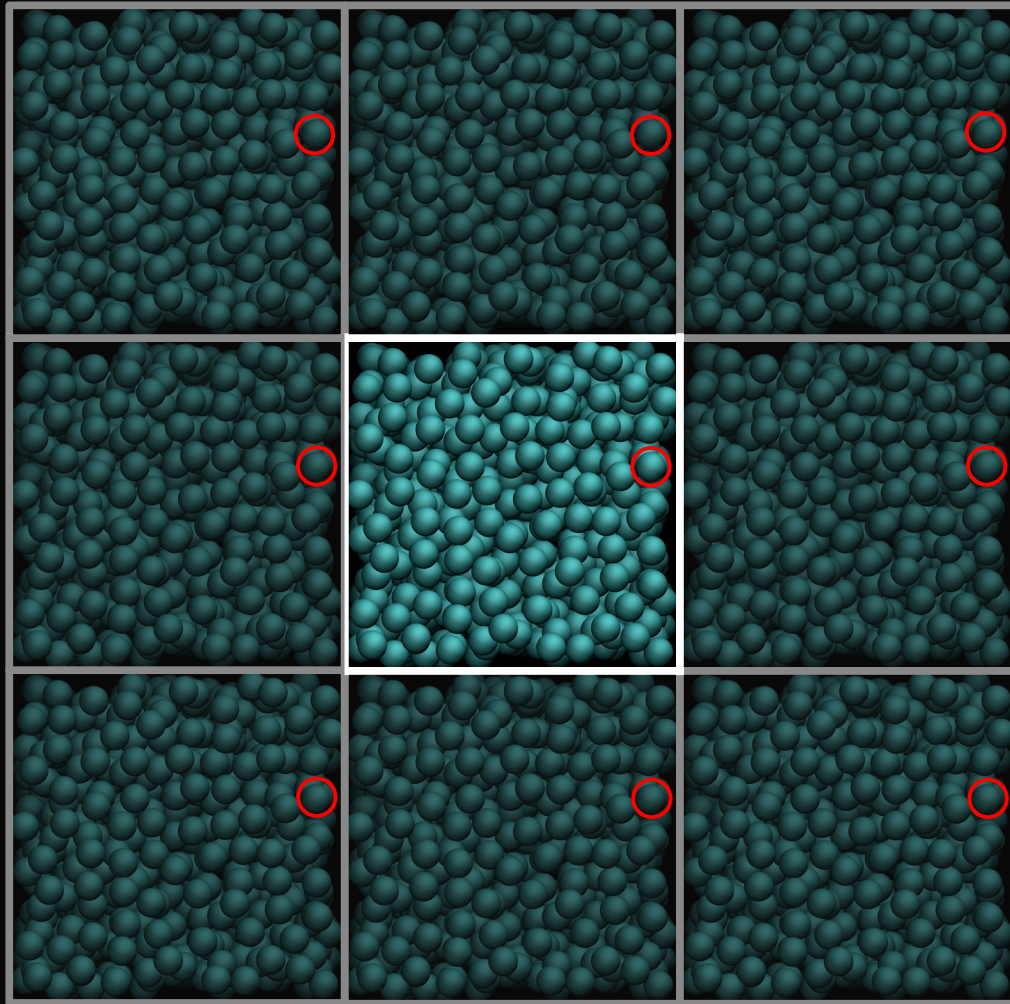
Periodic boundary conditions



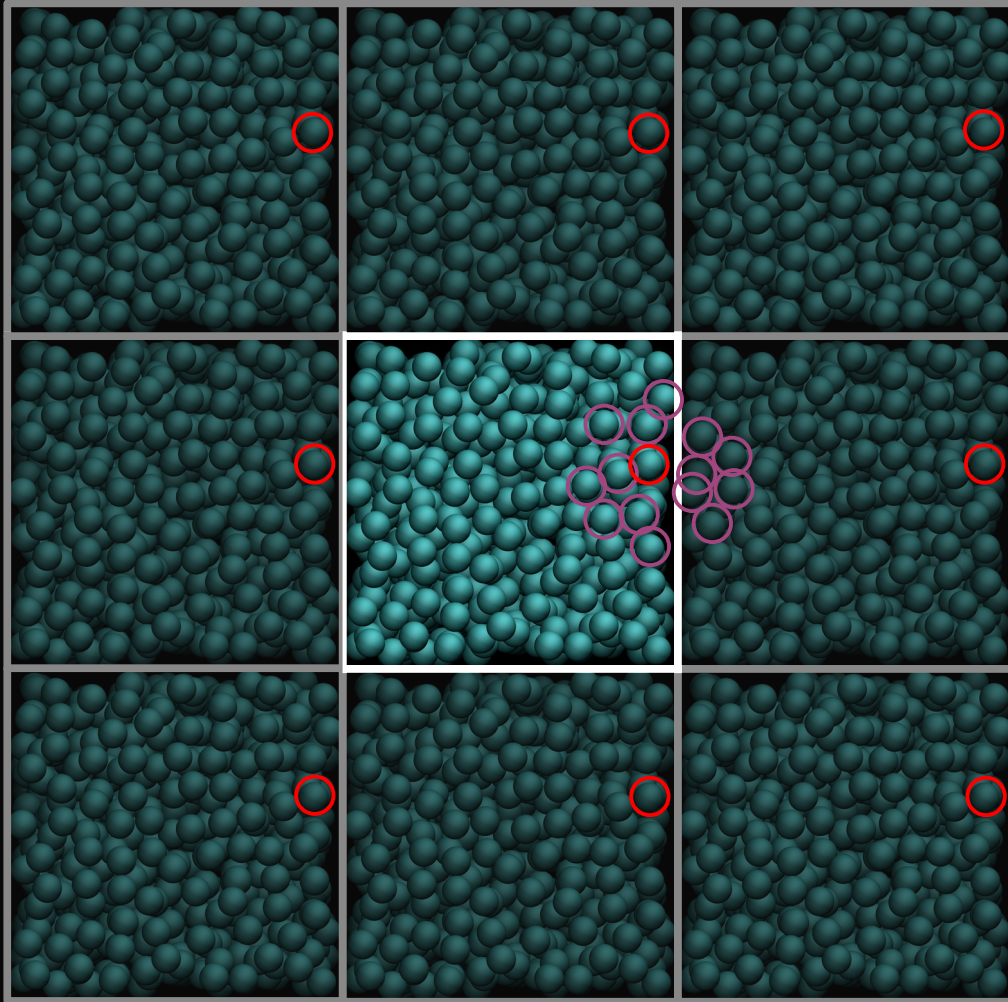
Periodic boundary conditions



Periodic boundary conditions



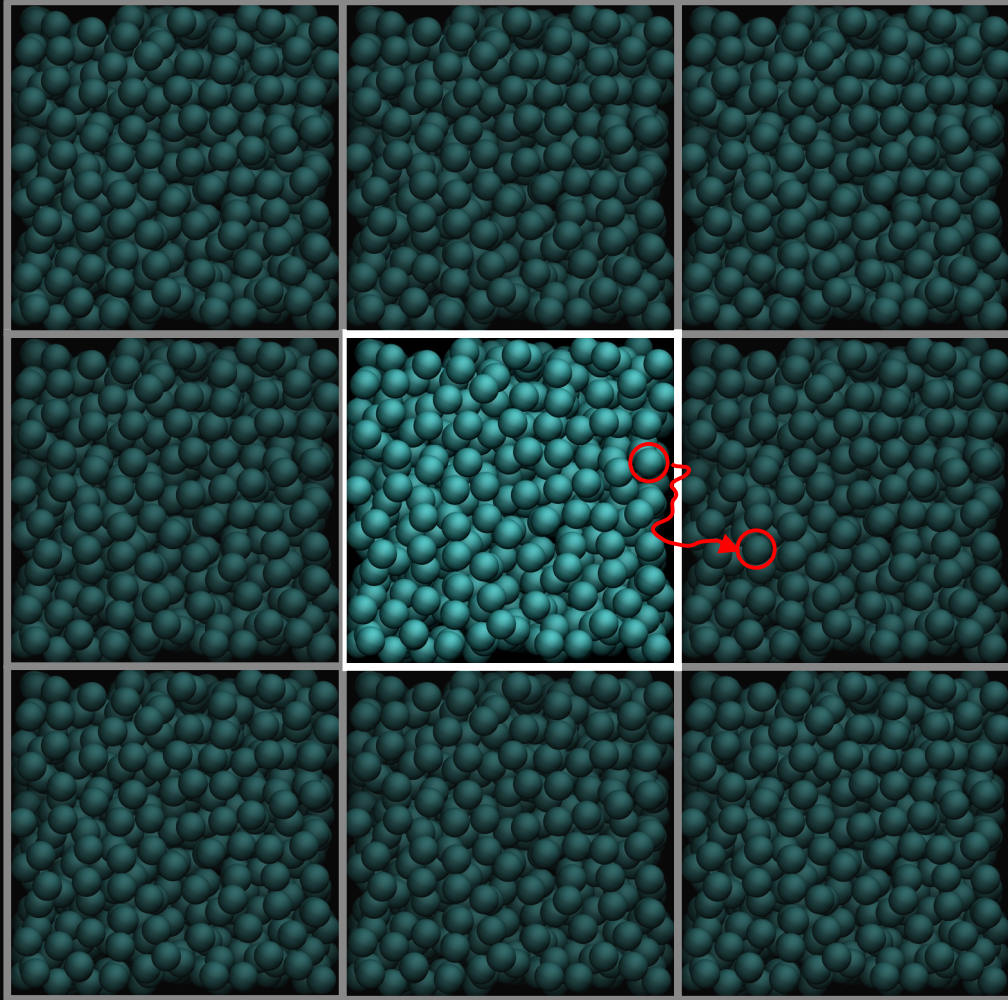
Periodic boundary conditions



Atoms:

- interact through the
boundaries

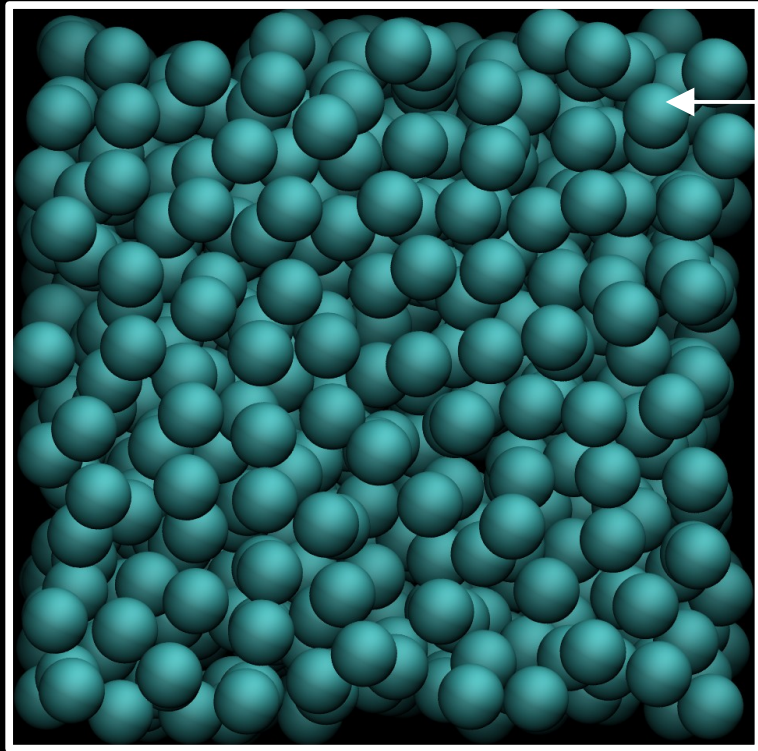
Periodic boundary conditions



Atoms:

- interact through the boundaries
- can cross the boundaries

Molecular dynamics



1) Define the system

Atom i (position $\mathbf{r}_i$, velocity $\dot{\mathbf{r}}_i$)

Box boundary (generally periodic)

2) Measure forces between atoms $\mathbf{F}_{ij}$

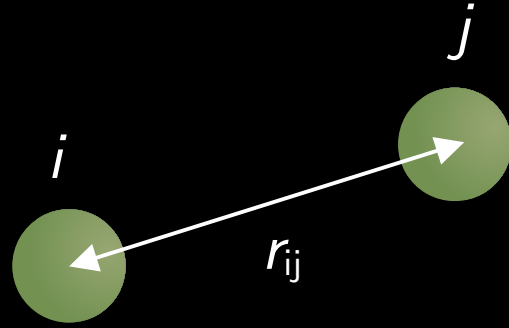
3) Solve Newton's equations of motion

$$\sum_{j \neq i} \mathbf{F}_{ij} = m_i \ddot{\mathbf{r}}_i$$

4) Move atoms

Forces between atoms

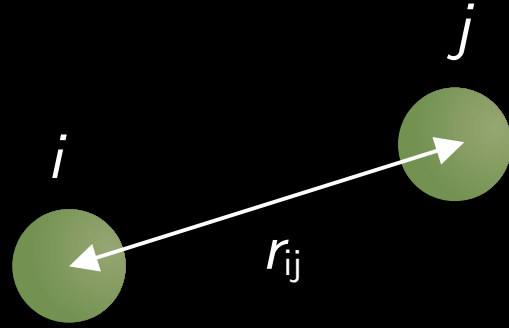
Conventional potential : Lennard Jones



Forces between atoms

Conventional potential : Lennard Jones

$$F_{ij} = -\frac{dU(r_{ij})}{dr_{ij}}$$

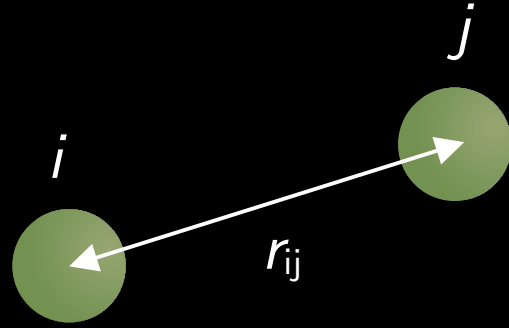


Forces between atoms

Conventional potential : Lennard Jones

$$F_{ij} = -\frac{dU(r_{ij})}{dr_{ij}}$$

$$U_{ij}(r_{ij}) = 4\epsilon_{ij} \left[\left(\frac{\sigma_{ij}}{r_{ij}} \right)^{12} - \left(\frac{\sigma_{ij}}{r_{ij}} \right)^6 \right]$$

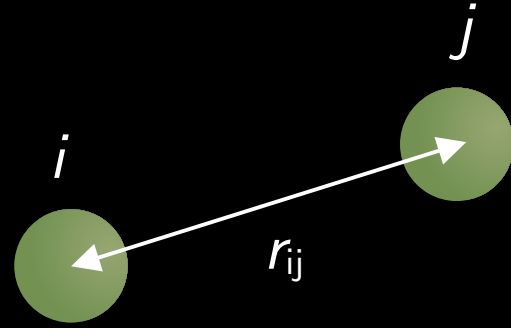


Forces between atoms

Conventional potential : Lennard Jones

$$F_{ij} = -\frac{dU(r_{ij})}{dr_{ij}}$$

$$U_{ij}(r_{ij}) = 4\epsilon_{ij} \left[\left(\frac{\sigma_{ij}}{r_{ij}} \right)^{12} - \left(\frac{\sigma_{ij}}{r_{ij}} \right)^6 \right]$$



σ_{ij} : effective particle size

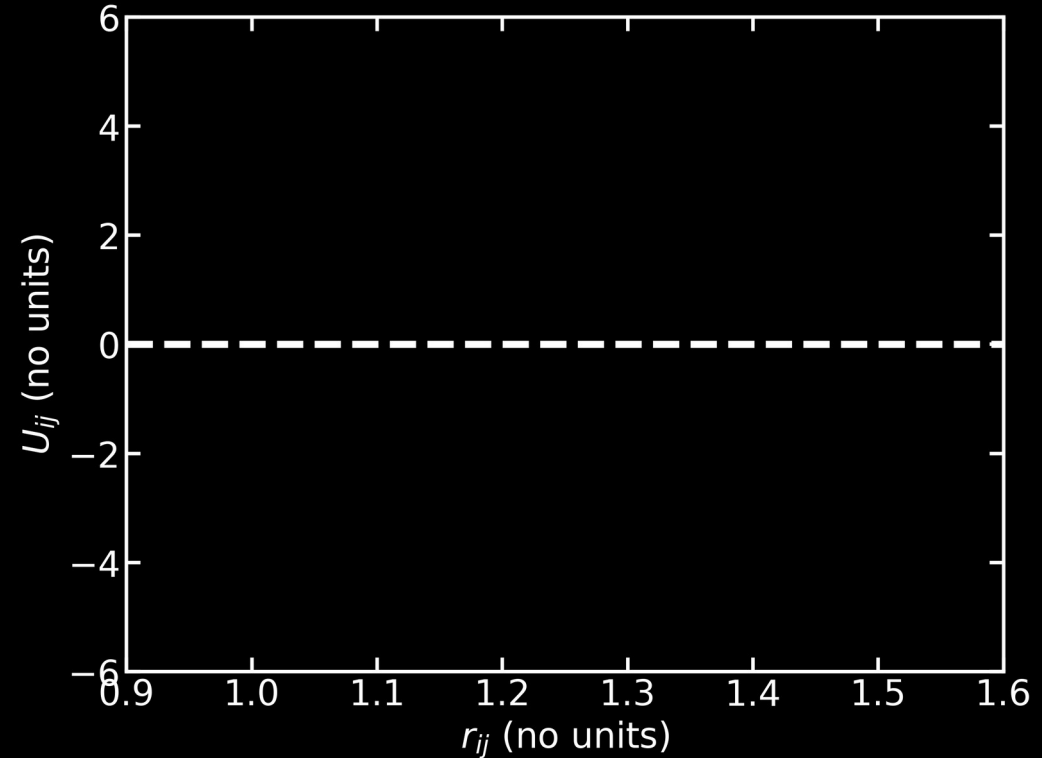
ϵ_{ij} : interaction strength

Forces between atoms

Conventional potential : Lennard Jones

$$F_{ij} = -\frac{dU(r_{ij})}{dr_{ij}}$$

$$U_{ij}(r_{ij}) = 4\epsilon_{ij} \left[\left(\frac{\sigma_{ij}}{r_{ij}} \right)^{12} - \left(\frac{\sigma_{ij}}{r_{ij}} \right)^6 \right]$$



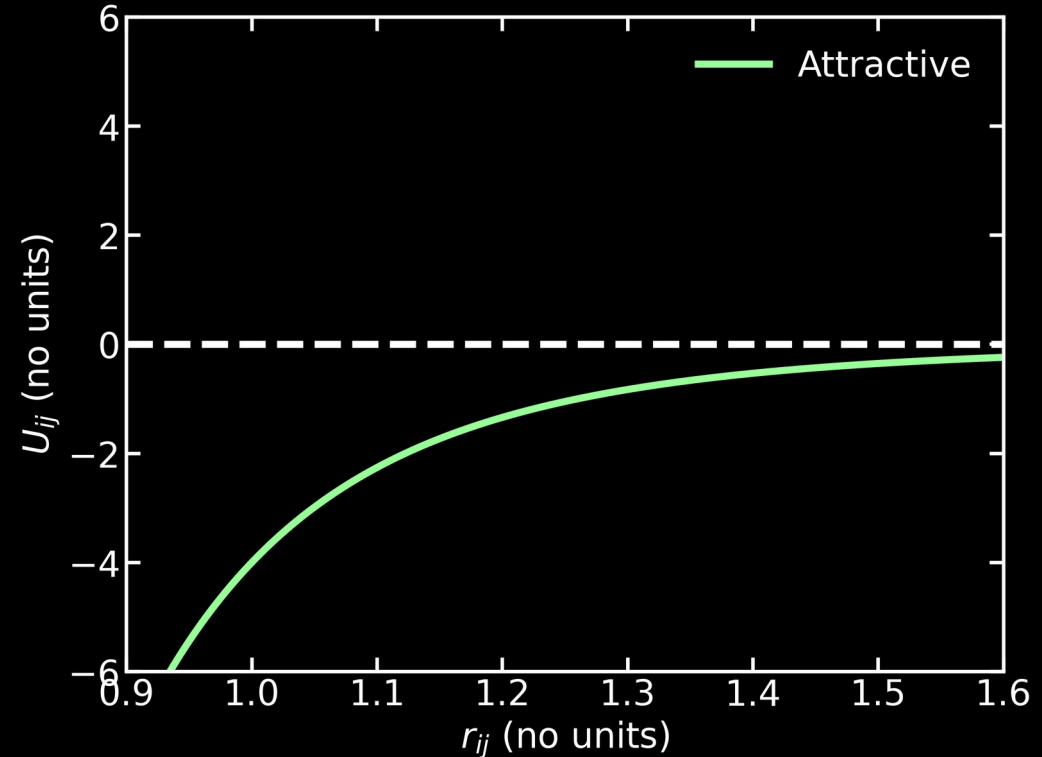
Forces between atoms

Conventional potential : Lennard Jones

$$F_{ij} = -\frac{dU(r_{ij})}{dr_{ij}}$$

$$U_{ij}(r_{ij}) = 4\epsilon_{ij} \left[\left(\frac{\sigma_{ij}}{r_{ij}} \right)^{12} - \left(\frac{\sigma_{ij}}{r_{ij}} \right)^6 \right]$$

Attractive (London dispersion)



Forces between atoms

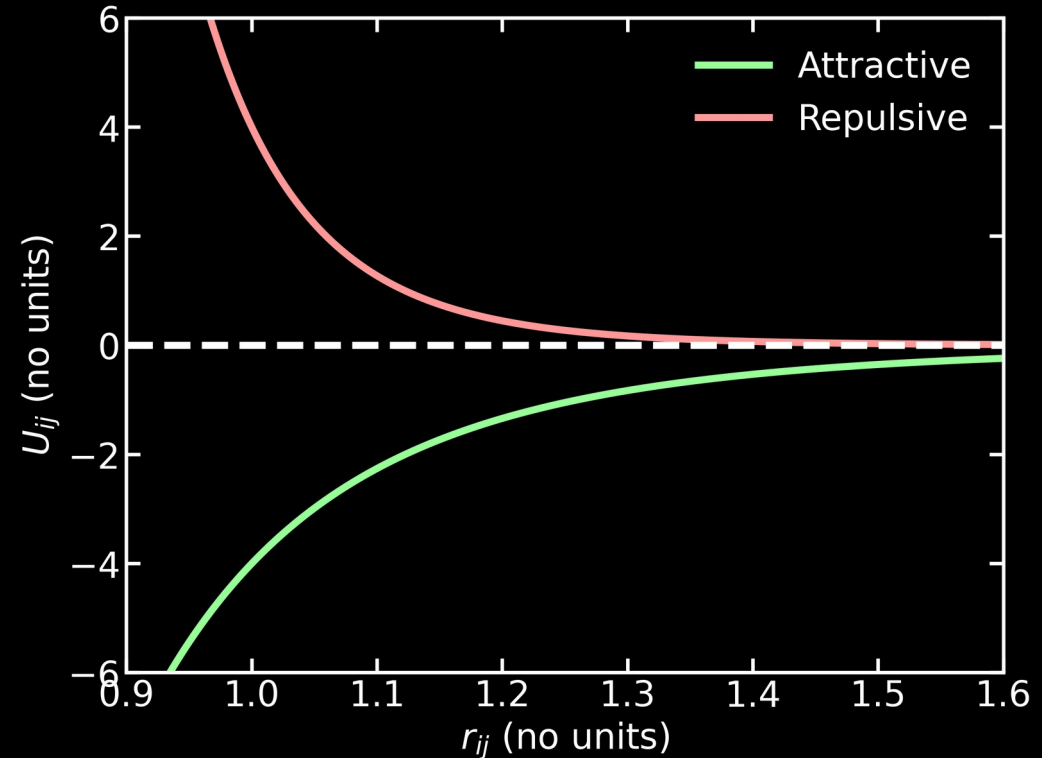
Conventional potential : Lennard Jones

$$F_{ij} = -\frac{dU(r_{ij})}{dr_{ij}}$$

$$U_{ij}(r_{ij}) = 4\epsilon_{ij} \left[\left(\frac{\sigma_{ij}}{r_{ij}} \right)^{12} - \left(\frac{\sigma_{ij}}{r_{ij}} \right)^6 \right]$$

Repulsive (Pauli repulsion)

Attractive (London dispersion)



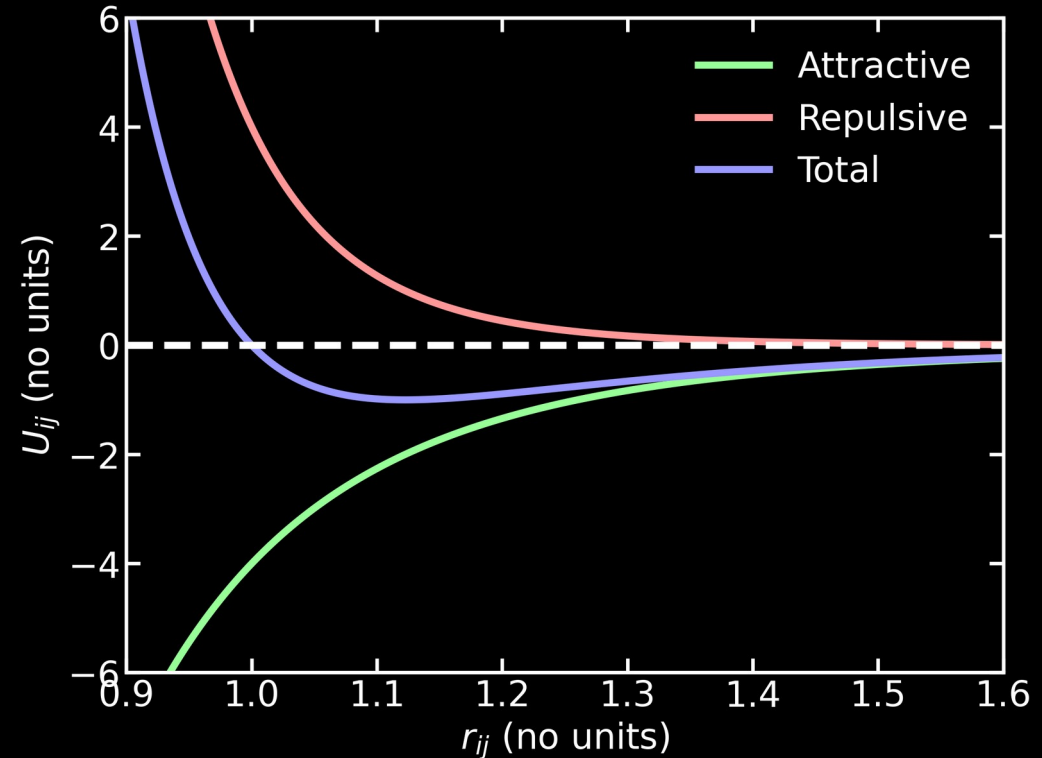
Forces between atoms

Conventional potential : Lennard Jones

$$F_{ij} = -\frac{dU(r_{ij})}{dr_{ij}}$$
$$U_{ij}(r_{ij}) = 4\epsilon_{ij} \left[\left(\frac{\sigma_{ij}}{r_{ij}} \right)^{12} - \left(\frac{\sigma_{ij}}{r_{ij}} \right)^6 \right]$$

Repulsive (Pauli repulsion)

Attractive (London dispersion)



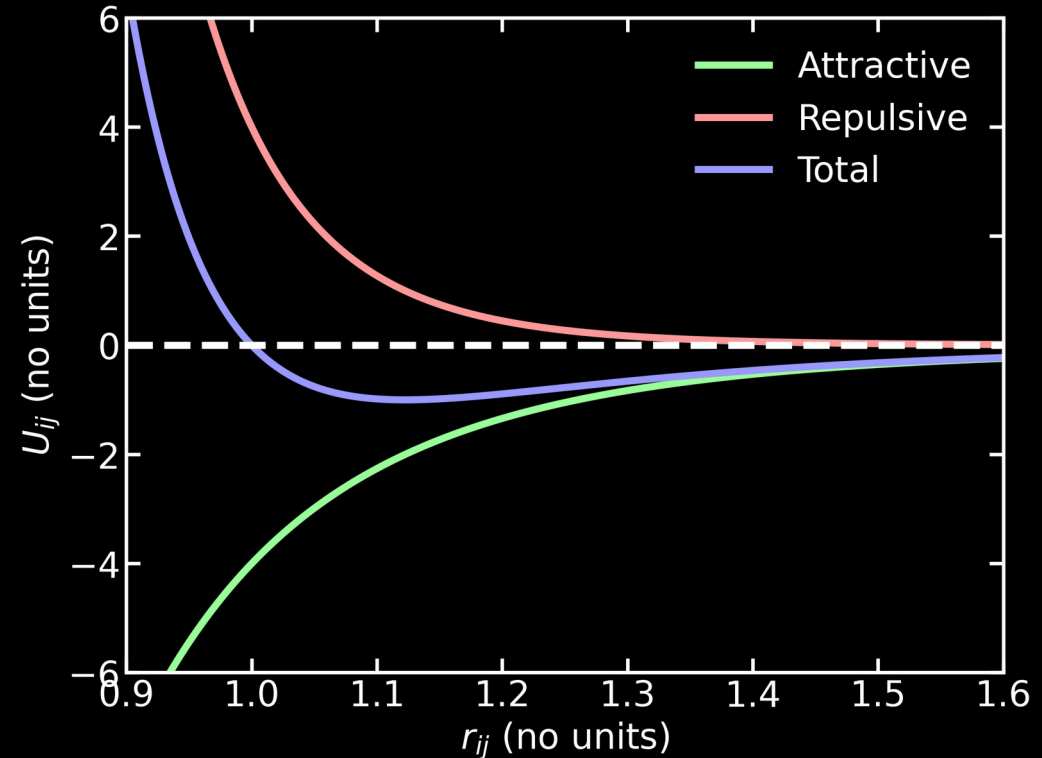
Forces between atoms

Conventional potential : Lennard Jones

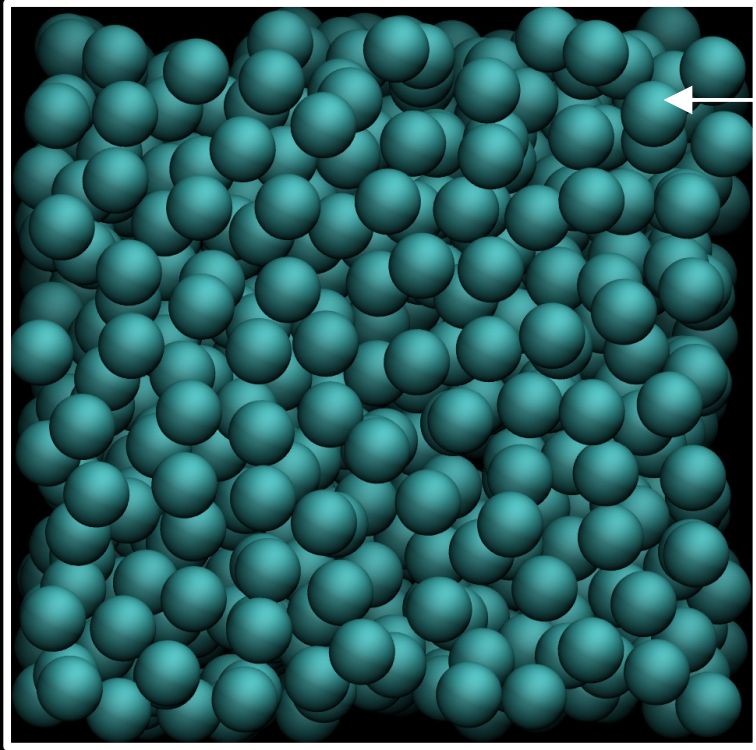
$$F_{ij} = -\frac{dU(r_{ij})}{dr_{ij}}$$

$$U_{ij}(r_{ij}) = 4\epsilon_{ij} \left[\left(\frac{\sigma_{ij}}{r_{ij}} \right)^{12} - \left(\frac{\sigma_{ij}}{r_{ij}} \right)^6 \right]$$

Generally only calculated for atoms within a certain distance



Molecular dynamics



1) Define the system

Atom i (position $\mathbf{r}_i$, velocity $\dot{\mathbf{r}}_i$)

Box boundary (generally periodic)

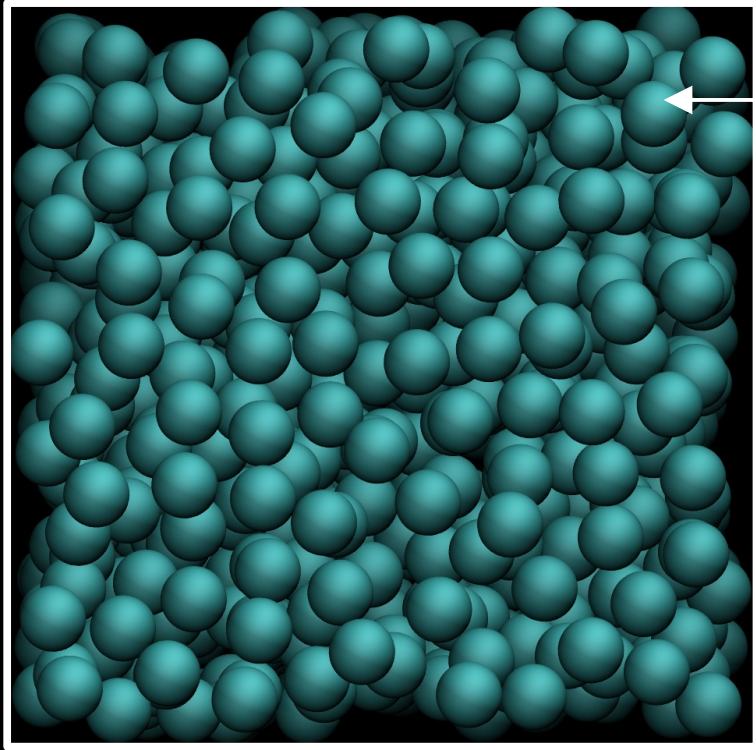
2) Measure forces between atoms $\mathbf{F}_{ij}$

3) Solve Newton's equations of motion

$$\sum_{j \neq i} \mathbf{F}_{ij} = m_i \ddot{\mathbf{r}}_i$$

4) Move atoms

Molecular dynamics



1) Define the system

Atom i (position $\mathbf{r}_i$, velocity $\dot{\mathbf{r}}_i$)

Box boundary (generally periodic)

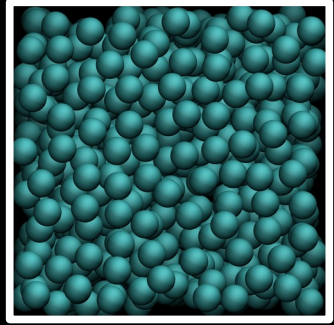
2) Measure forces between atoms $\mathbf{F}_{ij}$

3) Solve Newton's equations of motion

$$\sum_{j \neq i} \mathbf{F}_{ij} = m_i \ddot{\mathbf{r}}_i$$

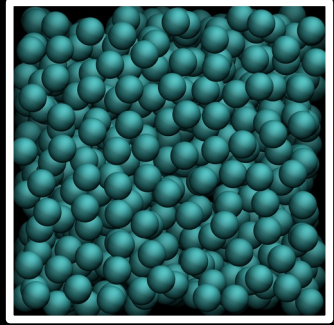
4) Move atoms

Solving Newton's equation



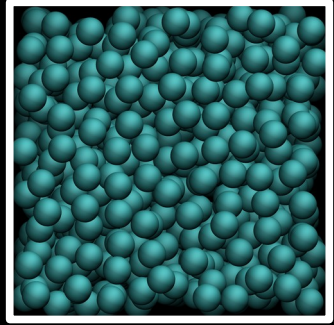
$$\sum_{j \neq i} \mathbf{F}_{ij} = m_i \ddot{\mathbf{r}}_i$$

Solving Newton's equation



$$\sum_{j \neq i} \mathbf{F}_{ij} = m_i \ddot{\mathbf{r}}_i \rightarrow \text{Continuous differential equation}$$

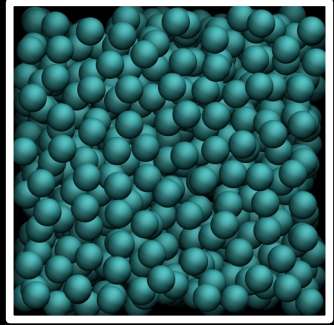
Solving Newton's equation



$$\sum_{j \neq i} \mathbf{F}_{ij} = m_i \ddot{\mathbf{r}}_i \rightarrow \text{Continuous differential equation}$$

Verlet algorithm : numerical way to solve Newton's equation

Solving Newton's equation

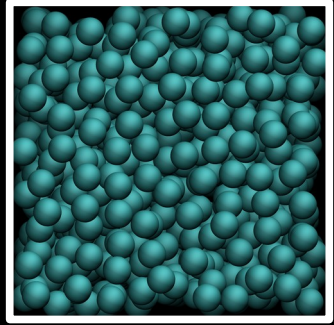


$$\sum_{j \neq i} \mathbf{F}_{ij} = m_i \ddot{\mathbf{r}}_i \rightarrow \text{Continuous differential equation}$$

Verlet algorithm : numerical way to solve Newton's equation

$$\mathbf{r}(t + \Delta t) = \mathbf{r}(t) + \dot{\mathbf{r}}(t)\Delta t + \frac{1}{2}\ddot{\mathbf{r}}(t)\Delta t^2$$

Solving Newton's equation



$$\sum_{j \neq i} \mathbf{F}_{ij} = m_i \ddot{\mathbf{r}}_i \rightarrow \text{Continuous differential equation}$$

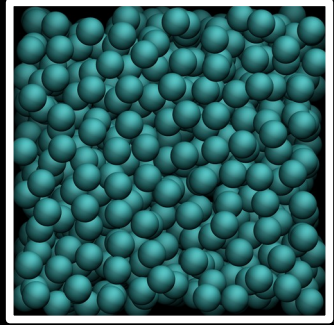
Verlet algorithm : numerical way to solve Newton's equation

$$\mathbf{r}(t + \Delta t) = \mathbf{r}(t) + \dot{\mathbf{r}}(t)\Delta t + \frac{1}{2}\ddot{\mathbf{r}}(t)\Delta t^2$$

Timestep



Solving Newton's equation



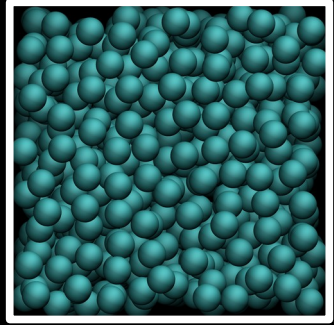
$$\sum_{j \neq i} \mathbf{F}_{ij} = m_i \ddot{\mathbf{r}}_i \rightarrow \text{Continuous differential equation}$$

Verlet algorithm : numerical way to solve Newton's equation

$$\mathbf{r}(t + \Delta t) = \mathbf{r}(t) + \dot{\mathbf{r}}(t)\Delta t + \frac{1}{2}\ddot{\mathbf{r}}(t)\Delta t^2$$

$$\dot{\mathbf{r}}(t + \Delta t) = \dot{\mathbf{r}}(t) + \frac{\ddot{\mathbf{r}}(t) + \ddot{\mathbf{r}}(t + \Delta t)}{2}\Delta t$$

Solving Newton's equation



$$\sum_{j \neq i} \mathbf{F}_{ij} = m_i \ddot{\mathbf{r}}_i \rightarrow \text{Continuous differential equation}$$

Verlet algorithm : numerical way to solve Newton's equation

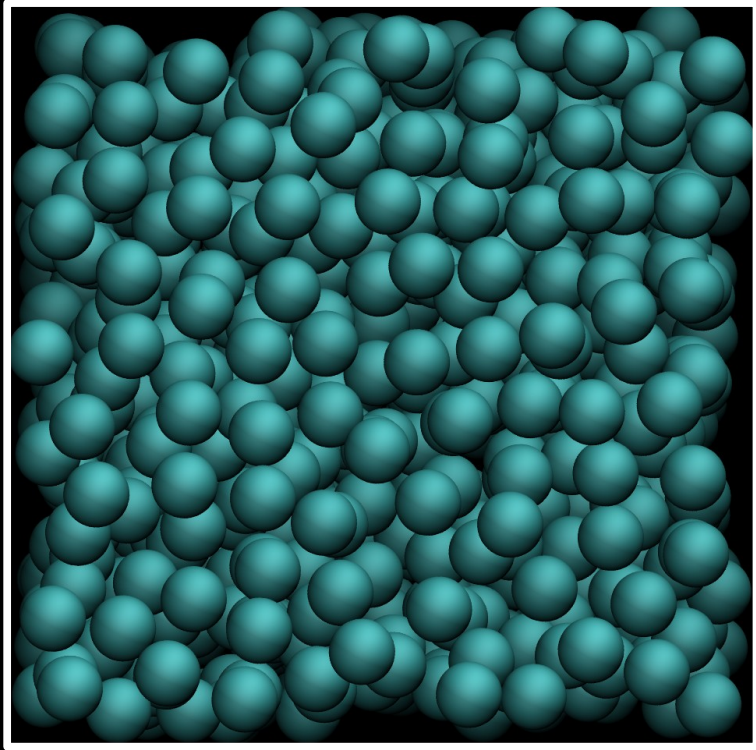
$$\mathbf{r}(t + \Delta t) = \mathbf{r}(t) + \dot{\mathbf{r}}(t)\Delta t + \frac{1}{2}\ddot{\mathbf{r}}(t)\Delta t^2$$

$$\dot{\mathbf{r}}(t + \Delta t) = \dot{\mathbf{r}}(t) + \frac{\ddot{\mathbf{r}}(t) + \ddot{\mathbf{r}}(t + \Delta t)}{2}\Delta t$$

Taylor expansion of the positions at time $t + \Delta t$:

$$\mathbf{r}(t + \Delta t) = \mathbf{r}(t) + \dot{\mathbf{r}}(t)\Delta t + \frac{1}{2!}\ddot{\mathbf{r}}(t)\Delta t^2 + \frac{1}{3!}\ddot{\mathbf{r}}^{(3)}(t)\Delta t^3 + \mathcal{O}(\Delta t^4)$$

Molecular dynamics



1) Define the system

2) Measure forces between atoms $\mathbf{F}_{ij}$

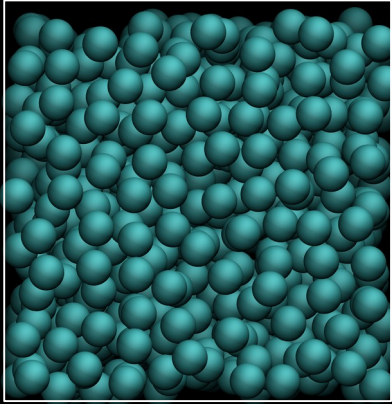
$$F_{ij} = -\frac{dU(r_{ij})}{dr_{ij}} \quad U_{ij}(r_{ij}) = 4\epsilon_{ij} \left[\left(\frac{\sigma_{ij}}{r_{ij}} \right)^{12} - \left(\frac{\sigma_{ij}}{r_{ij}} \right)^6 \right]$$

3) Solve Newton's equations of motion

4) Move atoms

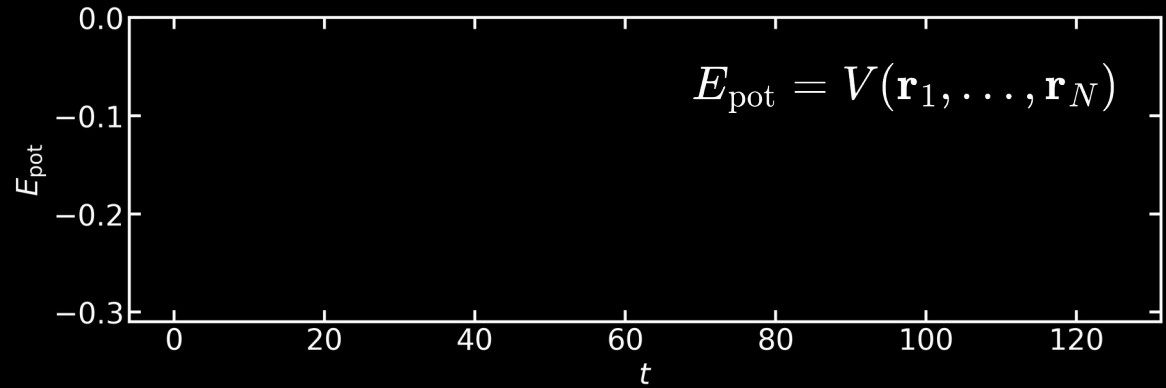
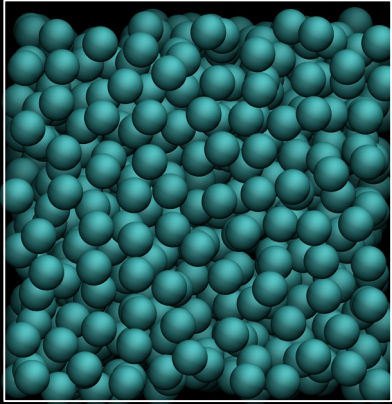
$$\mathbf{r}(t + \Delta t) = \mathbf{r}(t) + \dot{\mathbf{r}}(t)\Delta t + \frac{1}{2}\ddot{\mathbf{r}}(t)\Delta t^2$$

Energies from MD



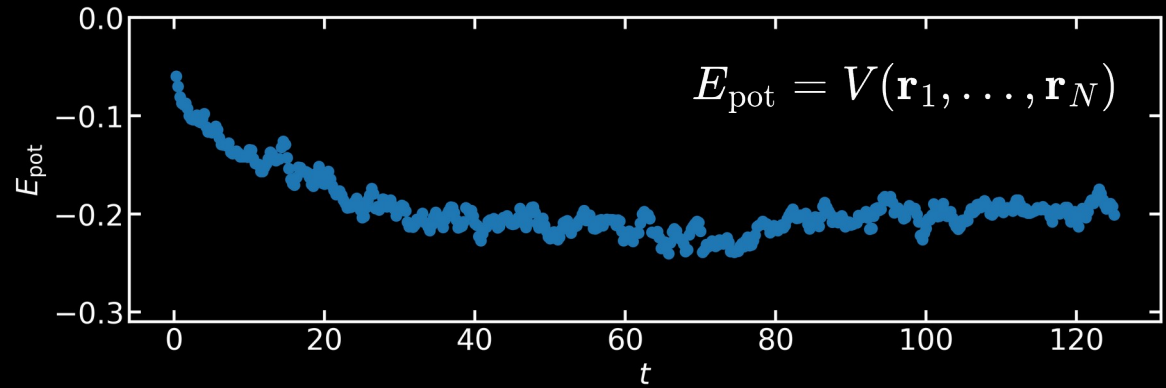
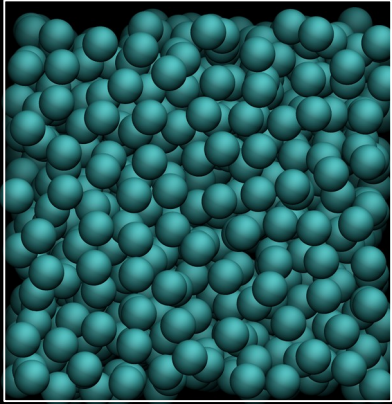
- 40 x 40 x 40 box
- 600 atoms
- Define LJ potential
- Apply Verlet algorithm

Energies from MD



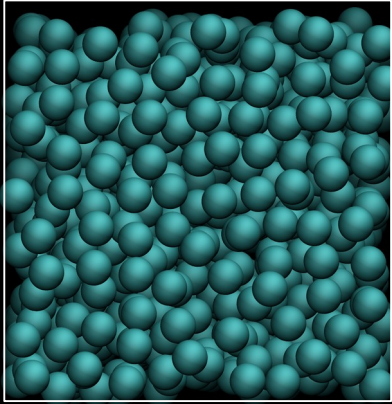
- 40 x 40 x 40 box
- 600 atoms
- Define LJ potential
- Apply Verlet algorithm

Energies from MD

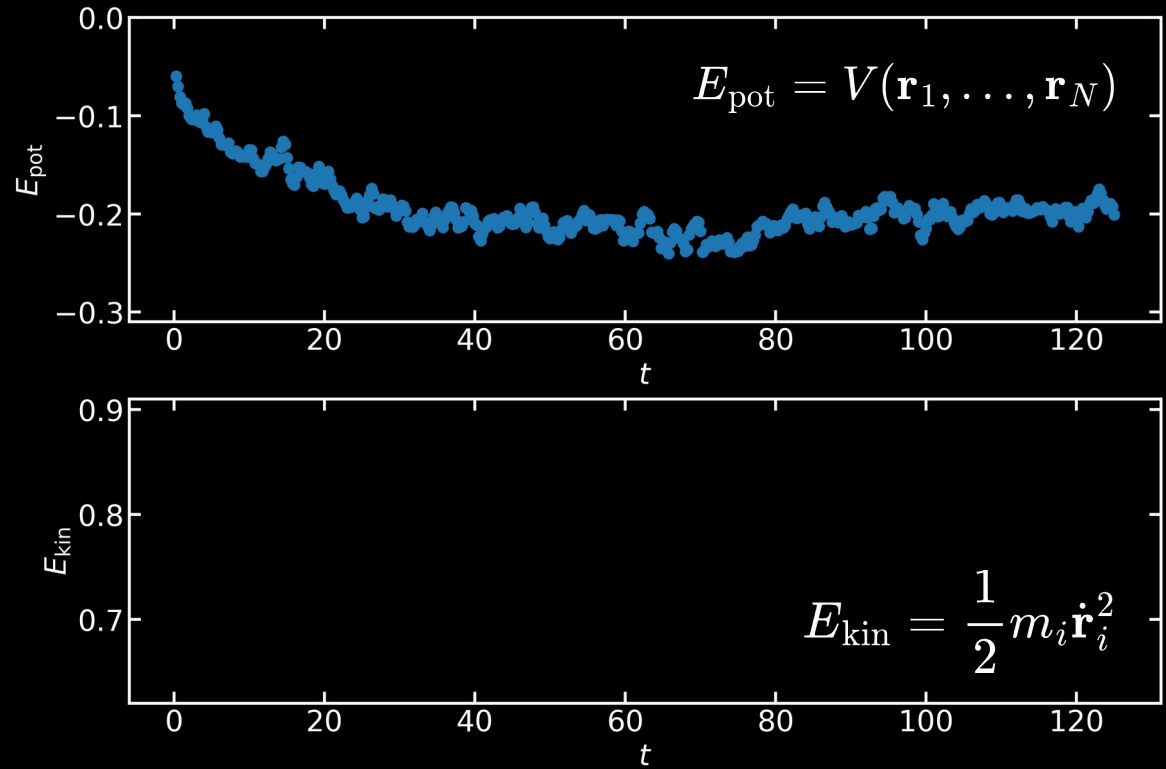


- 40 x 40 x 40 box
- 600 atoms
- Define LJ potential
- Apply Verlet algorithm

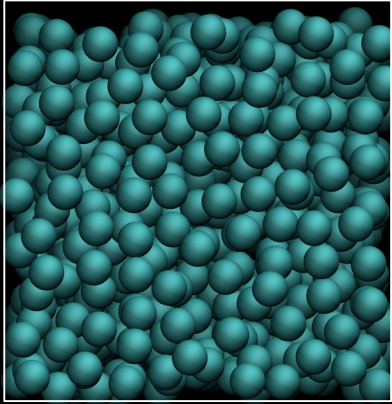
Energies from MD



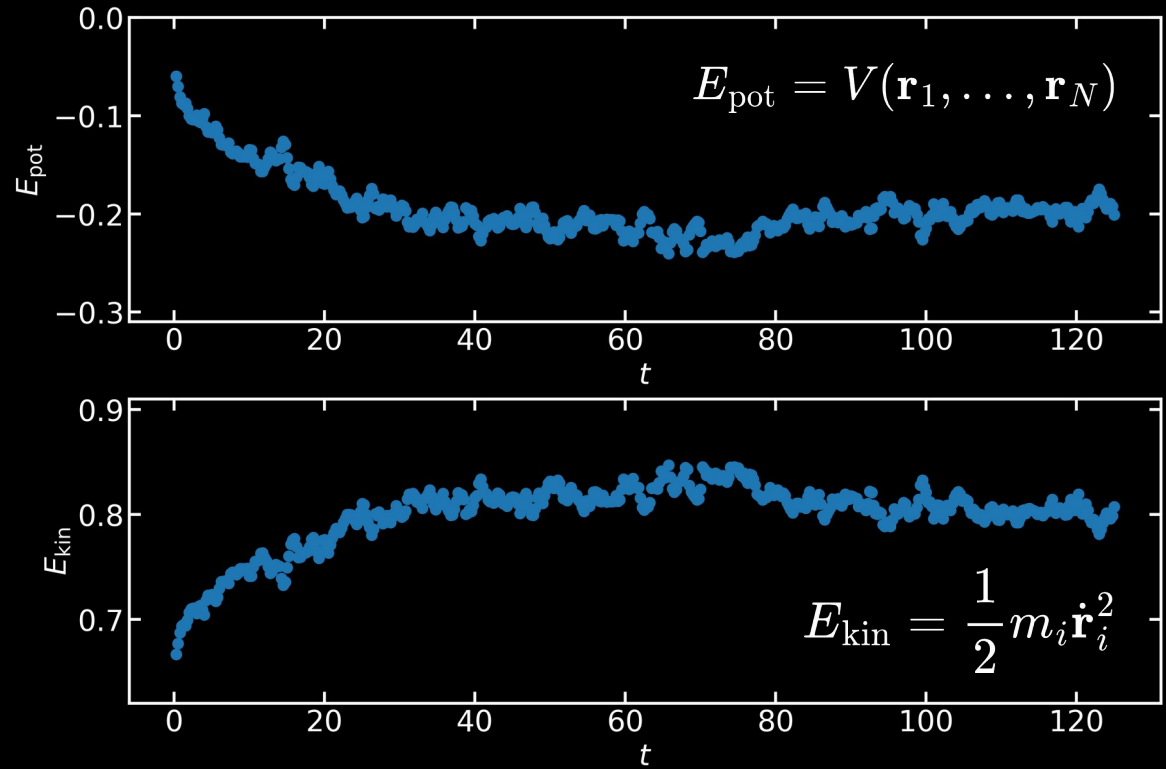
- 40 x 40 x 40 box
- 600 atoms
- Define LJ potential
- Apply Verlet algorithm



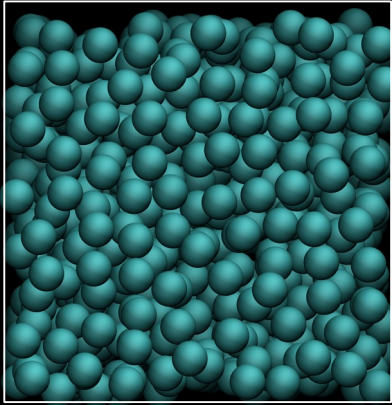
Energies from MD



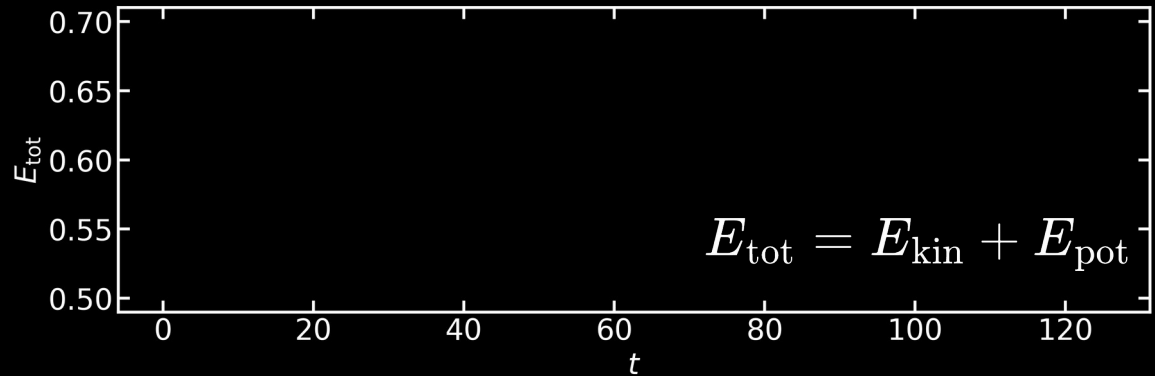
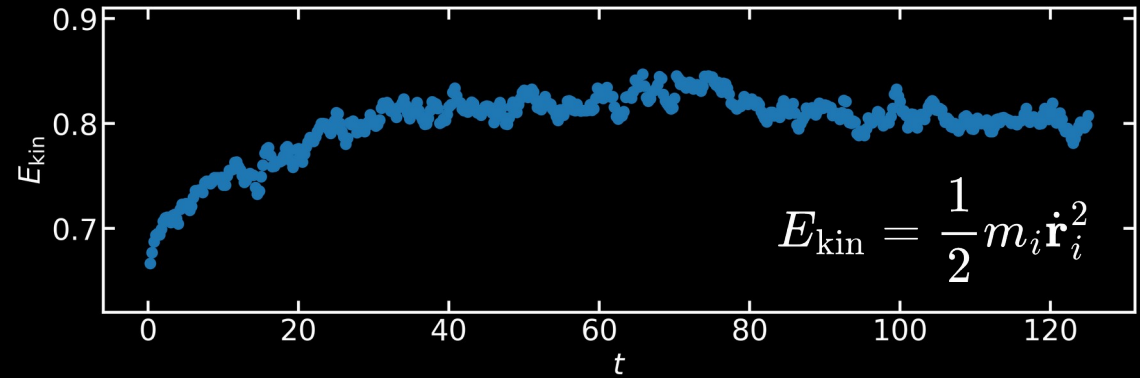
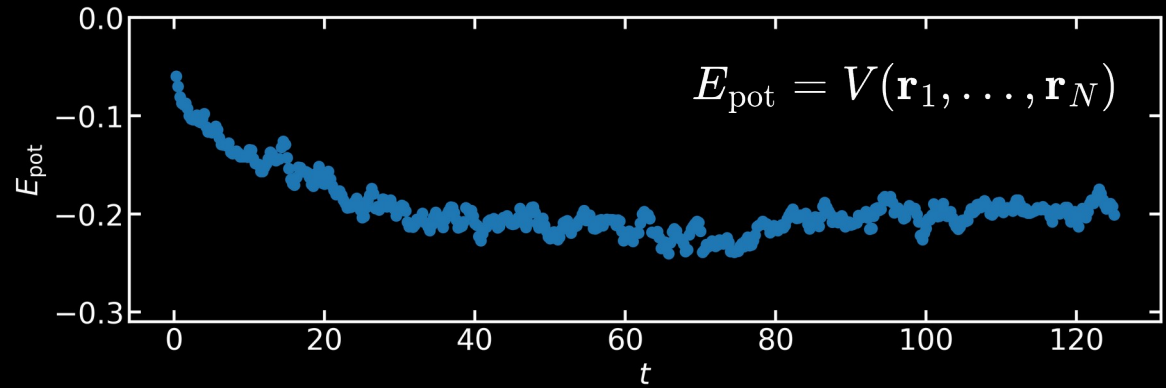
- 40 x 40 x 40 box
- 600 atoms
- Define LJ potential
- Apply Verlet algorithm



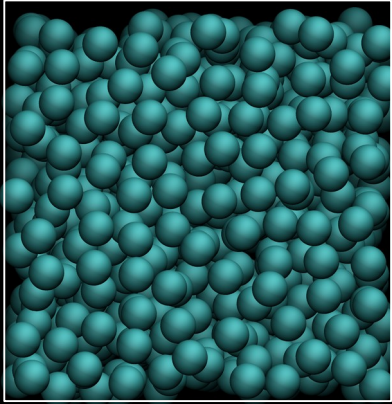
Energies from MD



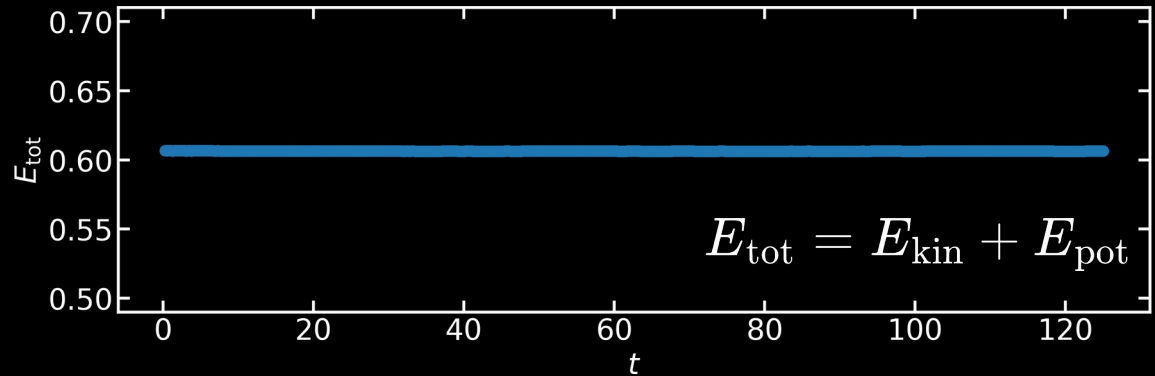
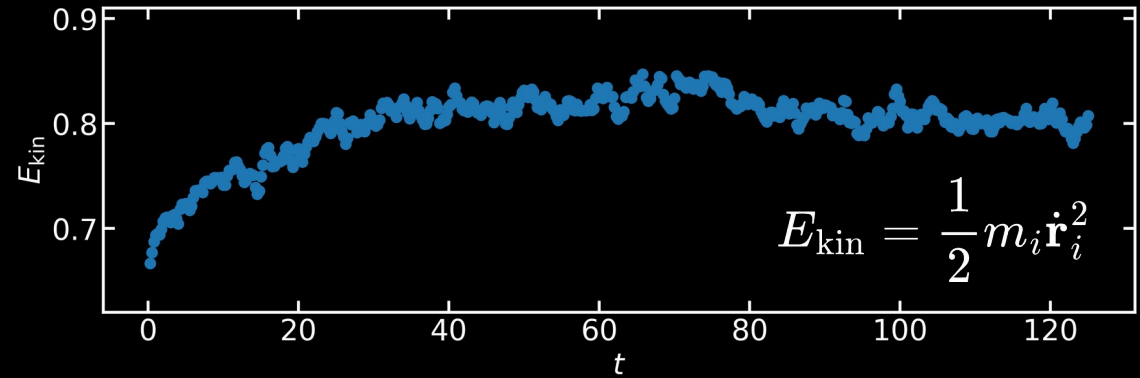
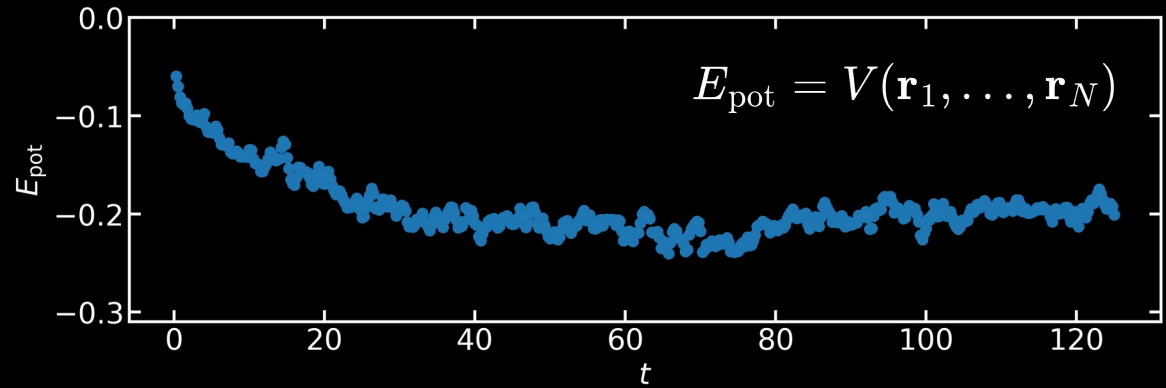
- 40 x 40 x 40 box
- 600 atoms
- Define LJ potential
- Apply Verlet algorithm



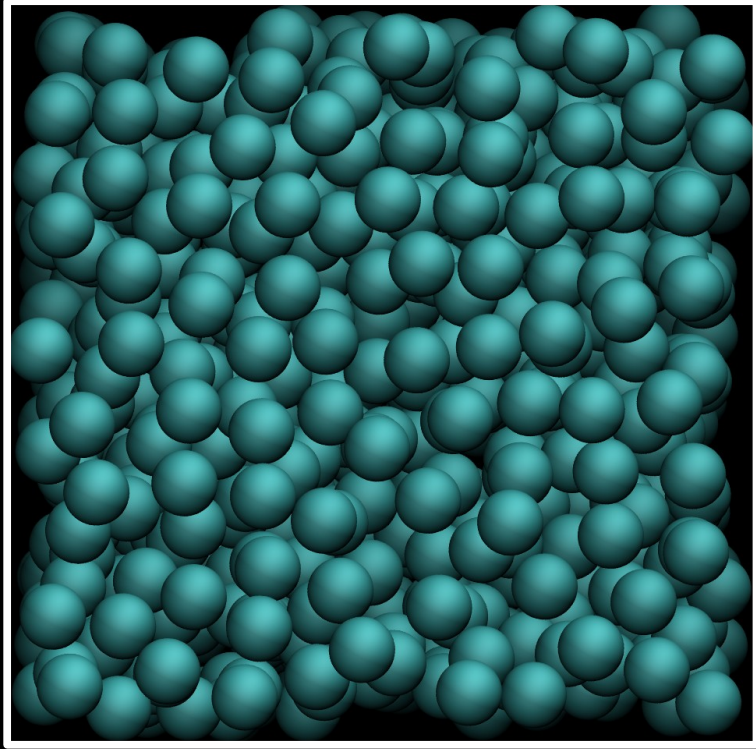
Energies from MD



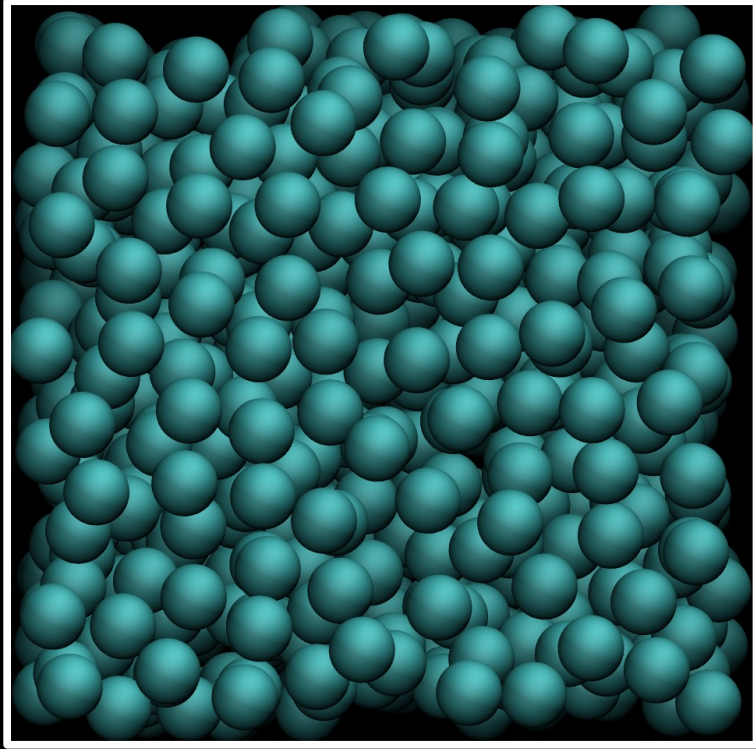
- 40 x 40 x 40 box
- 600 atoms
- Define LJ potential
- Apply Verlet algorithm



Thermodynamic ensembles

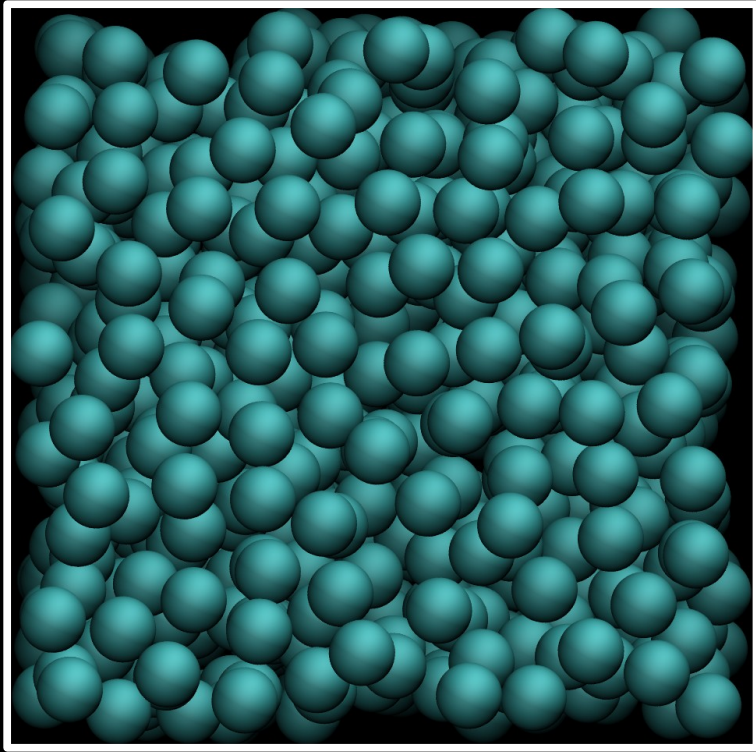


Thermodynamic ensembles



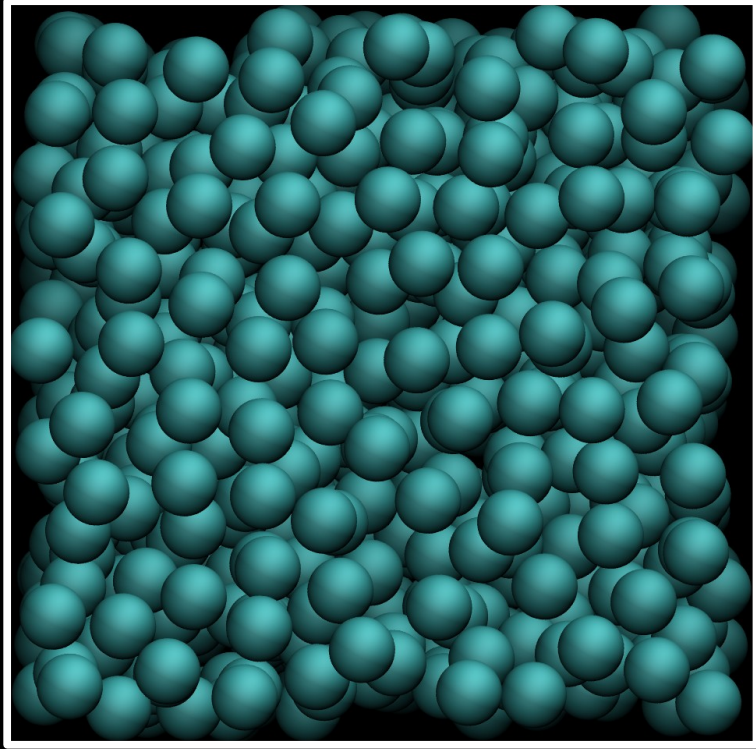
- fixed particle number (N)

Thermodynamic ensembles



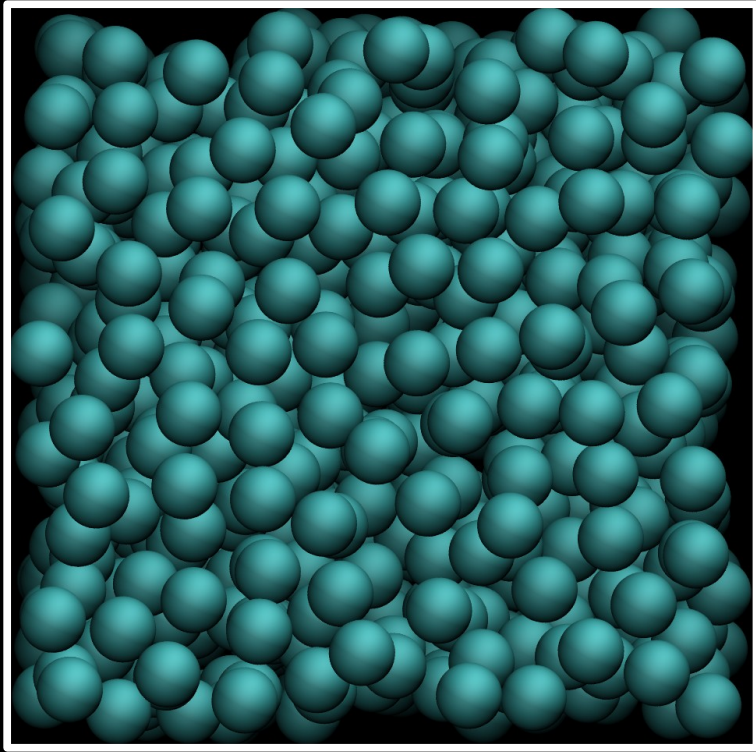
- fixed particle number (N)
- fixed box volume (V)

Thermodynamic ensembles



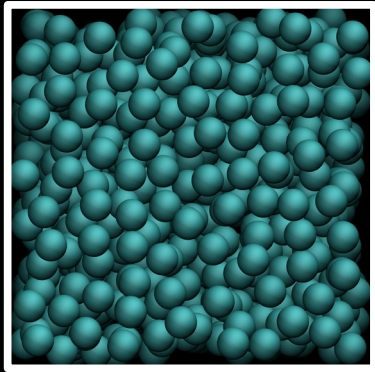
- fixed particle number (N)
- fixed box volume (V)
- constant energy (E)

Thermodynamic ensembles



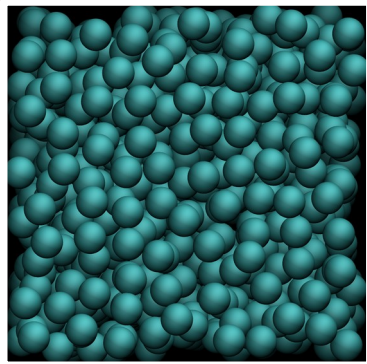
- fixed particle number (N)
 - fixed box volume (V)
 - constant energy (E)
- Microcanonical (NVE)

Thermostats : Mimic energy exchange with a bath



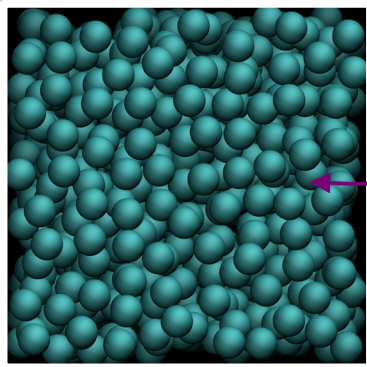
Thermostats : Mimic energy exchange with a bath

Bath with
temperature T_0



Thermostats : Mimic energy exchange with a bath

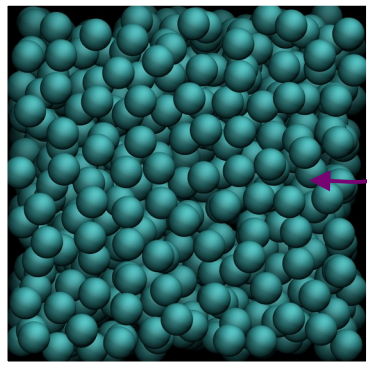
Bath with
temperature T_0



Energy

Thermostats : Mimic energy exchange with a bath

Bath with
temperature T_0



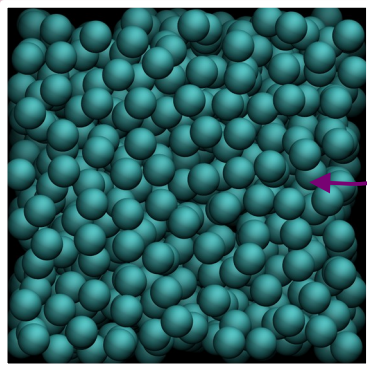
Energy

Simple thermostat: rescale velocities

$$\mathbf{v}_i \rightarrow \lambda \mathbf{v}_i$$

Thermostats : Mimic energy exchange with a bath

Bath with
temperature T_0



Energy

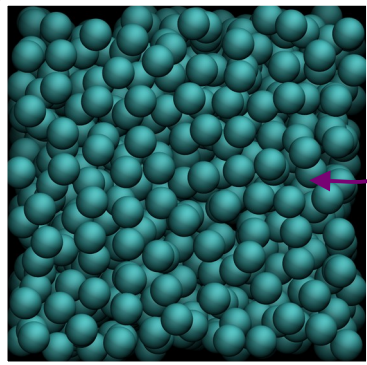
Simple thermostat: rescale velocities

$$\mathbf{v}_i \rightarrow \lambda \mathbf{v}_i$$

$$\lambda = \sqrt{1 + \frac{\Delta t}{\tau} \left(\frac{T_0}{T} - 1 \right)}$$

Thermostats : Mimic energy exchange with a bath

Bath with
temperature T_0



Energy

Simple thermostat: rescale velocities

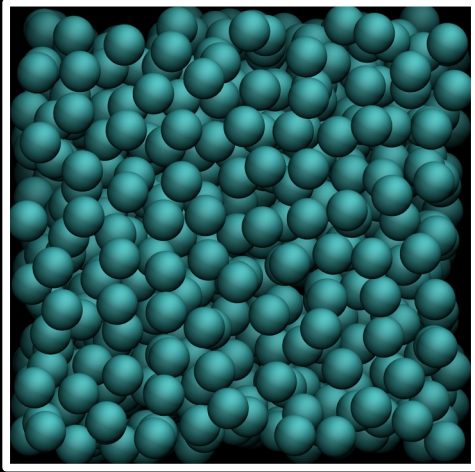
$$\mathbf{v}_i \rightarrow \lambda \mathbf{v}_i$$

Target temperature

$$\lambda = \sqrt{1 + \frac{\Delta t}{\tau} \left(\frac{T_0}{T} - 1 \right)}$$

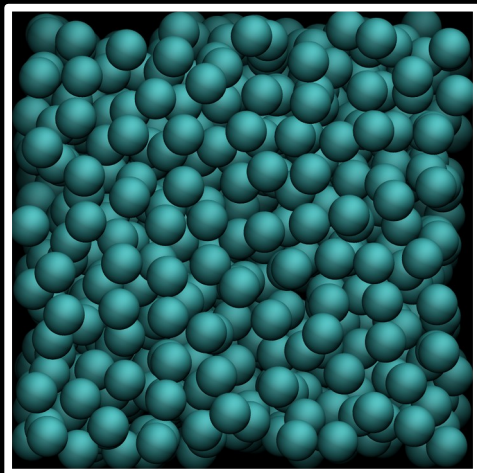
Coupling time

Effect thermostats

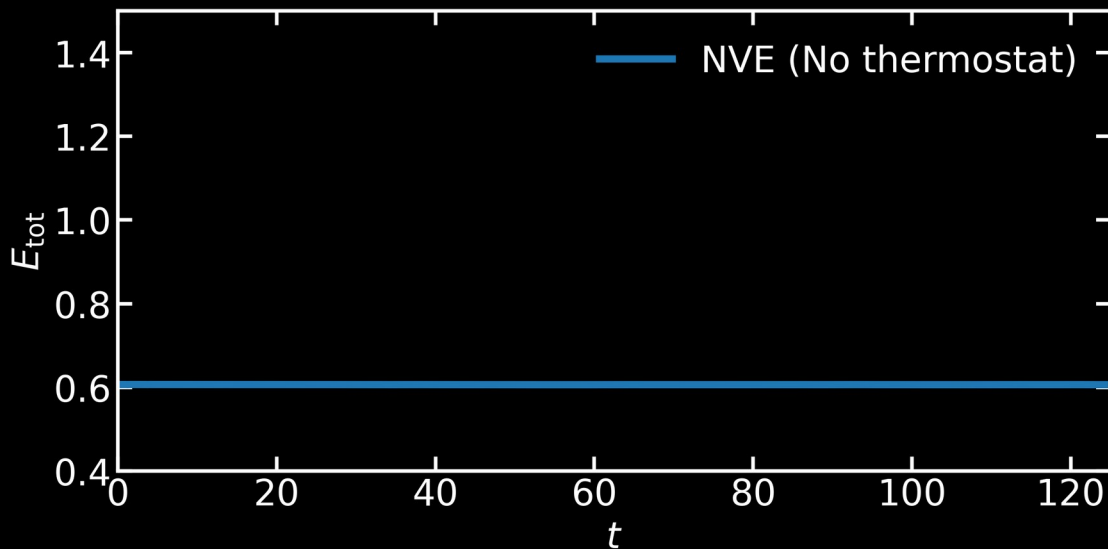
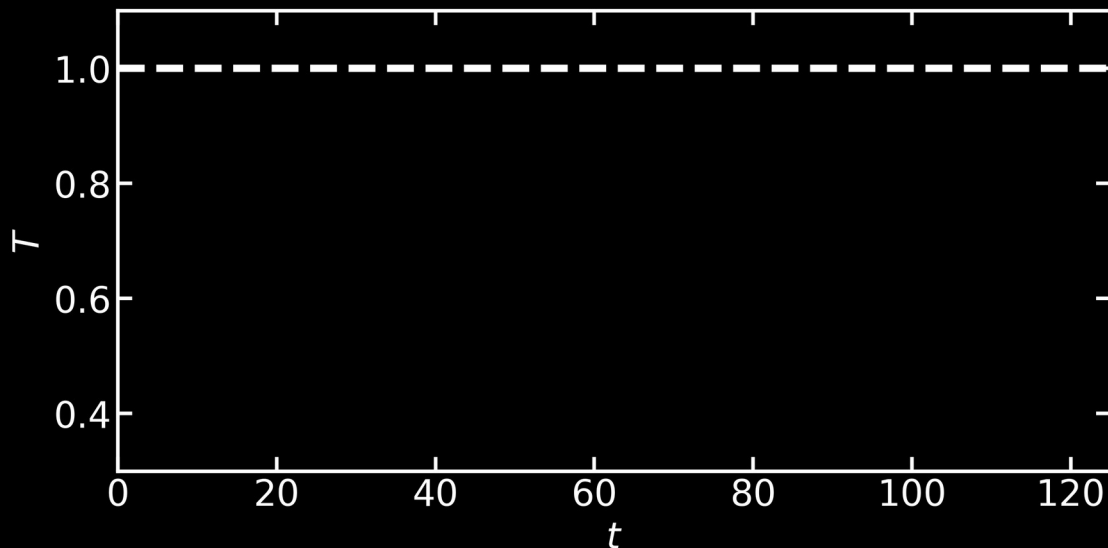


- Create a $40 \times 40 \times 40$ box
- Place 600 atoms into it
- Use LJ potential
- Apply Verlet algorithm

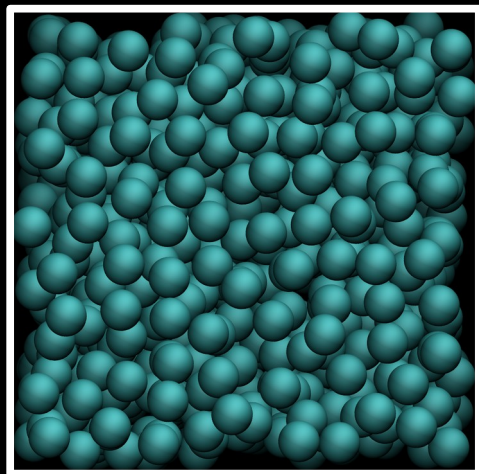
Effect thermostats



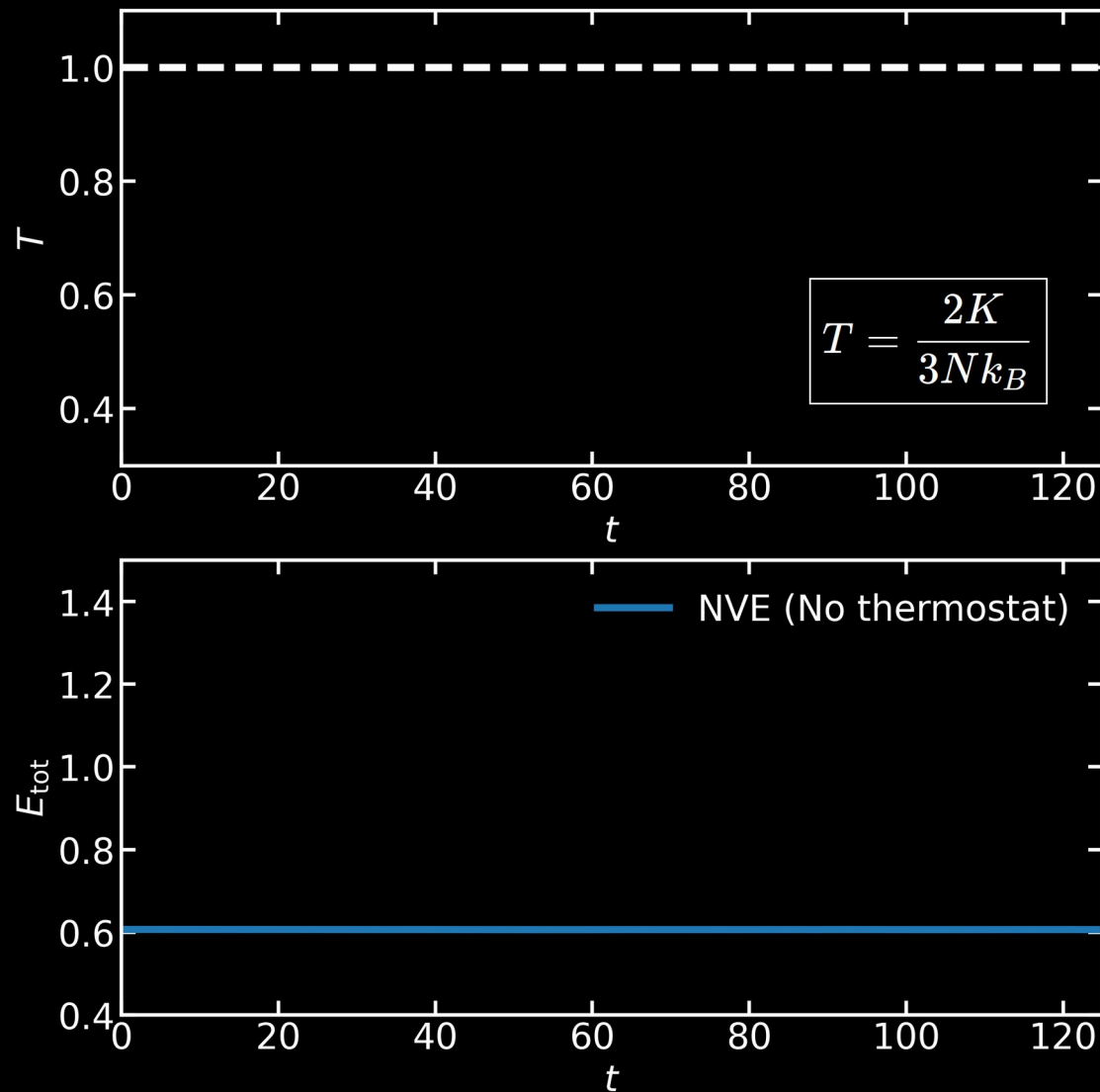
- Create a $40 \times 40 \times 40$ box
- Place 600 atoms into it
- Use LJ potential
- Apply Verlet algorithm



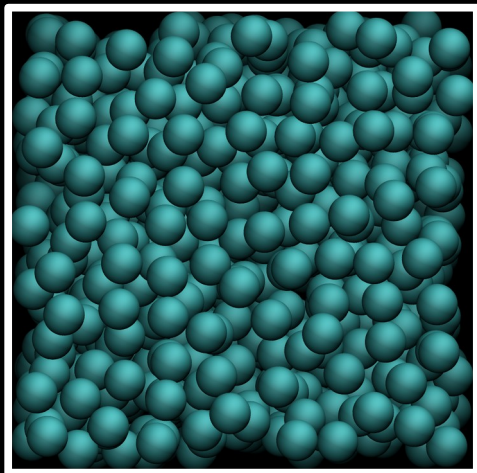
Effect thermostats



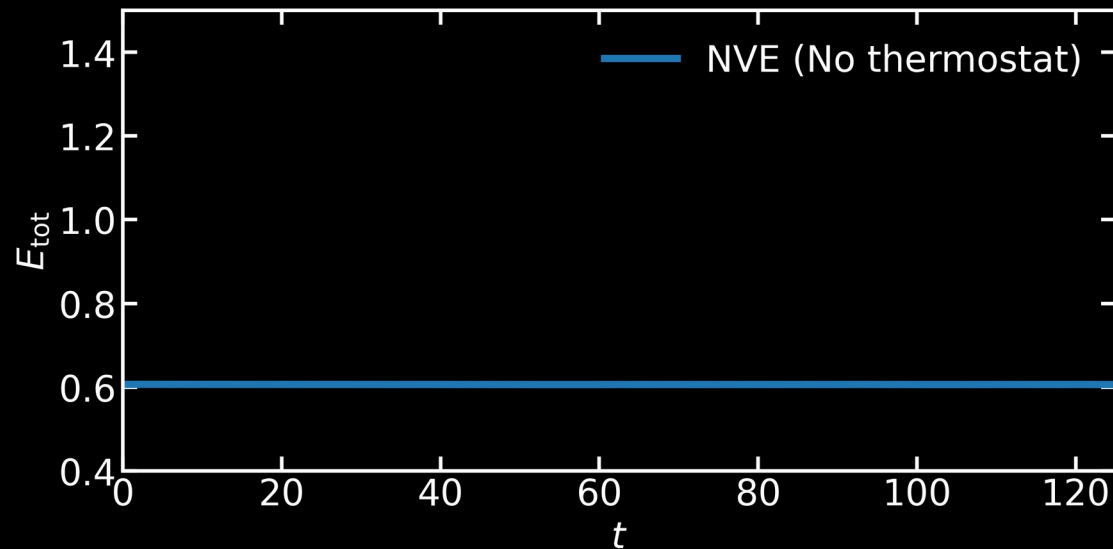
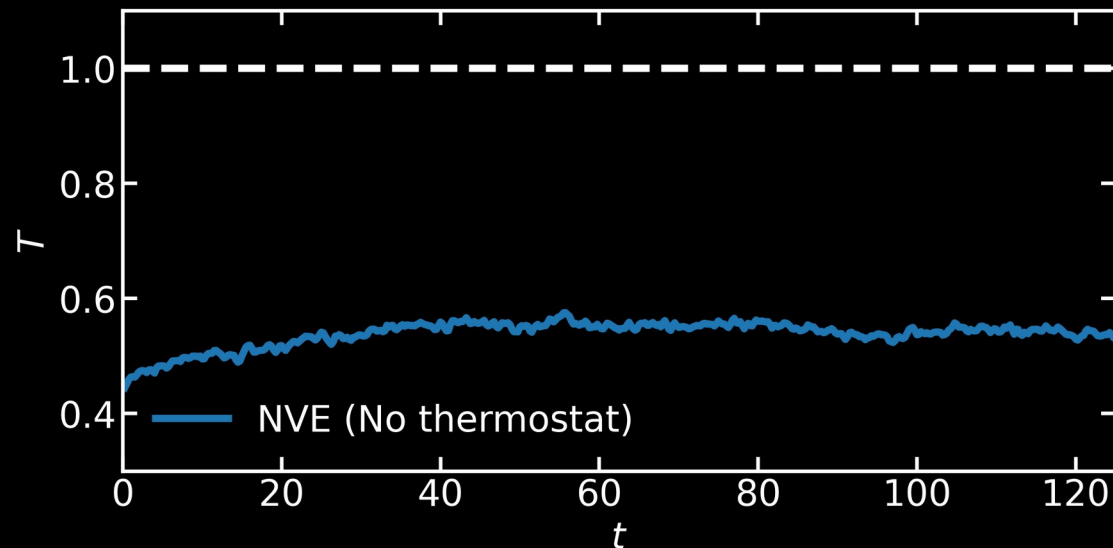
- Create a 40 x 40 x 40 box
- Place 600 atoms into it
- Use LJ potential
- Apply Verlet algorithm



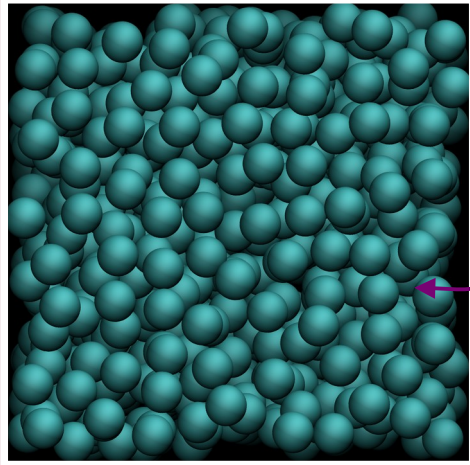
Effect thermostats



- Create a $40 \times 40 \times 40$ box
- Place 600 atoms into it
- Use LJ potential
- Apply Verlet algorithm



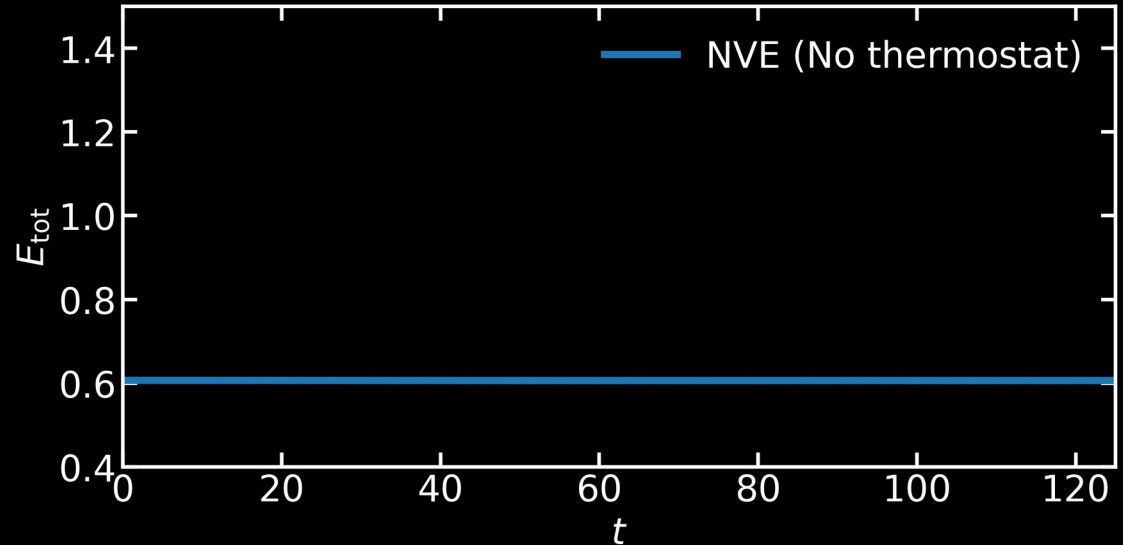
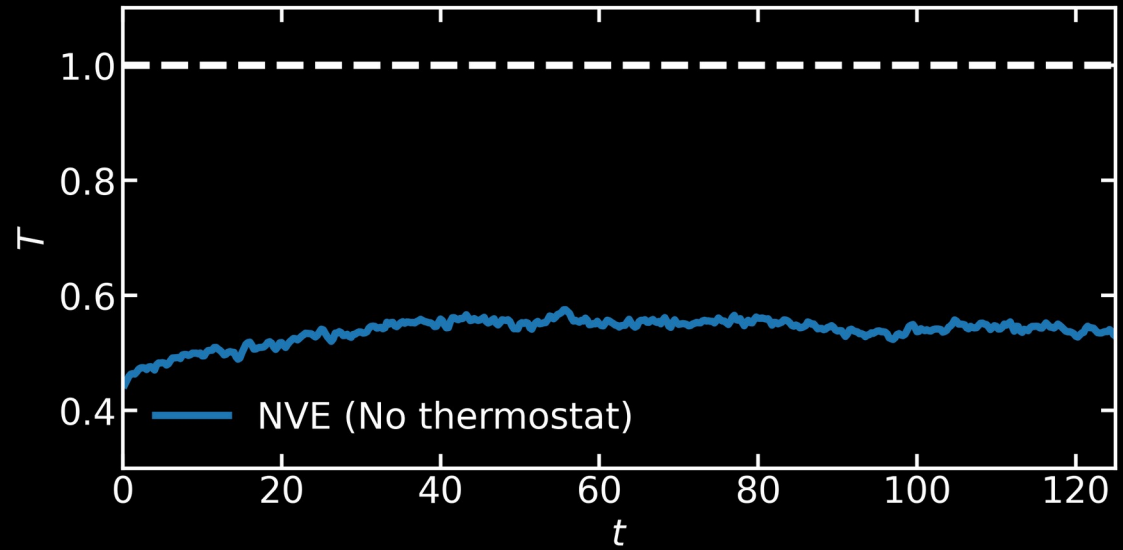
Effect thermostats



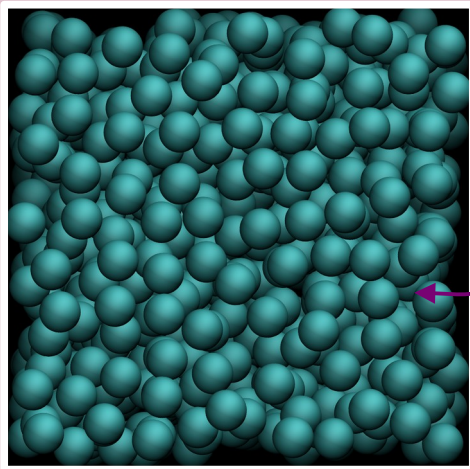
$$T_0 = 1$$

Energy

- Create a $40 \times 40 \times 40$ box
- Place 600 atoms into it
- Use LJ potential
- Apply Verlet algorithm
- Apply thermostat ($T_0 = 1$)



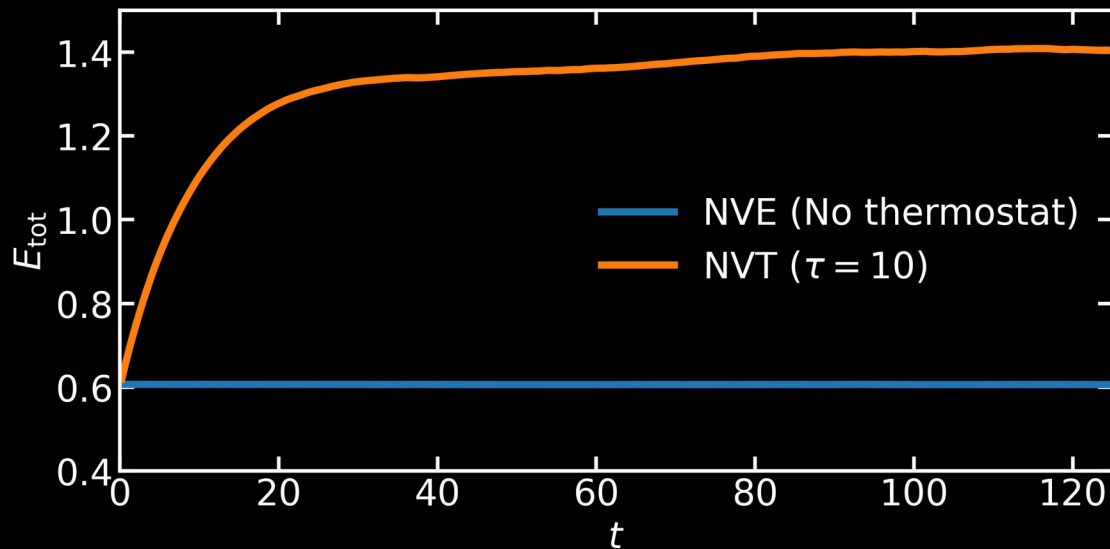
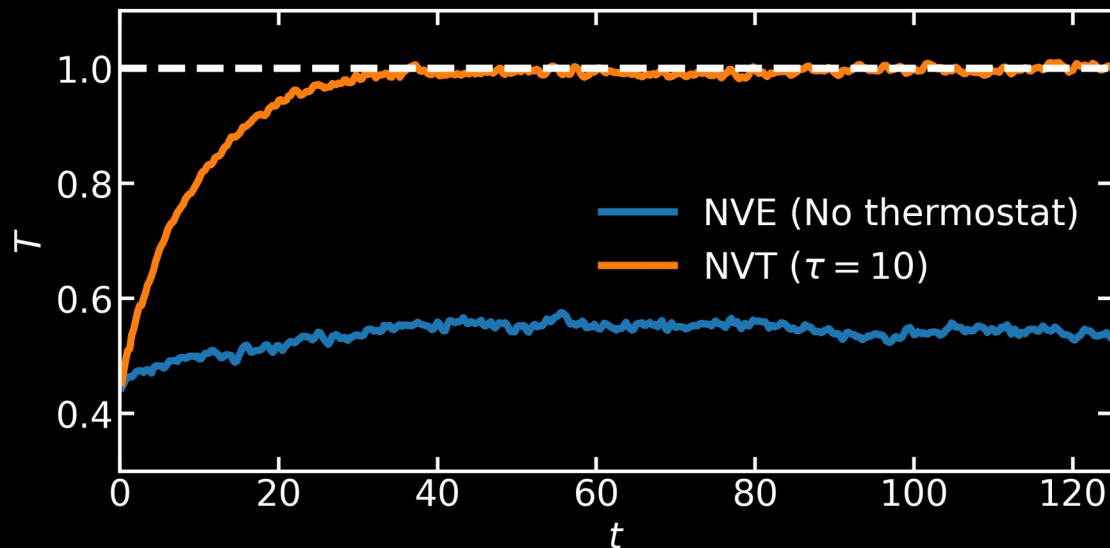
Effect thermostats



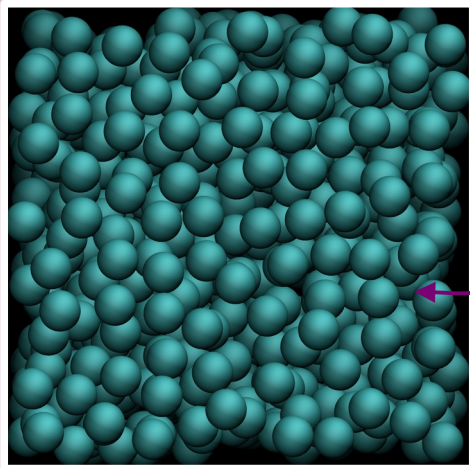
$$T_0 = 1$$

Energy

- Create a $40 \times 40 \times 40$ box
- Place 600 atoms into it
- Use LJ potential
- Apply Verlet algorithm
- Apply thermostat ($T_0 = 1$)



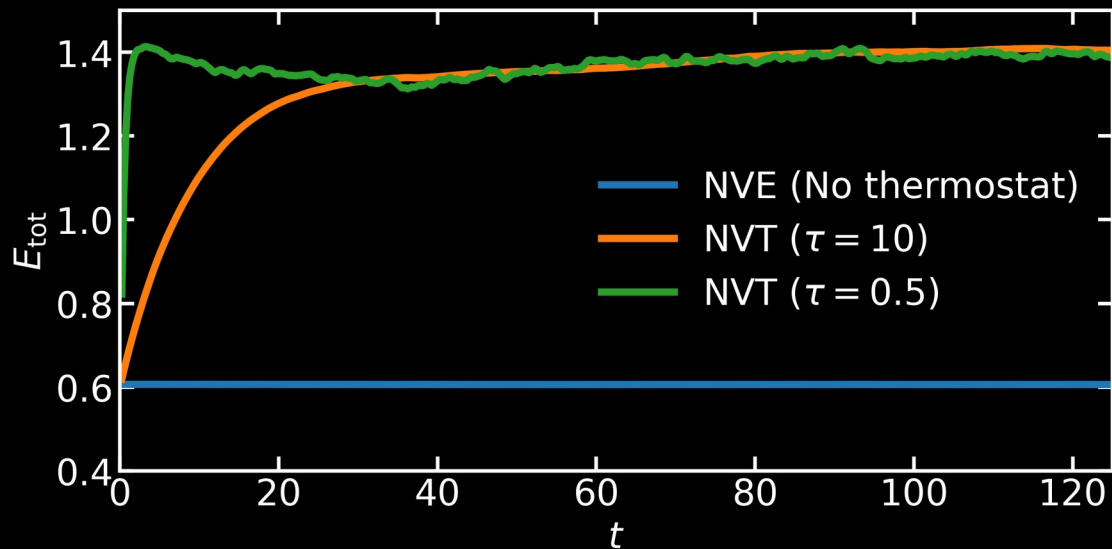
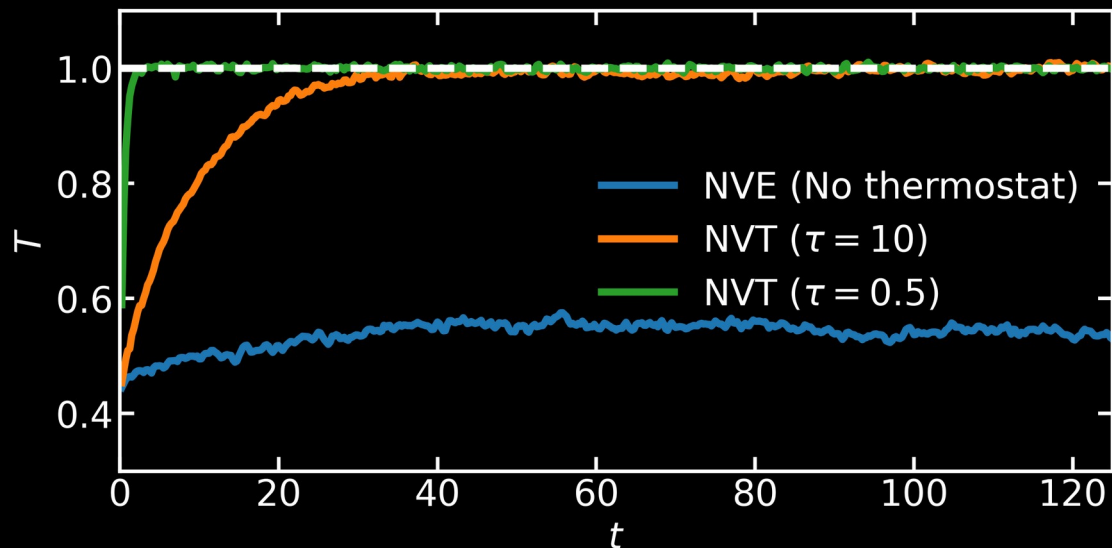
Effect thermostats



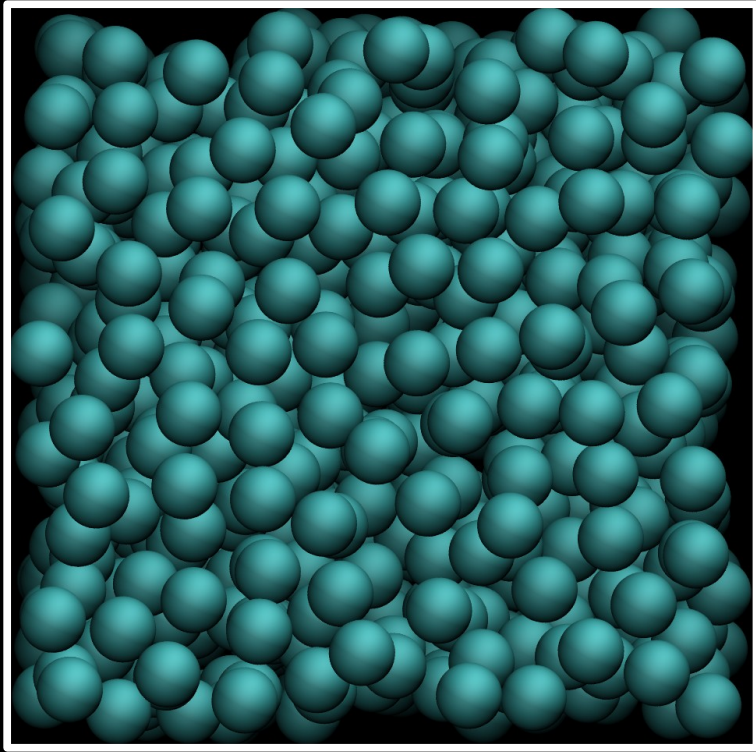
$$T_0 = 1$$

Energy

- Create a $40 \times 40 \times 40$ box
- Place 600 atoms into it
- Use LJ potential
- Apply Verlet algorithm
- Apply thermostat ($T_0 = 1$)



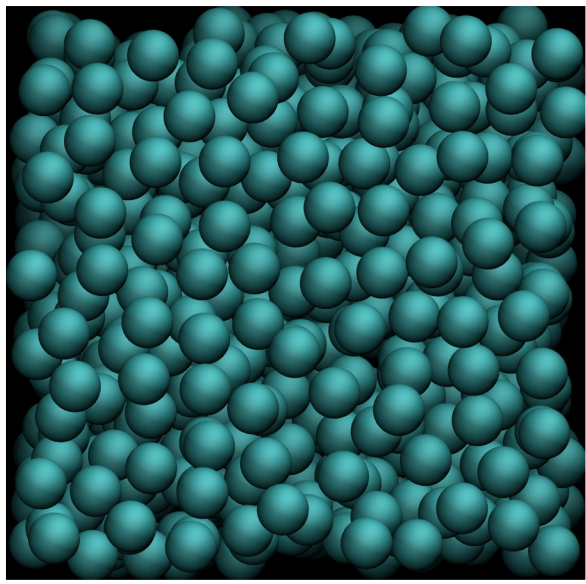
Thermodynamic ensembles



- fixed particle number (N)
 - fixed box volume (V)
 - constant energy (E)
- Microcanonical (NVE)

Thermodynamic ensembles

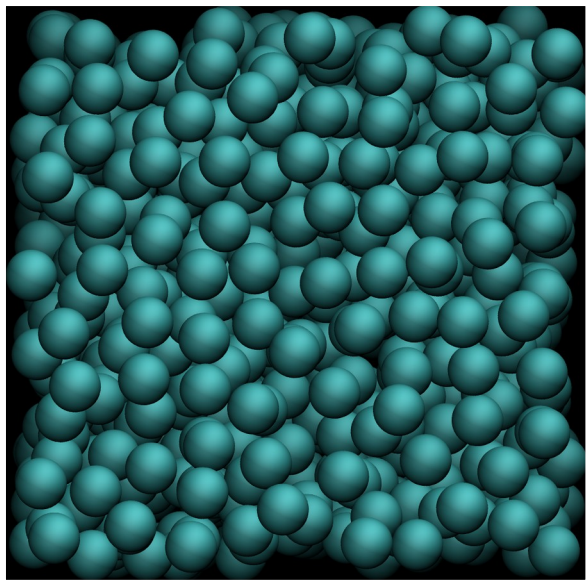
Bath (T_0)



- fixed particle number (N)
 - fixed box volume (V)
 - constant energy (E)
- Microcanonical (NVE)

Thermodynamic ensembles

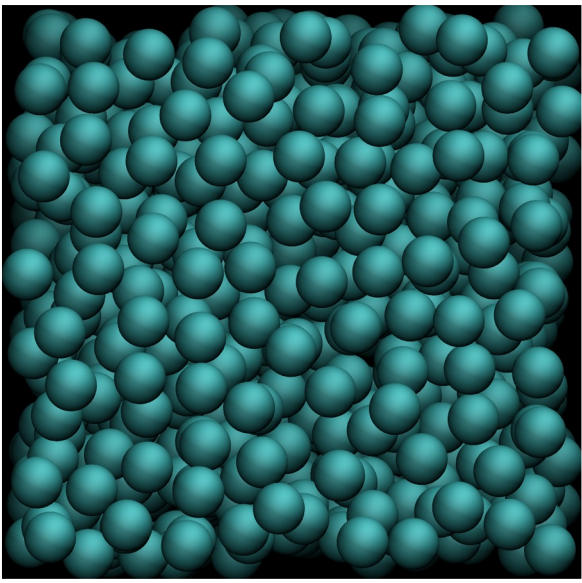
Bath (T_0)



- fixed particle number (N)
- fixed box volume (V)
- ~~- constant energy (E)~~
- ~~→ Microcanonical (NVE)~~

Thermodynamic ensembles

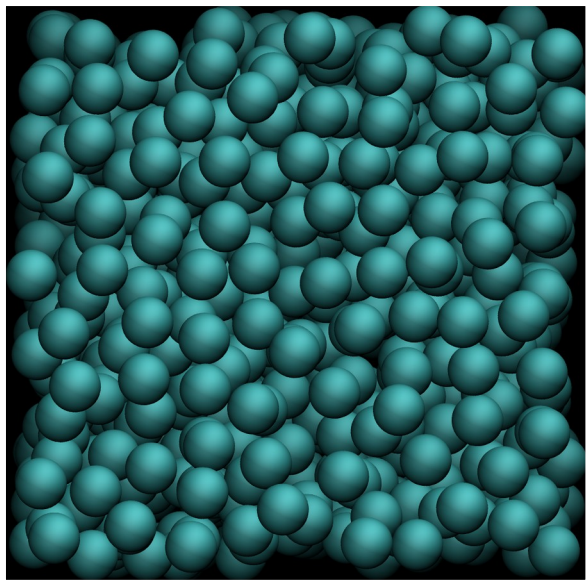
Bath (T_0)



- fixed particle number (N)
- fixed box volume (V)
- constant temperature (T)

Thermodynamic ensembles

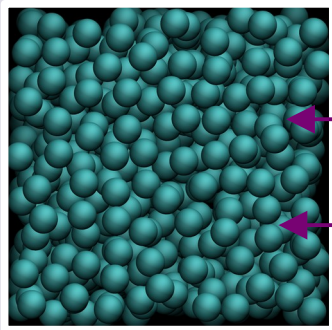
Bath (T_0)



- fixed particle number (N)
 - fixed box volume (V)
 - constant temperature (T)
- Canonical (NVT)

Barostat : Mimics the exchange of volume with a bath

Bath with temperature T_0 and pressure P_0)

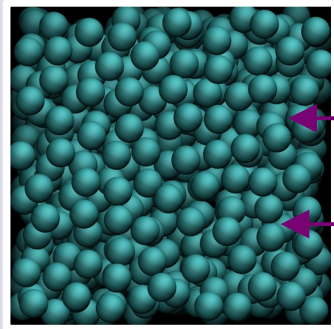


Energy

Volume

Barostat : Mimics the exchange of volume with a bath

Bath with temperature T_0 and pressure P_0)



Energy

Volume

Simple barostat: rescale box volume

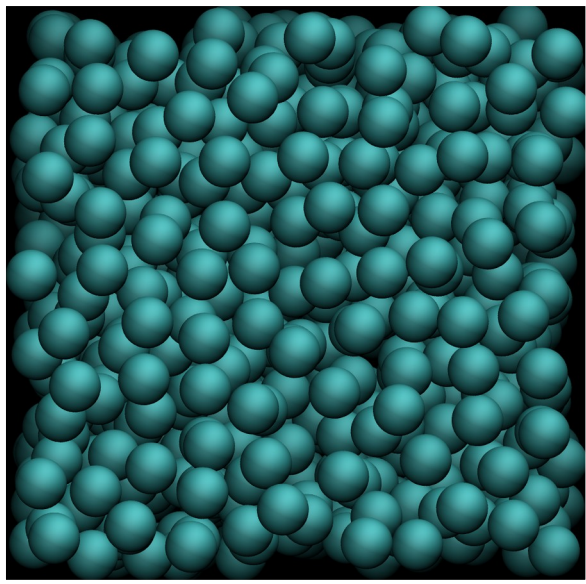
$$\mathbf{V}_i \rightarrow \mu \mathbf{V}_i$$

$$\mu = 1 - \frac{\Delta t}{\tau} \kappa_T (P_0 - P)$$

↖ Isothermal compressibility

Thermodynamic ensembles

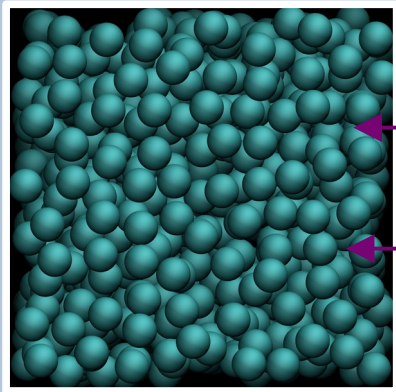
Bath (T_0, P_0)



- fixed particle number (N)
 - constant pressure (P)
 - constant temperature (T)
- Isothermal-isobaric (NPT)

Thermodynamic ensembles

Bath (T_0, μ_0)

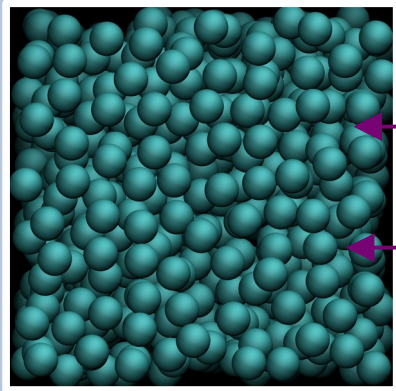


Energy

Particles

Thermodynamic ensembles

Bath (T_0, μ_0)



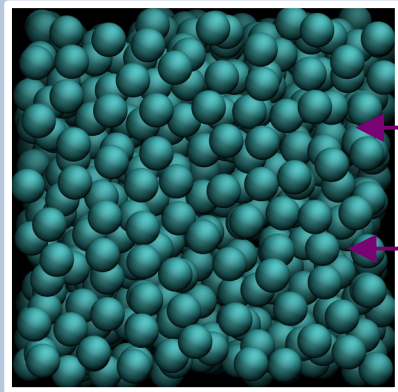
Energy

Particles

- constant chemical potential (μ)
- constant volume (V)
- constant temperature (T)

Thermodynamic ensembles

Bath (T_0, μ_0)



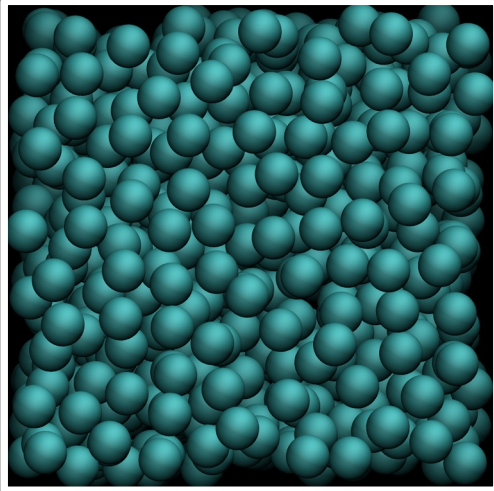
Energy

Particles

- constant chemical potential (μ)
 - constant volume (V)
 - constant temperature (T)
- Grand canonical (μVT)

Molecular dynamics

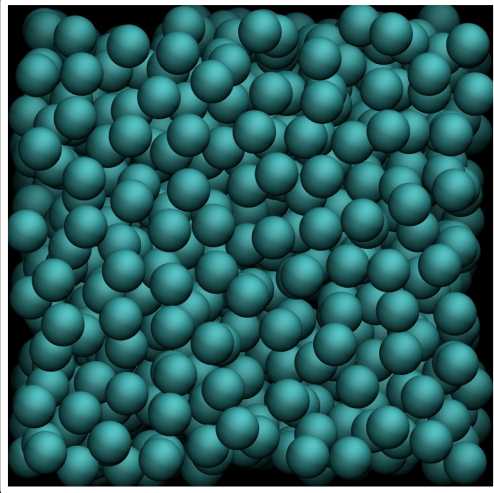
Periodic boundary conditions



- 1) Define the system
- 2) Measure forces between atoms $\mathbf{F}_{ij}$
- 3) Solve Newton's equations of motion
- 4) Move atoms $\mathbf{r}(t + \Delta t) = \mathbf{r}(t) + \dot{\mathbf{r}}(t)\Delta t + \frac{1}{2}\ddot{\mathbf{r}}(t)\Delta t^2$
- 5) Optional : Apply thermostat or barostat

Molecular dynamics

Periodic boundary conditions

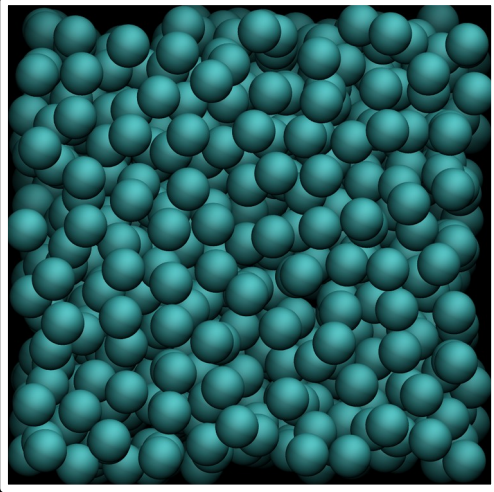


- 1) Define the system
- 2) Measure forces between atoms $\mathbf{F}_{ij}$
- 3) Solve Newton's equations of motion
- 4) Move atoms $\mathbf{r}(t + \Delta t) = \mathbf{r}(t) + \dot{\mathbf{r}}(t)\Delta t + \frac{1}{2}\ddot{\mathbf{r}}(t)\Delta t^2$
- 5) Optional : Apply thermostat or barostat

Good model for argon or other simple fluids

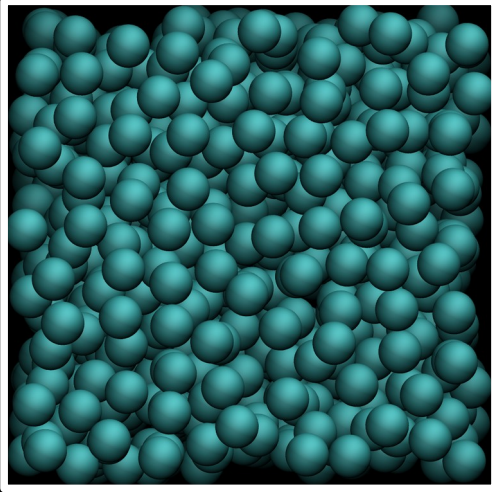
Measurements

What physics can we learn ?



Measurements

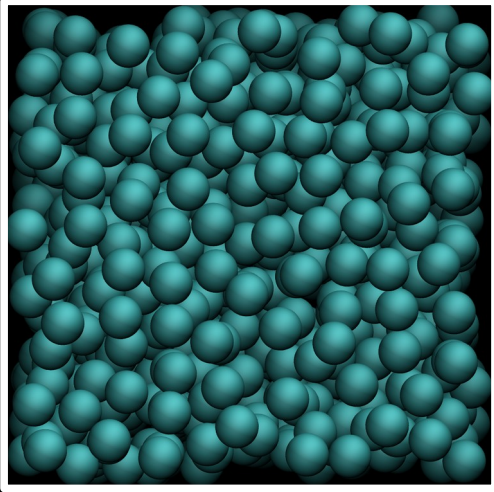
What physics can we learn ?



- Structural (spatial arrangement)

Measurements

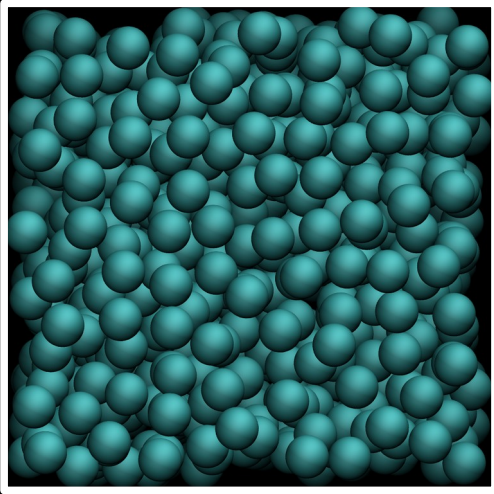
What physics can we learn ?



- Structural (spatial arrangement)
- Dynamics (time correlation and motion)

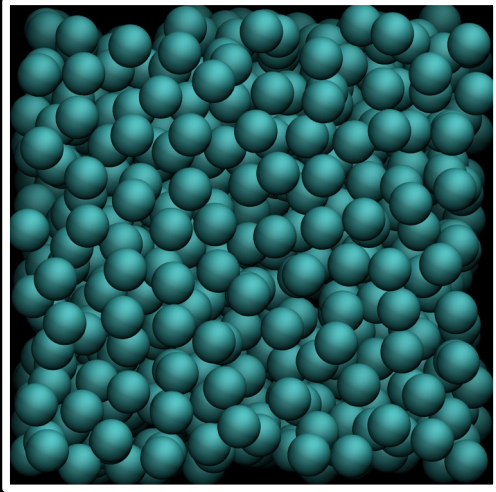
Measurements

What physics can we learn ?



- Structural (spatial arrangement)
- Dynamics (time correlation and motion)
- Thermodynamics (e.g. temperature)

Measurements



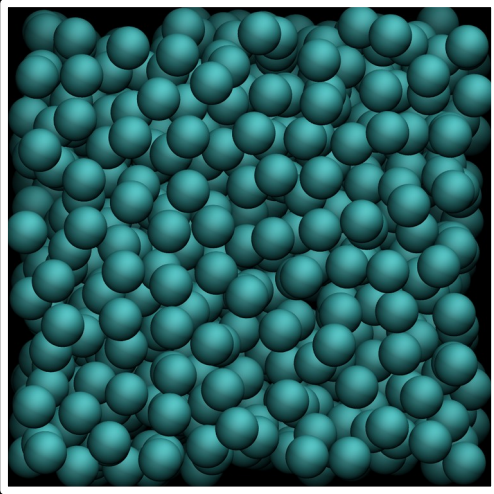
What physics can we learn ?

- Structural (spatial arrangement)
- Dynamics (time correlation and motion)
- Thermodynamics (e.g. temperature)
- Transport (viscosity)

Probing the structure

Radial distribution function $g(r) = \frac{\rho(r)}{\rho}$

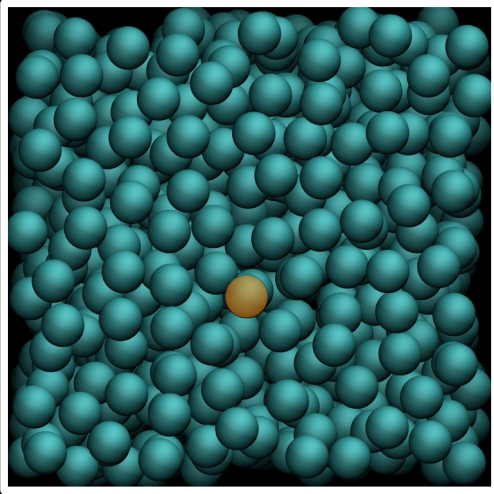
→ how particles are distributed around one another



Probing the structure

Radial distribution function $g(r) = \frac{\rho(r)}{\rho}$

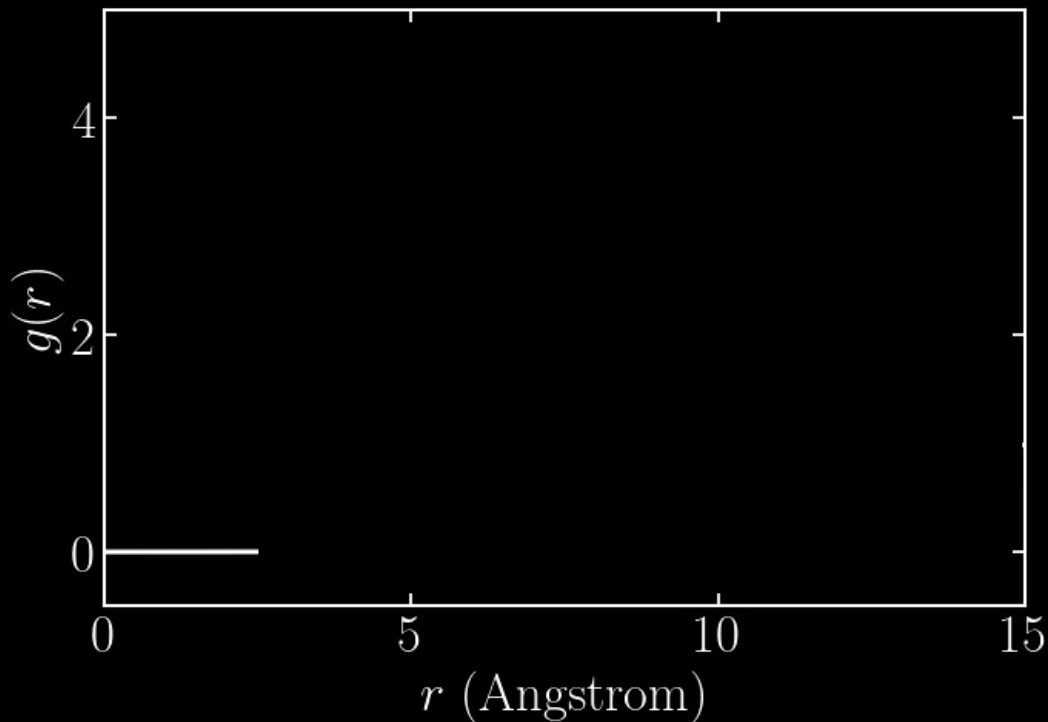
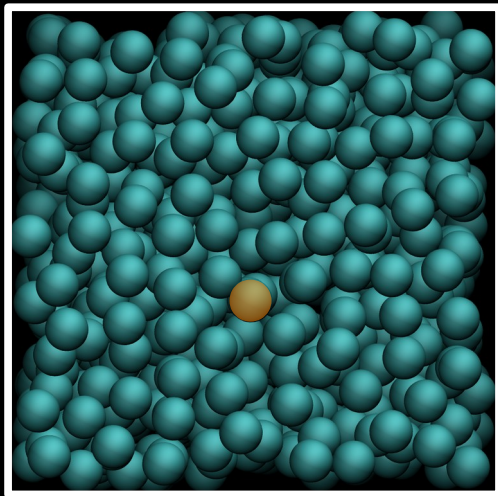
→ how particles are distributed around one another



Probing the structure

Radial distribution function $g(r) = \frac{\rho(r)}{\rho}$

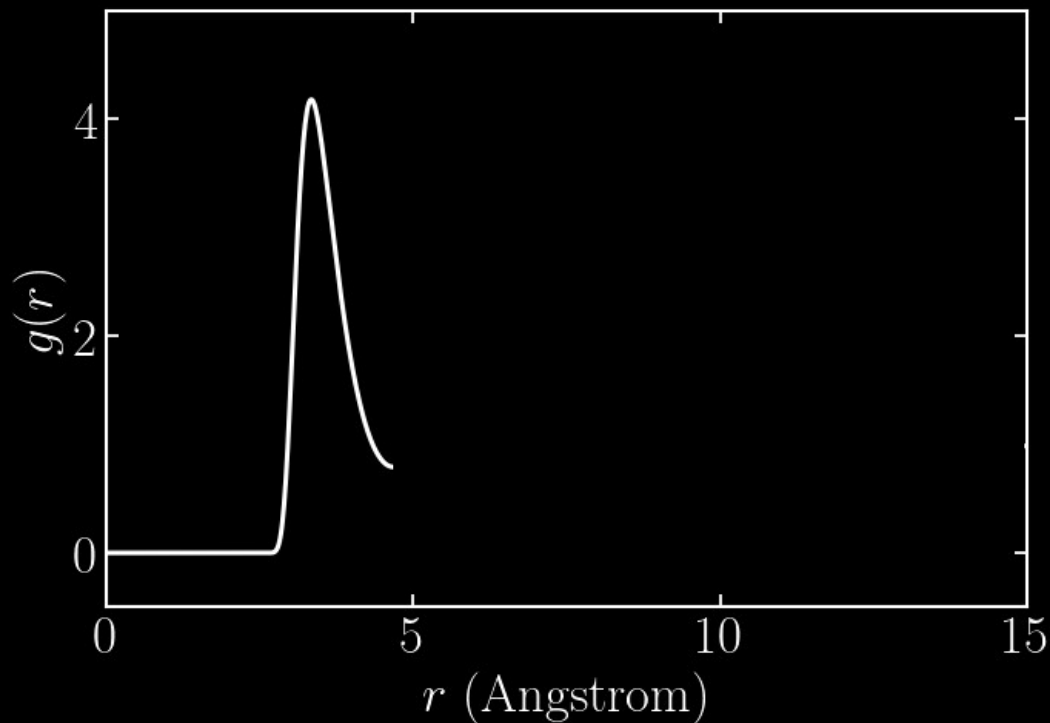
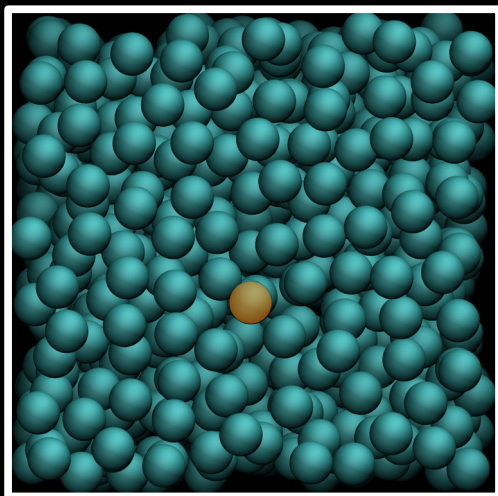
→ how particles are distributed around one another



Probing the structure

Radial distribution function $g(r) = \frac{\rho(r)}{\rho}$

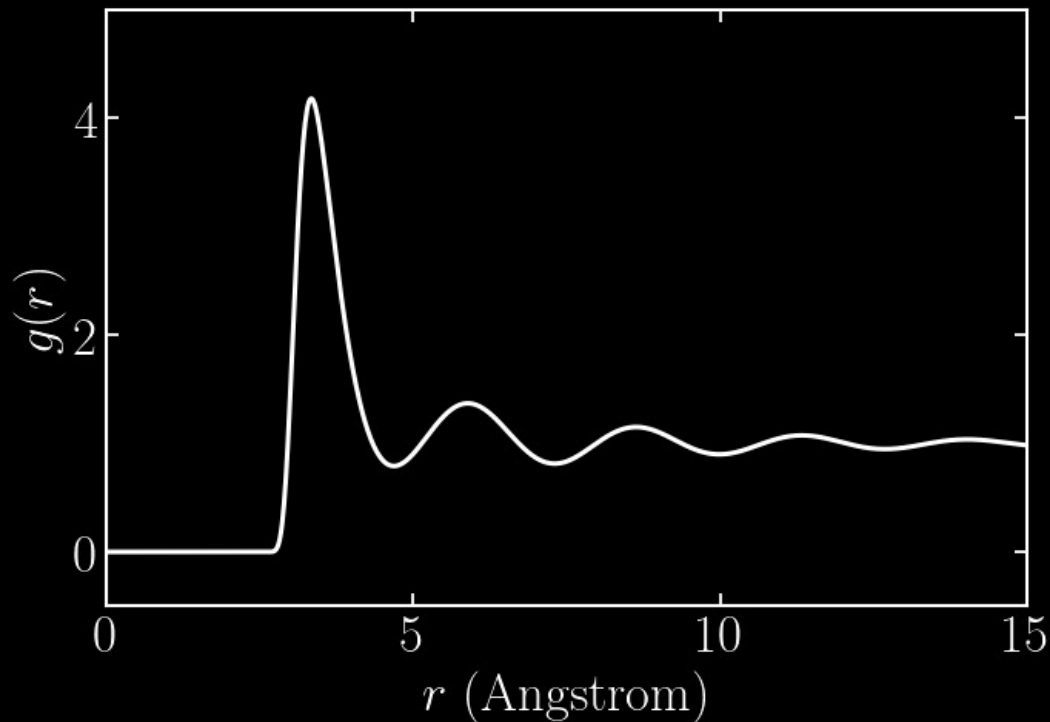
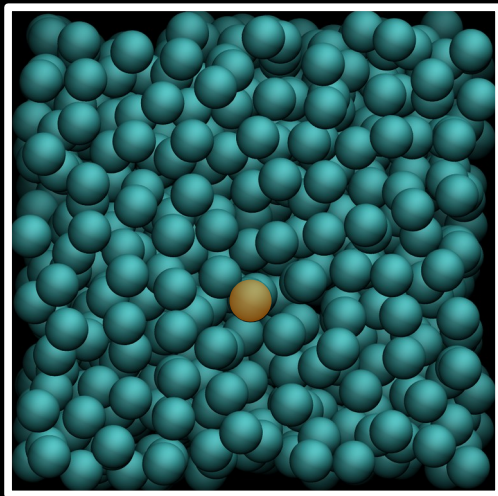
→ how particles are distributed around one another



Probing the structure

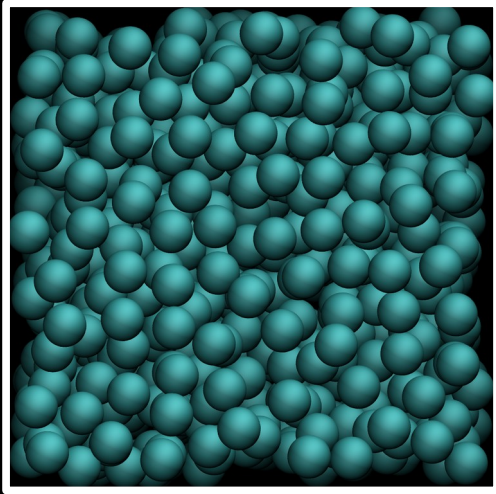
Radial distribution function $g(r) = \frac{\rho(r)}{\rho}$

→ how particles are distributed around one another



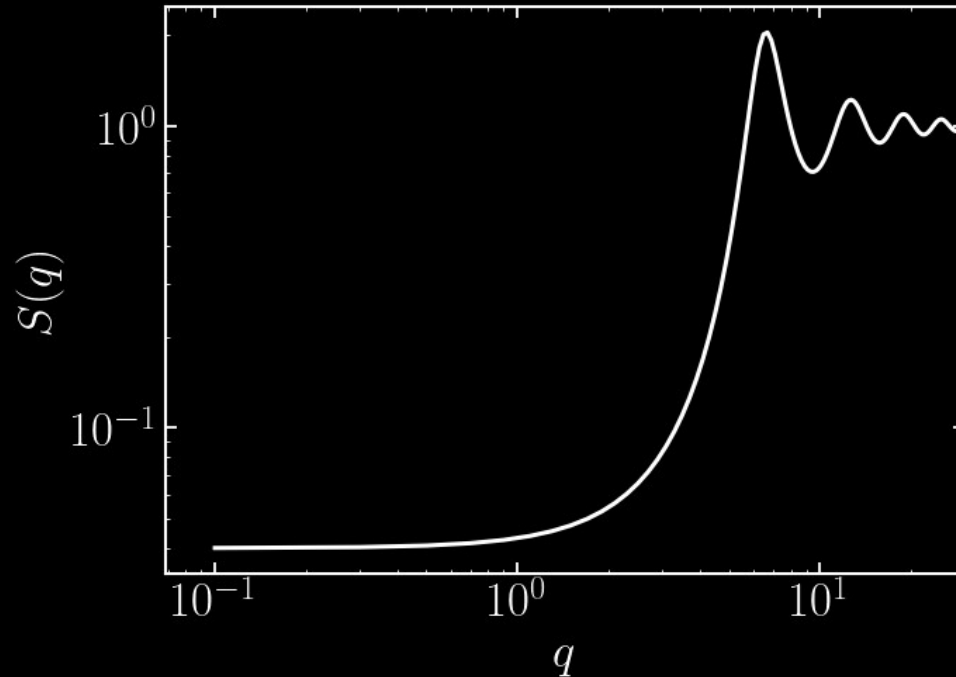
Probing the structure

Structure factor $S(q)$



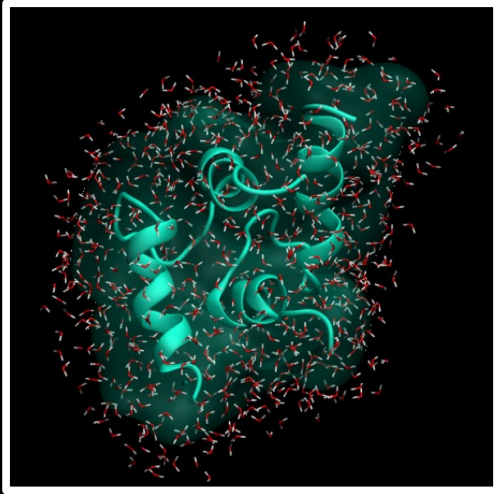
$$S(q) = 1 + \rho \int [g(r) - 1] e^{i\mathbf{q} \cdot \mathbf{r}} d\mathbf{r}$$

$$q = \frac{2\pi}{r}$$



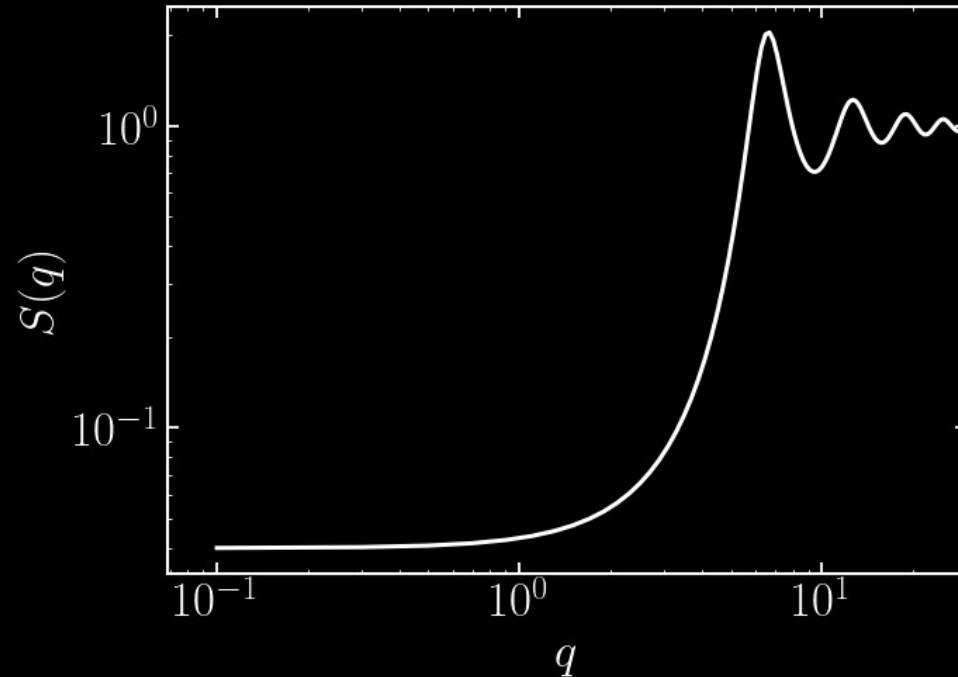
Probing the structure

Structure factor $S(q)$



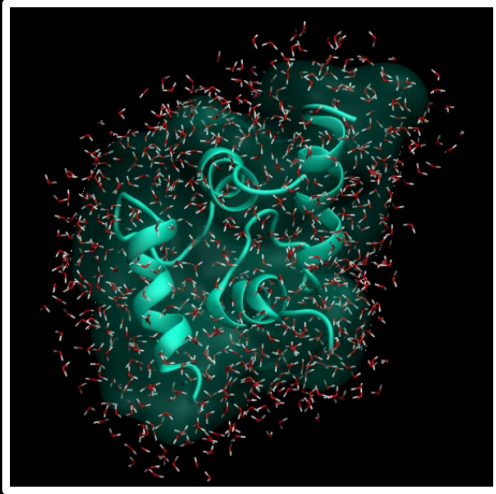
$$S(q) = 1 + \rho \int [g(r) - 1] e^{i\mathbf{q} \cdot \mathbf{r}} d\mathbf{r}$$

$$q = \frac{2\pi}{r}$$



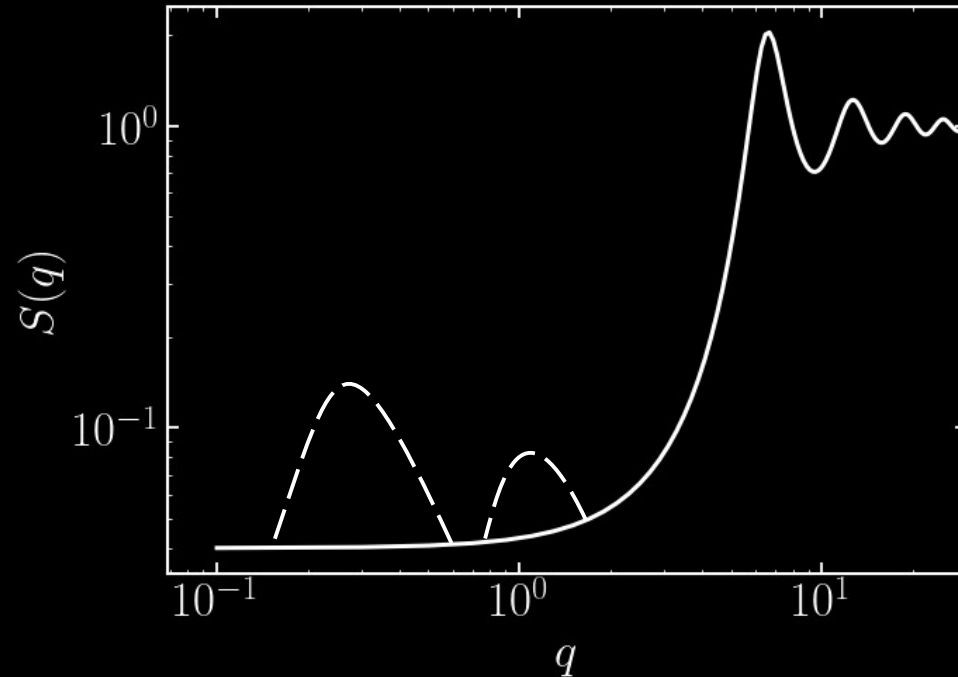
Probing the structure

Structure factor $S(q)$



$$S(q) = 1 + \rho \int [g(r) - 1] e^{i\mathbf{q} \cdot \mathbf{r}} d\mathbf{r}$$

$$q = \frac{2\pi}{r}$$



Probing the dynamics

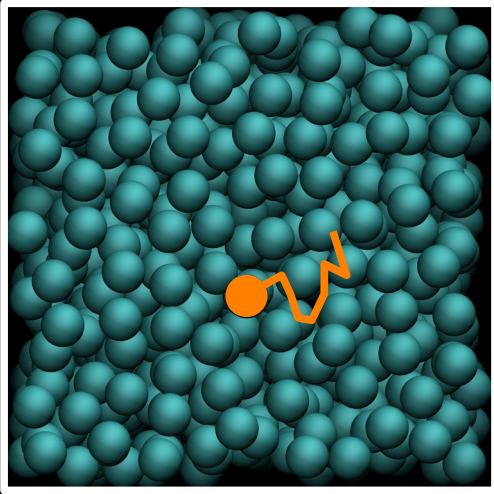
Mean square displacement

$$\text{MSD}(t) = \left\langle |\mathbf{r}_i(t) - \mathbf{r}_i(0)|^2 \right\rangle$$

Probing the dynamics

Mean square displacement

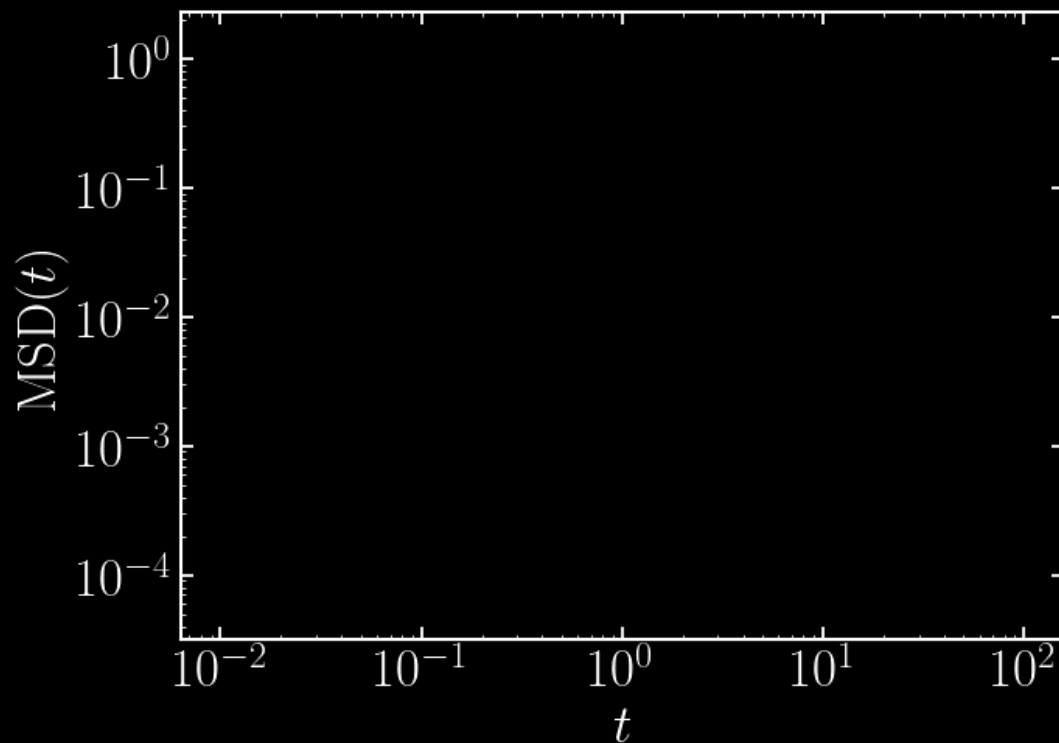
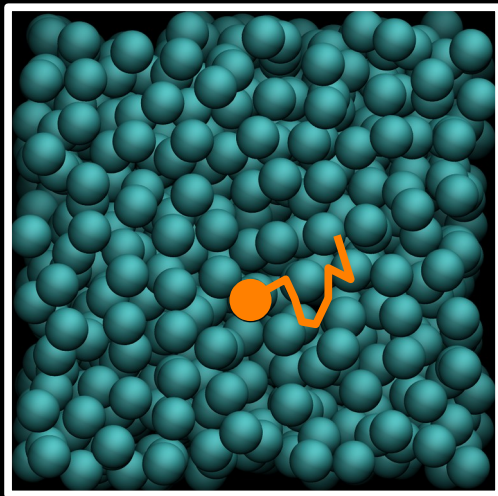
$$\text{MSD}(t) = \left\langle |\mathbf{r}_i(t) - \mathbf{r}_i(0)|^2 \right\rangle$$



Probing the dynamics

Mean square displacement

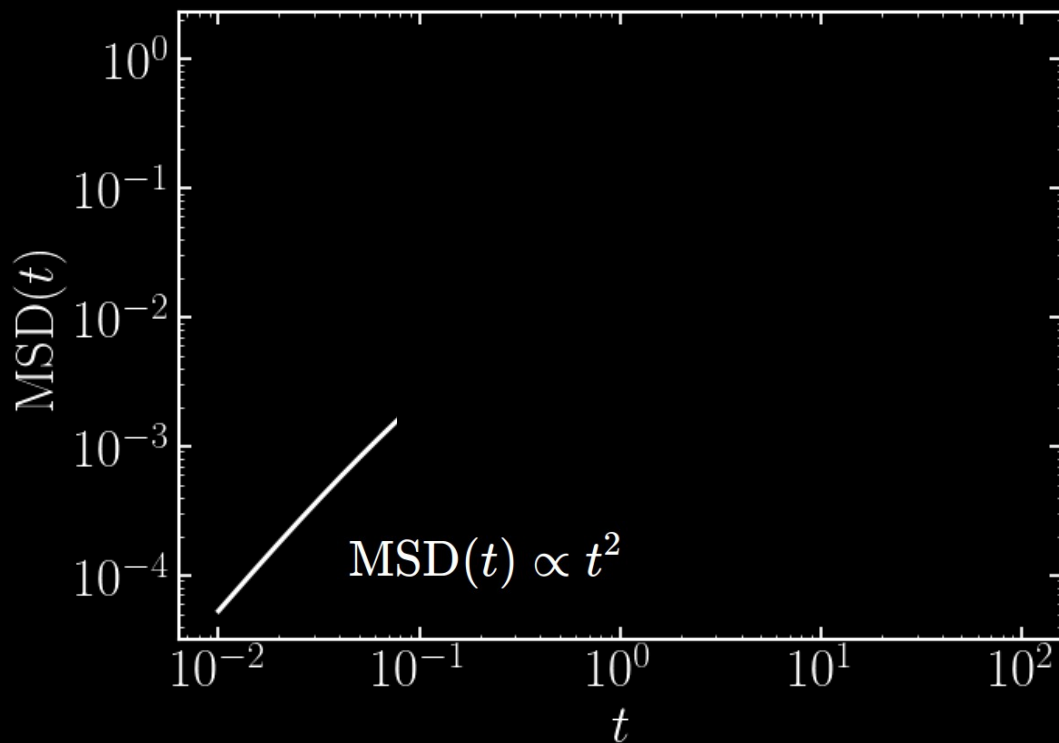
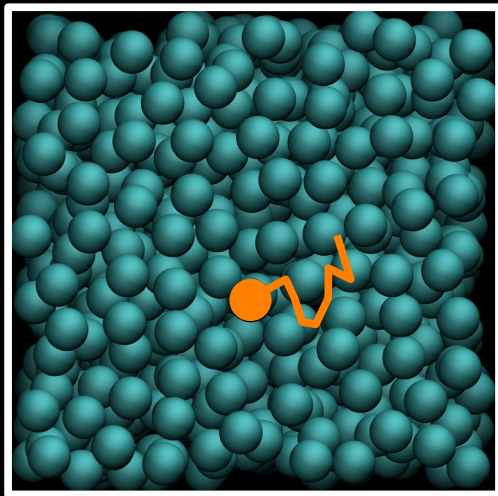
$$\text{MSD}(t) = \langle |\mathbf{r}_i(t) - \mathbf{r}_i(0)|^2 \rangle$$



Probing the dynamics

Mean square displacement

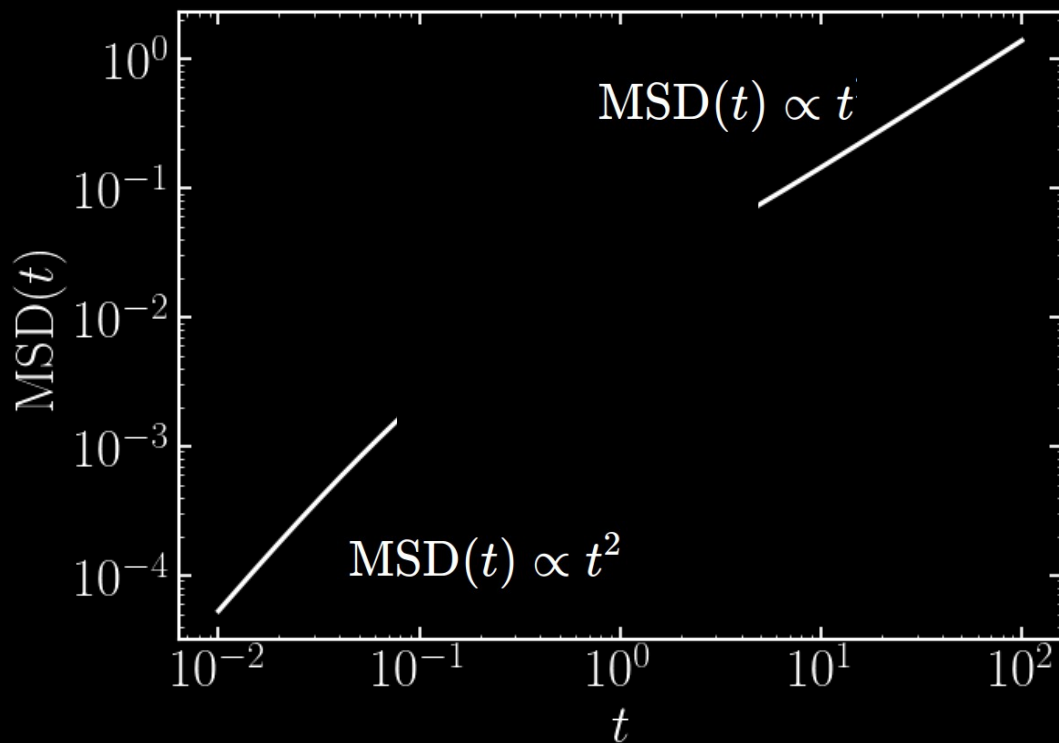
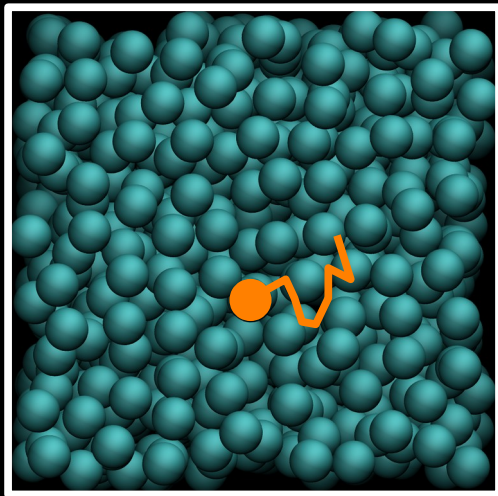
$$\text{MSD}(t) = \langle |\mathbf{r}_i(t) - \mathbf{r}_i(0)|^2 \rangle$$



Probing the dynamics

Mean square displacement

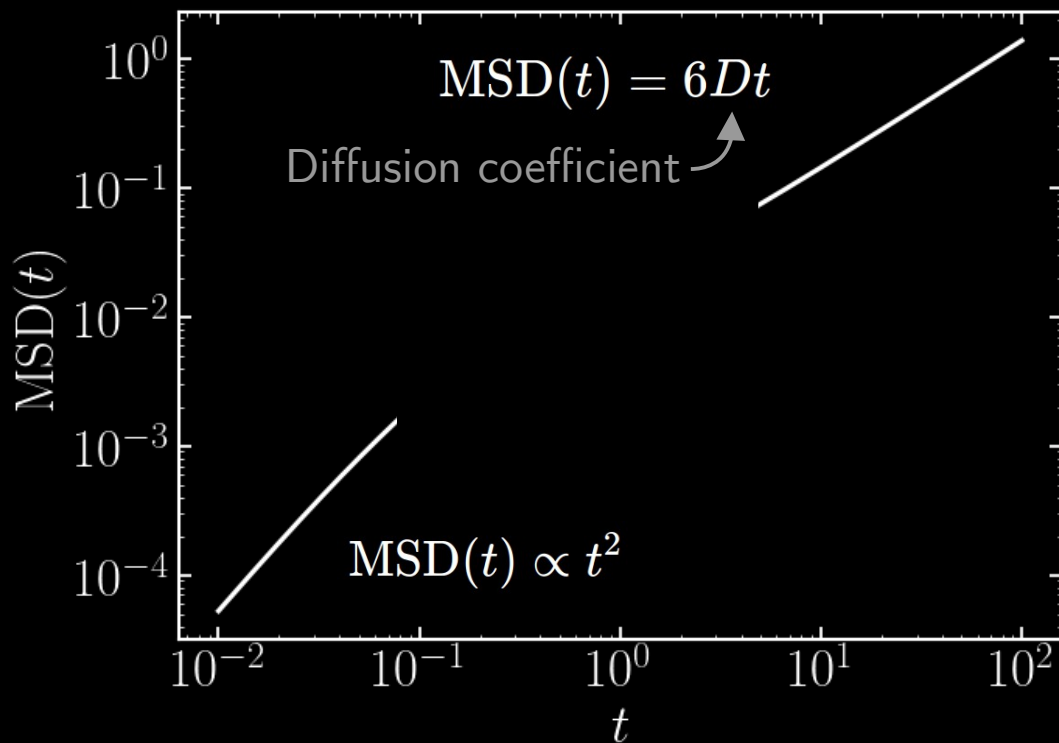
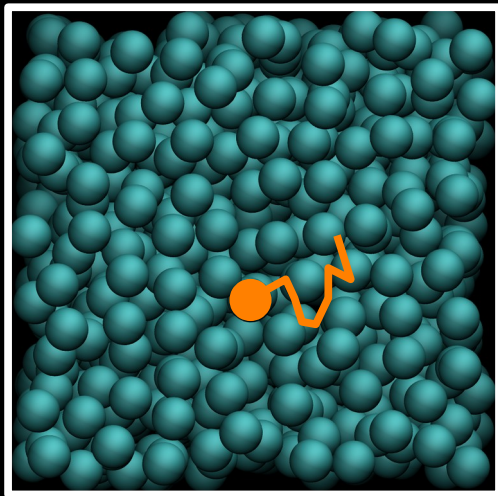
$$\text{MSD}(t) = \langle |\mathbf{r}_i(t) - \mathbf{r}_i(0)|^2 \rangle$$



Probing the dynamics

Mean square displacement

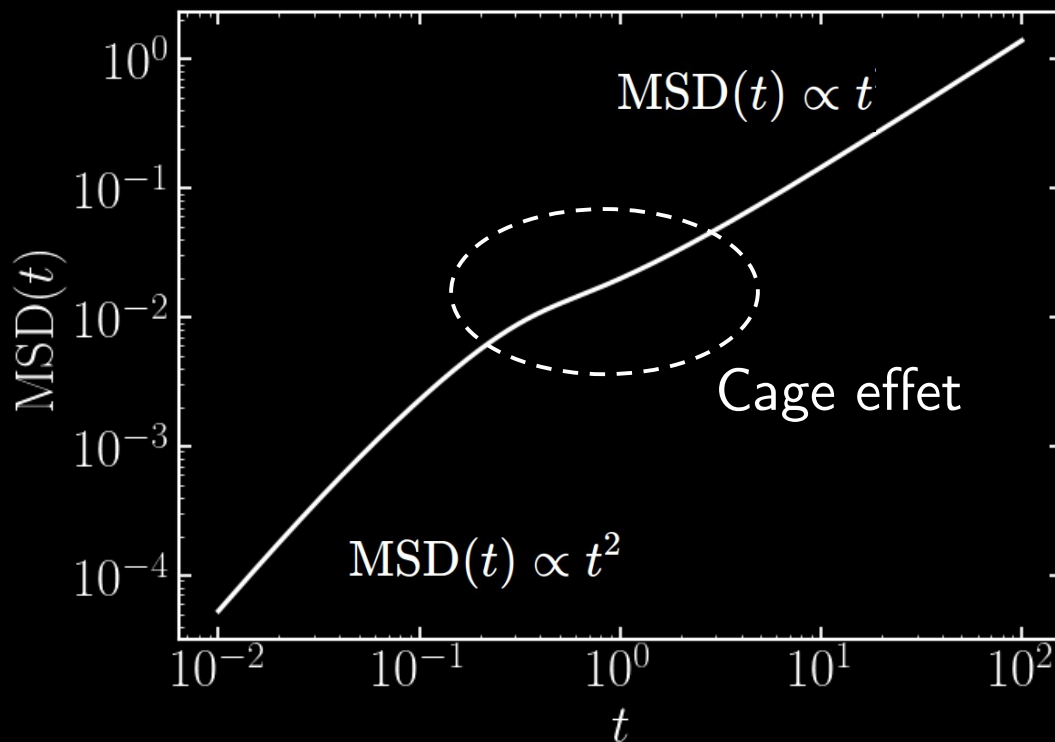
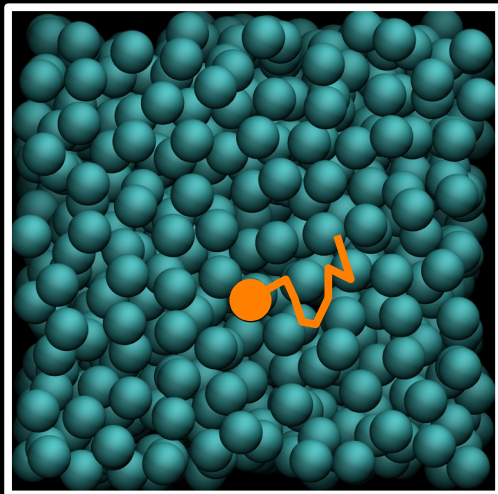
$$\text{MSD}(t) = \langle |\mathbf{r}_i(t) - \mathbf{r}_i(0)|^2 \rangle$$



Probing the dynamics

Mean square displacement

$$\text{MSD}(t) = \langle |\mathbf{r}_i(t) - \mathbf{r}_i(0)|^2 \rangle$$



Viscosity measurement

Green-Kubo (equilibrium)

Shear (non-equilibrium)

Viscosity measurement

Green-Kubo (equilibrium)

Measure spontaneous fluctuations of P_{xy}

P_{xy} : off-diagonal stress tensor

Shear (non-equilibrium)

Viscosity measurement

Green-Kubo (equilibrium)

Measure spontaneous fluctuations of P_{xy}

P_{xy} : off-diagonal stress tensor

$$\eta = \frac{V}{k_B T} \int_0^\infty \langle P_{xy}(0) P_{xy}(t) \rangle dt$$

viscosity

Shear (non-equilibrium)

Viscosity measurement

Green-Kubo (equilibrium)

Measure spontaneous fluctuations of P_{xy}

P_{xy} : off-diagonal stress tensor

$$\eta = \frac{V}{k_B T} \int_0^\infty \langle P_{xy}(0) P_{xy}(t) \rangle dt$$

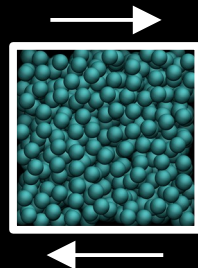
viscosity

Shear (non-equilibrium)

Impose a flow

$$v_x(y) = \dot{\gamma} y$$

Shear rate (1/s)



Viscosity measurement

Green-Kubo (equilibrium)

Measure spontaneous fluctuations of P_{xy}

P_{xy} : off-diagonal stress tensor

$$\eta = \frac{V}{k_B T} \int_0^\infty \langle P_{xy}(0) P_{xy}(t) \rangle dt$$

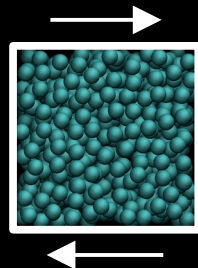
viscosity

Shear (non-equilibrium)

Impose a flow

$$v_x(y) = \dot{\gamma} y$$

Shear rate (1/s)

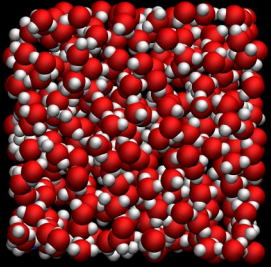


Measure the stress response

$$\sigma_{xy} = \eta \dot{\gamma} \quad \eta = \frac{\sigma_{xy}}{\dot{\gamma}}$$

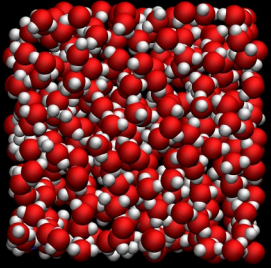
Simulating systems more complex than argon

Simulating systems more complex than argon

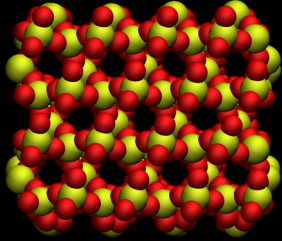


1) Ions, molecules, proteins, etc

Simulating systems more complex than argon

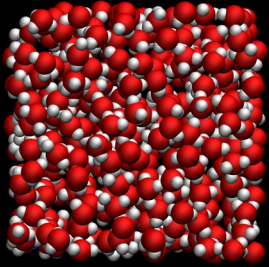


1) Ions, molecules, proteins, etc

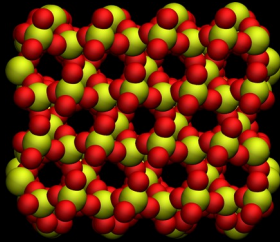


2) Metals and covalent materials

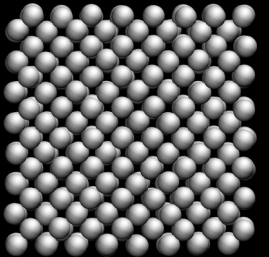
Simulating systems more complex than argon



1) Ions, molecules, proteins, etc

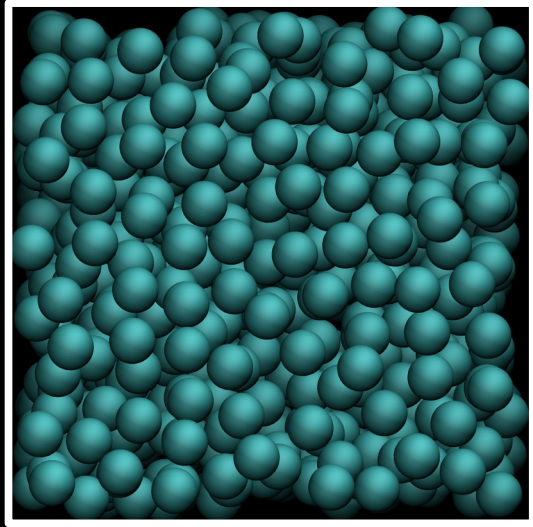


2) Metals and covalent materials

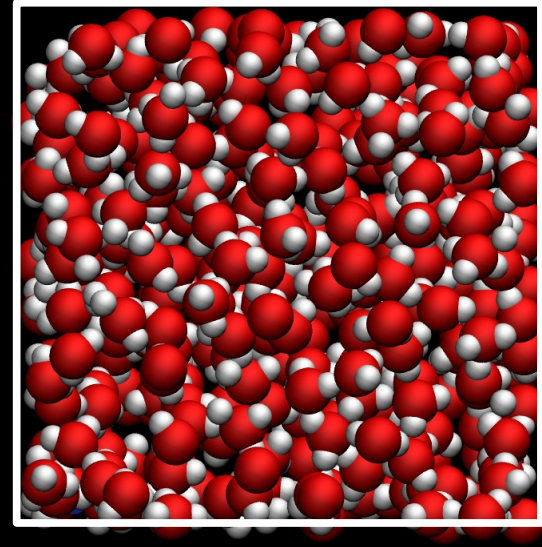


3) About MLPI

Simulating ions, molecules, proteins, ...

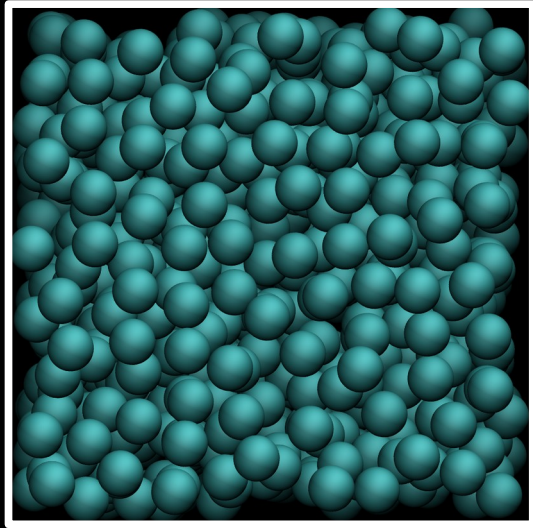


Simple fluid

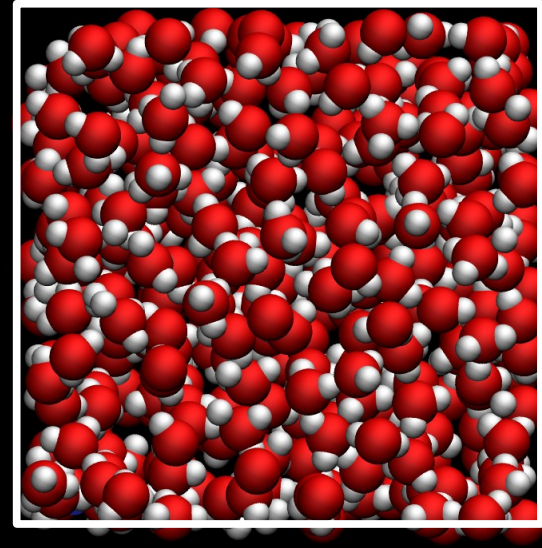


Water

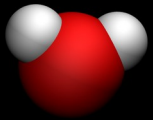
Simulating ions, molecules, proteins, ...



Simple fluid

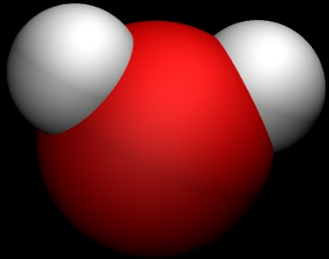


Water



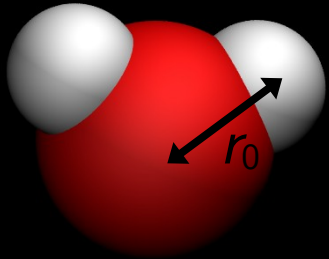
Simulating ions, molecules, proteins, ...

1) Connect atoms by bonds



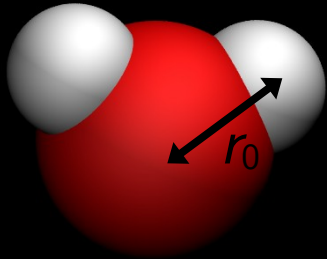
Simulating ions, molecules, proteins, ...

1) Connect atoms by bonds



Simulating ions, molecules, proteins, ...

1) Connect atoms by bonds

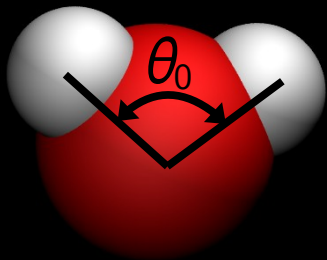


$$U_{\text{bond}}(r) = \frac{1}{2}k(r - r_0)^2$$

bond stiffness

Simulating ions, molecules, proteins, ...

1) Connect atoms by bonds



$$U_{\text{bond}}(r) = \frac{1}{2}k(r - r_0)^2$$

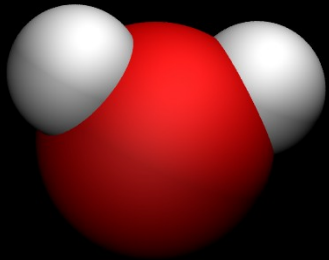
$$U_{\text{angle}}(\theta) = \frac{1}{2}k_{\theta}(\theta - \theta_0)^2$$

angle bending constant

Simulating ions, molecules, proteins, ...

1) Connect atoms by bonds

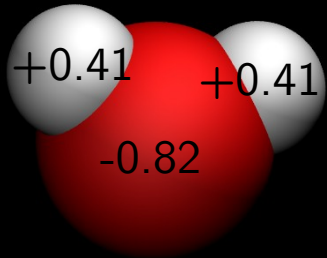
2) Attribute partial charges to atoms



Simulating ions, molecules, proteins, ...

1) Connect atoms by bonds

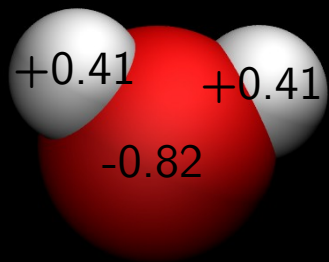
2) Attribute partial charges to atoms



Simulating ions, molecules, proteins, ...

1) Connect atoms by bonds

2) Attribute partial charges to atoms

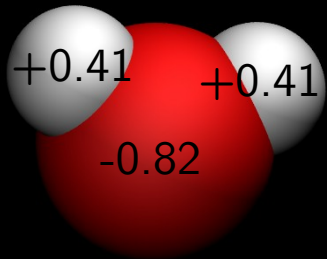


- Represent average redistribution of electron density
- Help reproduce the molecule's electrostatic behavior

Simulating ions, molecules, proteins, ...

1) Connect atoms by bonds

2) Attribute partial charges to atoms



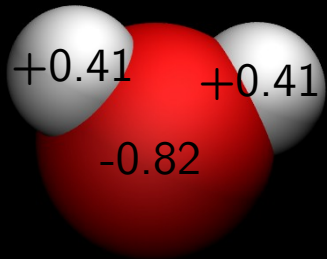
- Represent average redistribution of electron density
- Help reproduce the molecule's electrostatic behavior
- Additional potential :

$$U_{\text{elec}} = \frac{1}{4\pi\epsilon_0} \frac{q_i q_j}{r_{ij}}$$

Simulating ions, molecules, proteins, ...

1) Connect atoms by bonds

2) Attribute partial charges to atoms



- Represent average redistribution of electron density
- Help reproduce the molecule's electrostatic behavior
- Additional potential :

$$U_{\text{elec}} = \frac{1}{4\pi\epsilon_0} \frac{q_i q_j}{r_{ij}}$$

→ long ranged
(Ewald-based methods)

Ewald sum

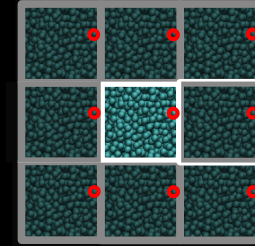
Coulomb decays slowly :

$$U_{\text{elec}} = \frac{1}{4\pi\epsilon_0} \frac{q_i q_j}{r_{ij}}$$

Ewald sum

Coulomb decays slowly :

$$U_{\text{elec}} = \frac{1}{4\pi\epsilon_0} \frac{q_i q_j}{r_{ij}}$$



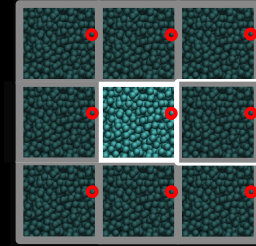
Ewald sum

Coulomb decays slowly :

$$U_{\text{elec}} = \frac{1}{4\pi\epsilon_0} \frac{q_i q_j}{r_{ij}}$$

Ewald decomposes :

$$\frac{1}{r} = \frac{\text{erfc}(\alpha r)}{r} + \frac{\text{erf}(\alpha r)}{r}$$



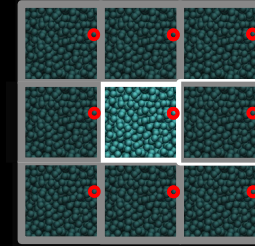
Ewald sum

Coulomb decays slowly :

$$U_{\text{elec}} = \frac{1}{4\pi\epsilon_0} \frac{q_i q_j}{r_{ij}}$$

Ewald decomposes :

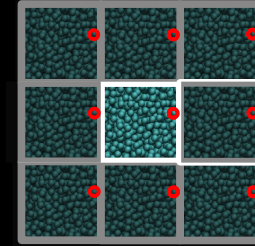
$$\frac{1}{r} = \underbrace{\frac{\text{erfc}(\alpha r)}{r}}_{\text{Short range}} + \underbrace{\frac{\text{erf}(\alpha r)}{r}}_{\text{Long range}}$$



Ewald sum

Coulomb decays slowly :

$$U_{\text{elec}} = \frac{1}{4\pi\epsilon_0} \frac{q_i q_j}{r_{ij}}$$



Ewald decomposes :

$$\frac{1}{r} = \frac{\text{erfc}(\alpha r)}{r} + \frac{\text{erf}(\alpha r)}{r}$$

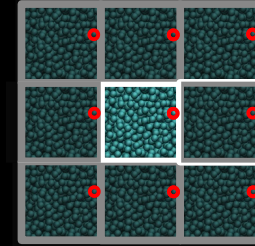
and $\rho(\mathbf{r}) = \sum_{\mathbf{k}} \rho_{\mathbf{k}} e^{i\mathbf{k} \cdot \mathbf{r}}$

$$\mathcal{F} \left[\frac{\text{erf}(\alpha r)}{r} \right] = \frac{4\pi}{k^2} e^{-k^2/(4\alpha^2)}$$

Ewald sum

Coulomb decays slowly :

$$U_{\text{elec}} = \frac{1}{4\pi\epsilon_0} \frac{q_i q_j}{r_{ij}}$$



Ewald decomposes :

$$\frac{1}{r} = \frac{\text{erfc}(\alpha r)}{r} + \frac{\text{erf}(\alpha r)}{r}$$

and $\rho(\mathbf{r}) = \sum_{\mathbf{k}} \rho_{\mathbf{k}} e^{i\mathbf{k} \cdot \mathbf{r}}$

$$\mathcal{F} \left[\frac{\text{erf}(\alpha r)}{r} \right] = \frac{4\pi}{k^2} e^{-k^2/(4\alpha^2)}$$

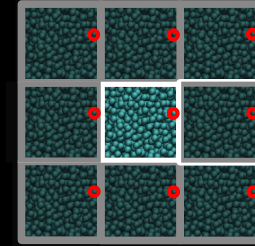
In reciprocal space

$$U_{\text{recip}} = \frac{1}{2V\epsilon_0} \sum_{\mathbf{k} \neq 0} \frac{e^{-k^2/(4\alpha^2)}}{k^2} \left| \sum_i q_i e^{i\mathbf{k} \cdot \mathbf{r}_i} \right|^2$$

Ewald sum

Coulomb decays slowly :

$$U_{\text{elec}} = \frac{1}{4\pi\epsilon_0} \frac{q_i q_j}{r_{ij}}$$



Ewald decomposes :

$$\frac{1}{r} = \frac{\text{erfc}(\alpha r)}{r} + \frac{\text{erf}(\alpha r)}{r}$$

and $\rho(\mathbf{r}) = \sum_{\mathbf{k}} \rho_{\mathbf{k}} e^{i\mathbf{k} \cdot \mathbf{r}}$

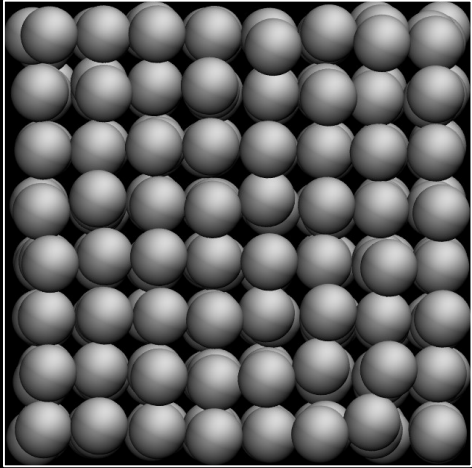
$$\mathcal{F} \left[\frac{\text{erf}(\alpha r)}{r} \right] = \frac{4\pi}{k^2} e^{-k^2/(4\alpha^2)}$$

In reciprocal space

$$U_{\text{recip}} = \frac{1}{2V\epsilon_0} \sum_{\mathbf{k} \neq 0} \left(\frac{e^{-k^2/(4\alpha^2)}}{k^2} \right) \left| \sum_i q_i e^{i\mathbf{k} \cdot \mathbf{r}_i} \right|^2$$

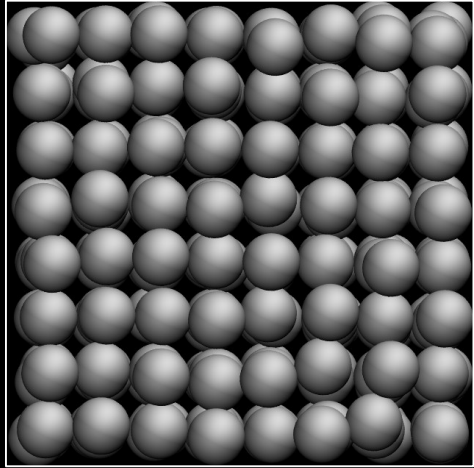
reciprocal-space sum converge very rapidly

Simulating metals



Al (fcc)

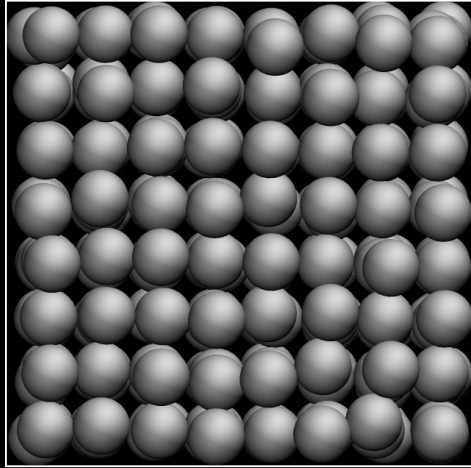
Simulating metals



Al (fcc)

Metals : Bonding depends on the local electronic environment (not just on pairwise distances)

Simulating metals



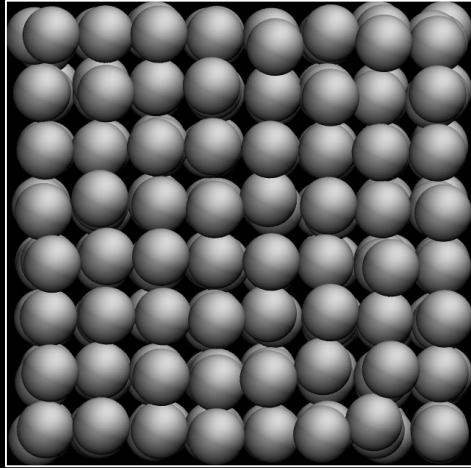
Al (fcc)

Metals : Bonding depends on the local electronic environment (not just on pairwise distances)

Embedded atom model (EAM)

$$U_i = F_i(\rho_i) + \frac{1}{2} \sum_{j \neq i} \phi_{ij}(r_{ij})$$

Simulating metals



Al (fcc)

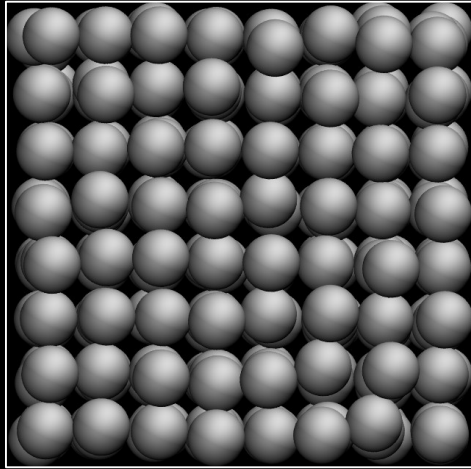
Metals : Bonding depends on the local electronic environment (not just on pairwise distances)

Embedded atom model (EAM)

$$U_i = F_i(\rho_i) + \frac{1}{2} \sum_{j \neq i} \phi_{ij}(r_{ij})$$

Pairwise function

Simulating metals



Al (fcc)

Metals : Bonding depends on the local electronic environment (not just on pairwise distances)

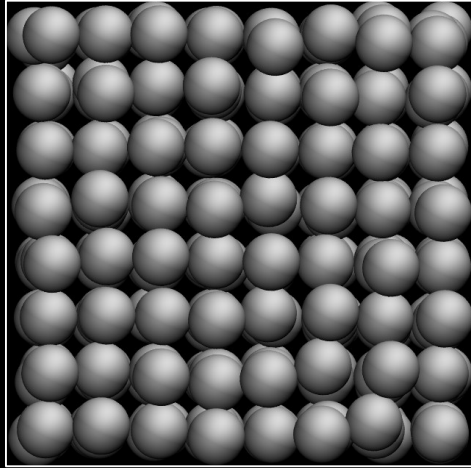
Embedded atom model (EAM)

$$U_i = F_i(\rho_i) + \frac{1}{2} \sum_{j \neq i} \phi_{ij}(r_{ij})$$

Many-body term : Embedding energy
(function of local electron density ρ_i)

Pairwise function

Simulating metals



Al (fcc)

Metals : Bonding depends on the local electronic environment (not just on pairwise distances)

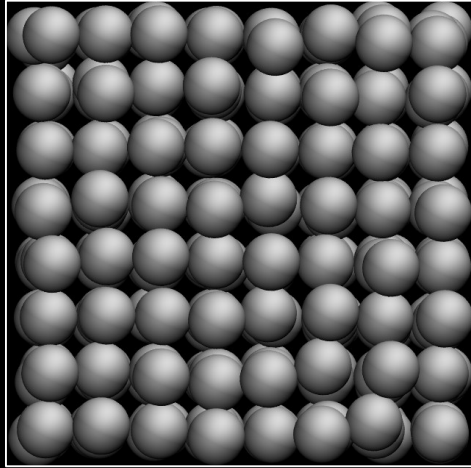
Embedded atom model (EAM)

$$U_i = F_i(\rho_i) + \frac{1}{2} \sum_{j \neq i} \phi_{ij}(r_{ij})$$

$$\rho_i = \sum_{j \neq i} f_j(r_{ij})$$

Electron-density contribution
from atom j

Simulating metals



Al (fcc)

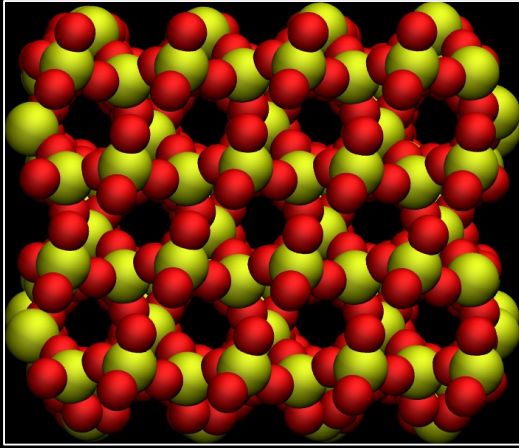
Metals : Bonding depends on the local electronic environment (not just on pairwise distances)

Embedded atom model (EAM)

Many-body interactions

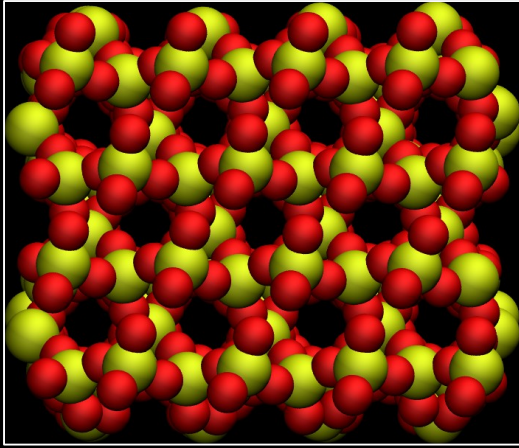
→ coordination, crystal structure, surface, defects

Simulating covalent materials



SiO₂ (Quartz)

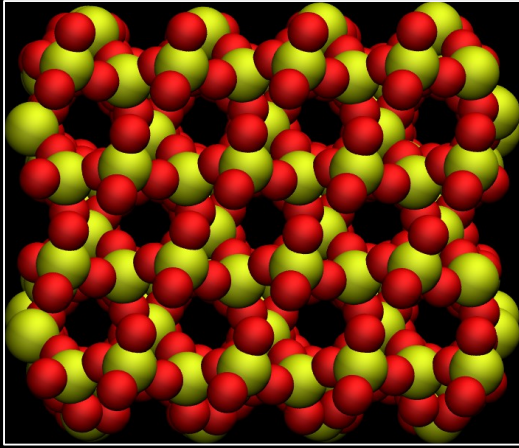
Simulating covalent materials



SiO_2 (Quartz)

Covalent materials : atoms are held together by covalent chemical bonds that are directional

Simulating covalent materials



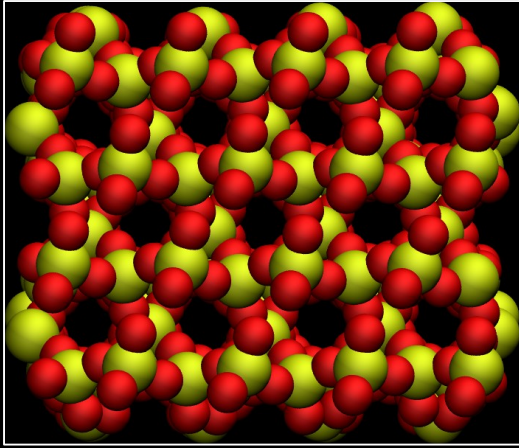
SiO₂ (Quartz)

Covalent materials : atoms are held together by covalent chemical bonds that are directional

Tersoff

$$E = \frac{1}{2} \sum_i \sum_{j \neq i} f_c(r_{ij}) [f_R(r_{ij}) + b_{ij} f_A(r_{ij})]$$

Simulating covalent materials



SiO₂ (Quartz)

Covalent materials : atoms are held together by covalent chemical bonds that are directional

Tersoff

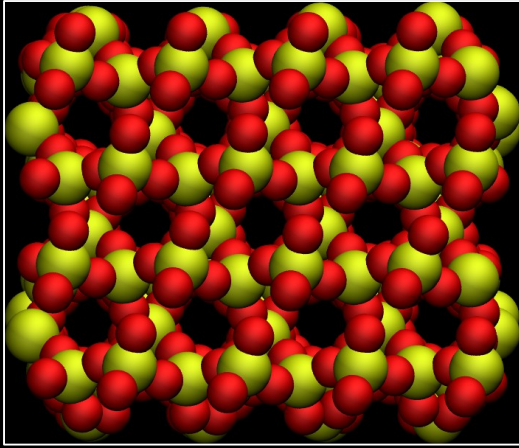
$$E = \frac{1}{2} \sum_i \sum_{j \neq i} f_c(r_{ij}) [f_R(r_{ij}) + b_{ij} f_A(r_{ij})]$$

Attractive term

Bond-order term

Repulsive pairwise term

Simulating covalent materials



SiO₂ (Quartz)

Covalent materials : atoms are held together by covalent chemical bonds that are directional

Tersoff

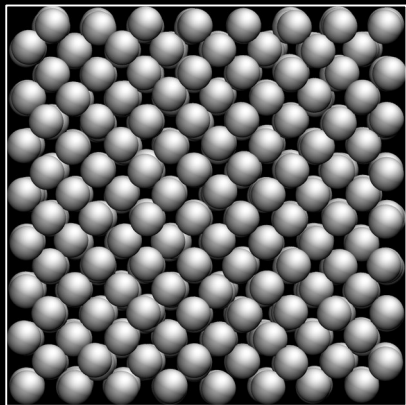
$$E = \frac{1}{2} \sum_i \sum_{j \neq i} f_c(r_{ij}) [f_R(r_{ij}) + b_{ij} f_A(r_{ij})]$$

$$b_{ij} = (1 + \beta^n \zeta_{ij}^n)^{-1/(2n)}$$

$$\zeta_{ij} = \sum_{k \neq i, j} f_c(r_{ik}) g(\theta_{ijk})$$

Angle between 3 atoms

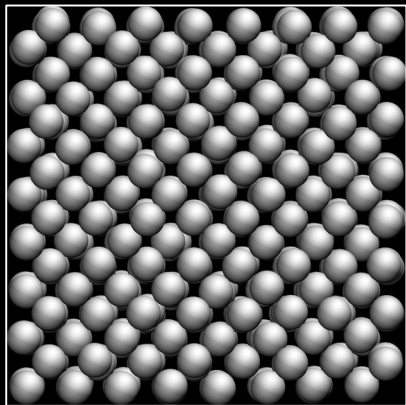
Machine-learning interatomic potentials MLPI



Ge (diamond)

No fixed functional form

Machine-learning interatomic potentials MLPI

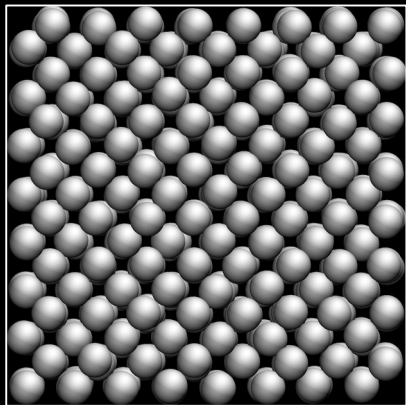


Ge (diamond)

No fixed functional form

Learns the energy landscape from quantum-mechanical data

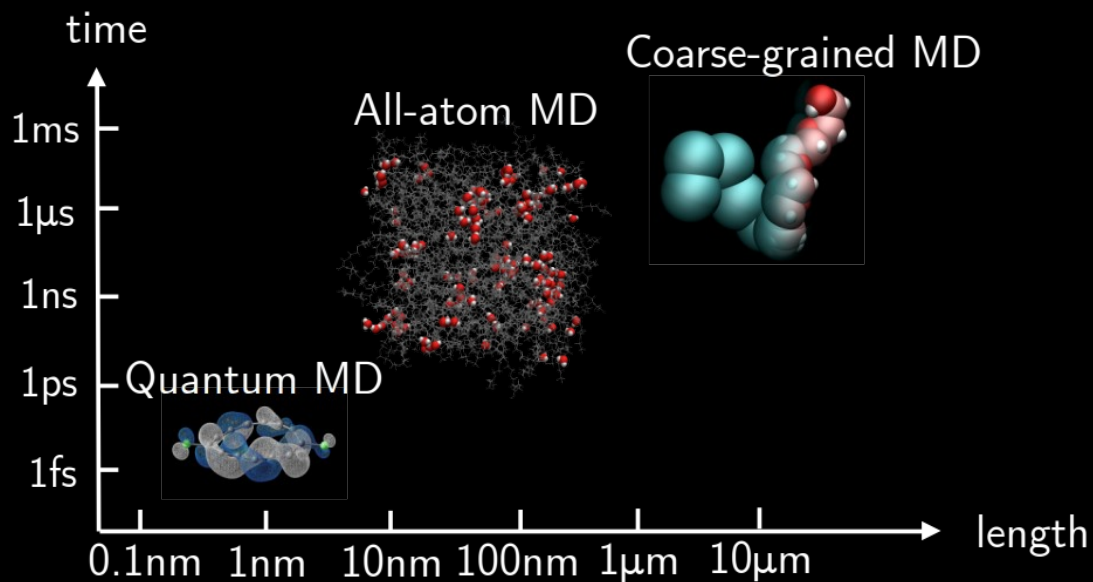
Machine-learning interatomic potentials MLPI



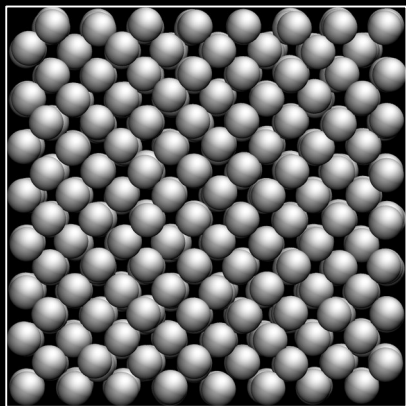
Ge (diamond)

No fixed functional form

Learns the energy landscape from quantum-mechanical data



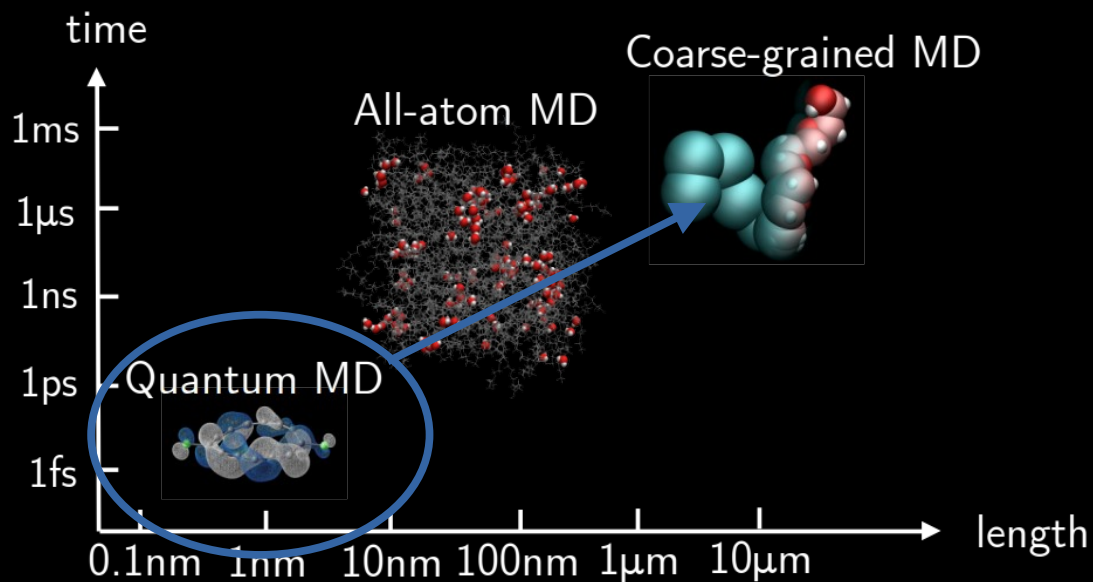
Machine-learning interatomic potentials MLPI



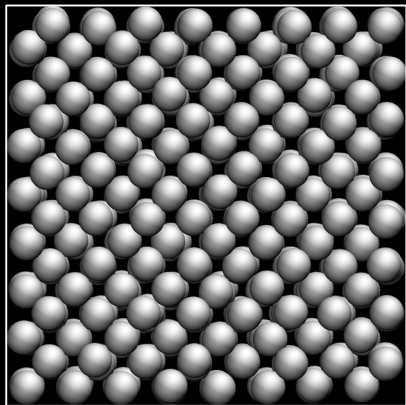
Ge (diamond)

No fixed functional form

Learns the energy landscape from quantum-mechanical data



Machine-learning interatomic potentials MLPI



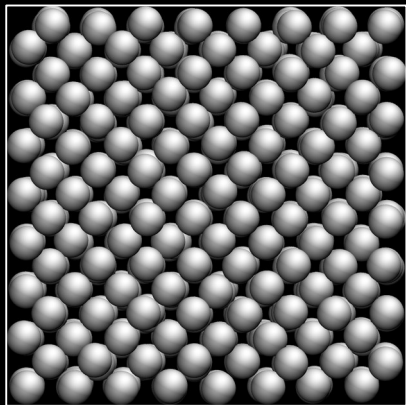
Ge (diamond)

No fixed functional form

Learns the energy landscape from quantum-mechanical data

Not inherently “more accurate”, depends strongly on the training dataset and descriptor

Machine-learning interatomic potentials MLPI



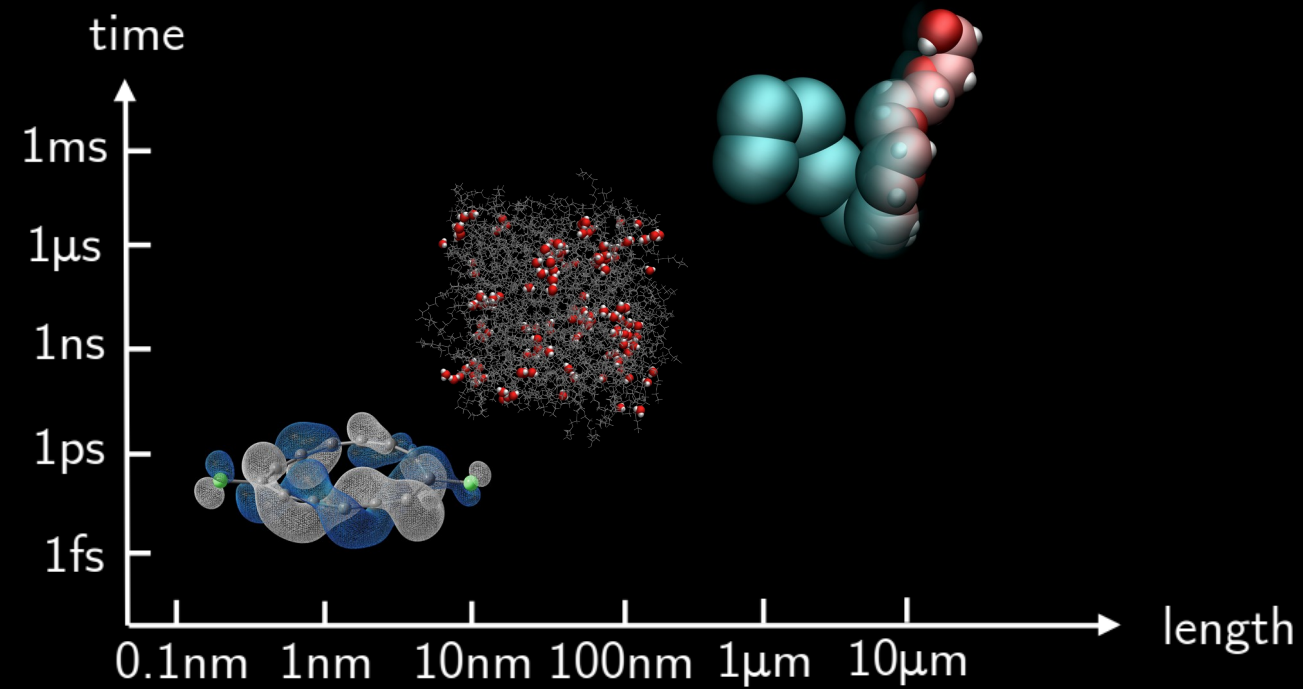
Ge (diamond)

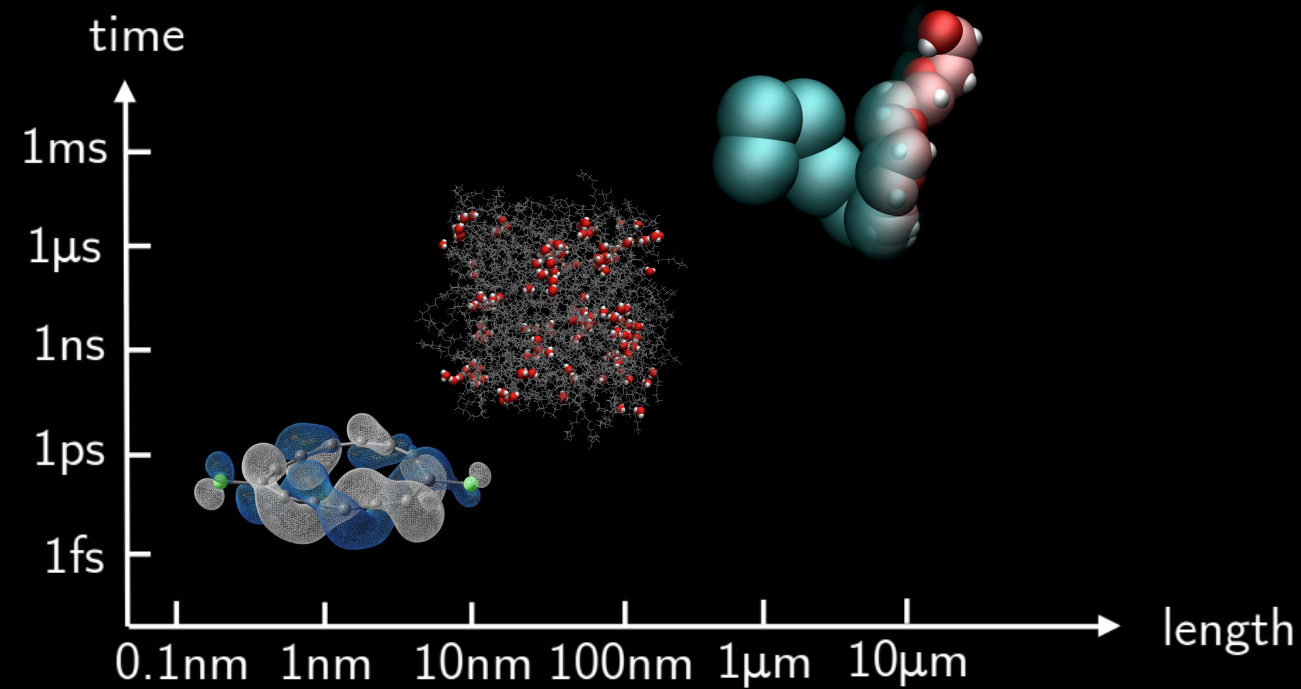
No fixed functional form

Learns the energy landscape from quantum-mechanical data

Not inherently “more accurate”, depends strongly on the training dataset and descriptor

From the LAMMPS user perspective : as easy to use as EAM or Tersoff





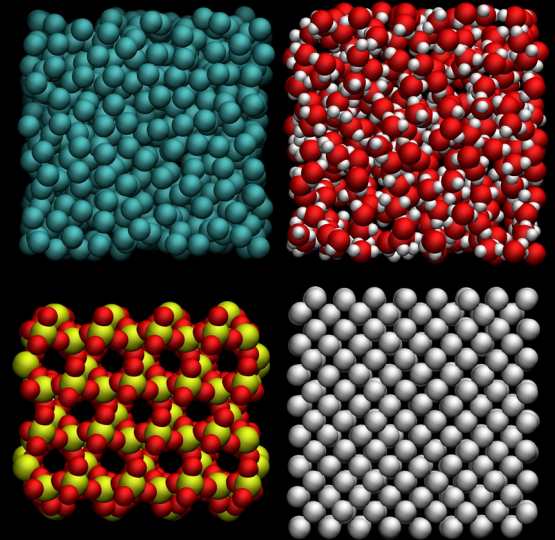
- 1) Define the system
- 2) Measure forces $\mathbf{F}_{ij}$
- 3) Solve Newton's eq
- 4) Move atoms

Takeaway message : MD

- 1) Define the system

Takeaway message : MD

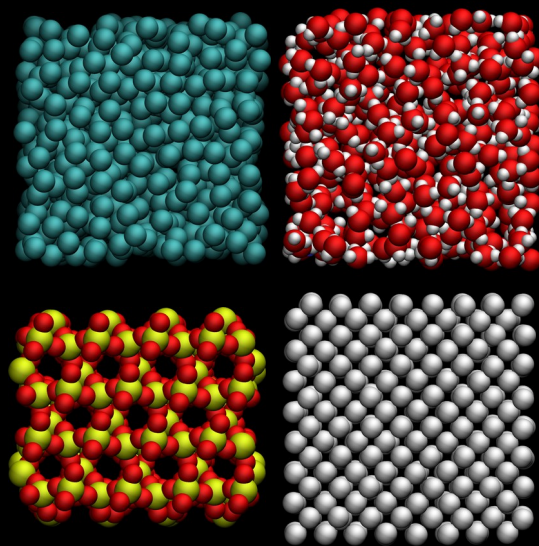
1) Define the system



Takeaway message : MD

1) Define the system

2) Measure forces between atoms $F_{ij} = -\frac{dU(r_{ij})}{dr_{ij}}$



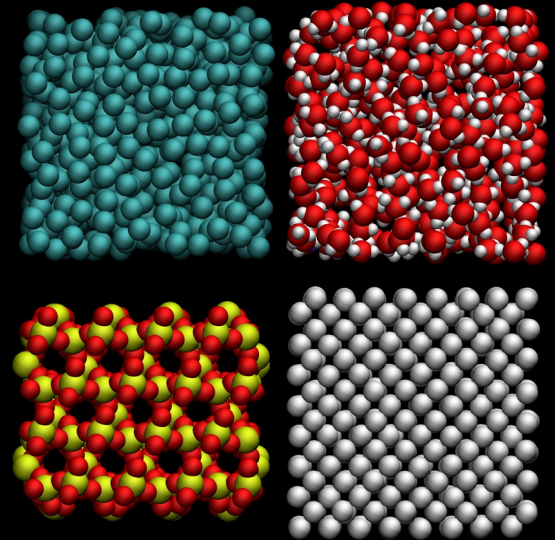
Takeaway message : MD

1) Define the system

2) Measure forces between atoms $F_{ij} = -\frac{dU(r_{ij})}{dr_{ij}}$

$$U_{ij}(r_{ij}) = 4\varepsilon_{ij} \left[\left(\frac{\sigma_{ij}}{r_{ij}} \right)^{12} - \left(\frac{\sigma_{ij}}{r_{ij}} \right)^6 \right] \quad (\text{Lennard-Jones})$$

$$U_i = F_i(\rho_i) + \frac{1}{2} \sum_{j \neq i} \phi_{ij}(r_{ij}) \quad (\text{EAM})$$



Takeaway message : MD

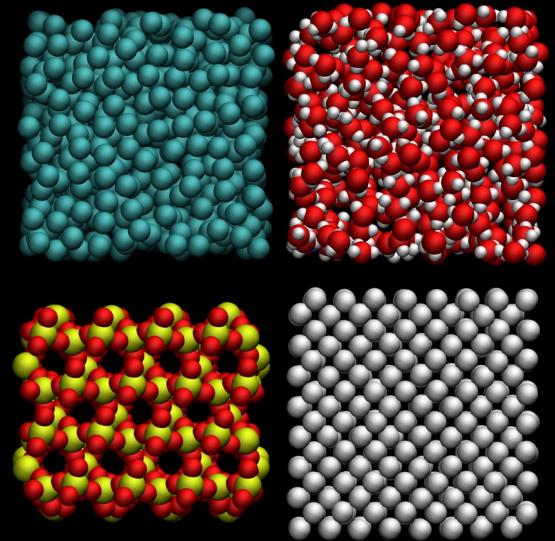
1) Define the system

2) Measure forces between atoms $F_{ij} = -\frac{dU(r_{ij})}{dr_{ij}}$

$$U_{ij}(r_{ij}) = 4\varepsilon_{ij} \left[\left(\frac{\sigma_{ij}}{r_{ij}} \right)^{12} - \left(\frac{\sigma_{ij}}{r_{ij}} \right)^6 \right] \quad (\text{Lennard-Jones})$$

$$U_i = F_i(\rho_i) + \frac{1}{2} \sum_{j \neq i} \phi_{ij}(r_{ij}) \quad (\text{EAM})$$

3) Solve Newton's equations of motion



Takeaway message : MD

1) Define the system

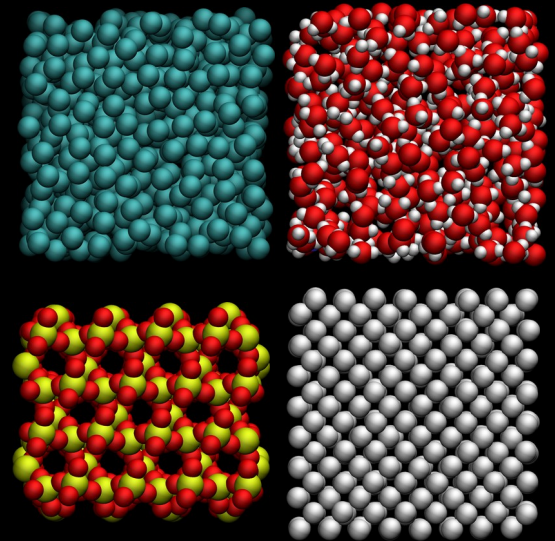
2) Measure forces between atoms $F_{ij} = -\frac{dU(r_{ij})}{dr_{ij}}$

$$U_{ij}(r_{ij}) = 4\epsilon_{ij} \left[\left(\frac{\sigma_{ij}}{r_{ij}} \right)^{12} - \left(\frac{\sigma_{ij}}{r_{ij}} \right)^6 \right] \quad (\text{Lennard-Jones})$$

$$U_i = F_i(\rho_i) + \frac{1}{2} \sum_{j \neq i} \phi_{ij}(r_{ij}) \quad (\text{EAM})$$

3) Solve Newton's equations of motion

4) Move atoms using Verlet algorithm $\mathbf{r}(t + \Delta t) = \mathbf{r}(t) + \dot{\mathbf{r}}(t)\Delta t + \frac{1}{2}\ddot{\mathbf{r}}(t)\Delta t^2$



Takeaway message : MD

1) Define the system

2) Measure forces between atoms $F_{ij} = -\frac{dU(r_{ij})}{dr_{ij}}$

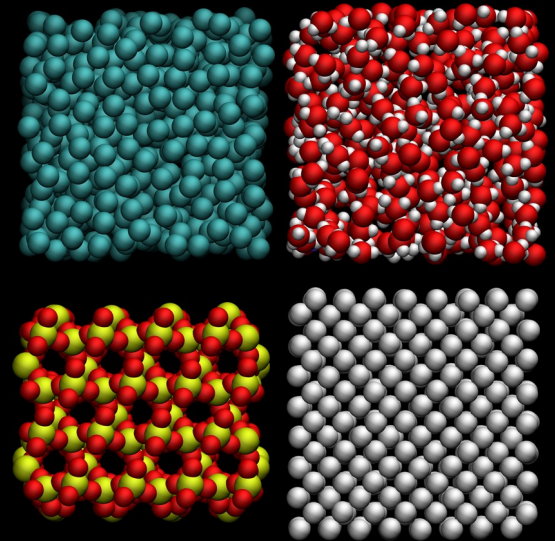
$$U_{ij}(r_{ij}) = 4\varepsilon_{ij} \left[\left(\frac{\sigma_{ij}}{r_{ij}} \right)^{12} - \left(\frac{\sigma_{ij}}{r_{ij}} \right)^6 \right] \quad (\text{Lennard-Jones})$$

$$U_i = F_i(\rho_i) + \frac{1}{2} \sum_{j \neq i} \phi_{ij}(r_{ij}) \quad (\text{EAM})$$

3) Solve Newton's equations of motion

4) Move atoms using Verlet algorithm $\mathbf{r}(t + \Delta t) = \mathbf{r}(t) + \dot{\mathbf{r}}(t)\Delta t + \frac{1}{2}\ddot{\mathbf{r}}(t)\Delta t^2$

5) Optional : Apply thermostat or barostat $\rightarrow$ NVE, NVT, NPT ensemble



Takeaway message : MD

1) Define the system

2) Measure forces between atoms $F_{ij} = -\frac{dU(r_{ij})}{dr_{ij}}$

$$U_{ij}(r_{ij}) = 4\varepsilon_{ij} \left[\left(\frac{\sigma_{ij}}{r_{ij}} \right)^{12} - \left(\frac{\sigma_{ij}}{r_{ij}} \right)^6 \right] \quad (\text{Lennard-Jones})$$

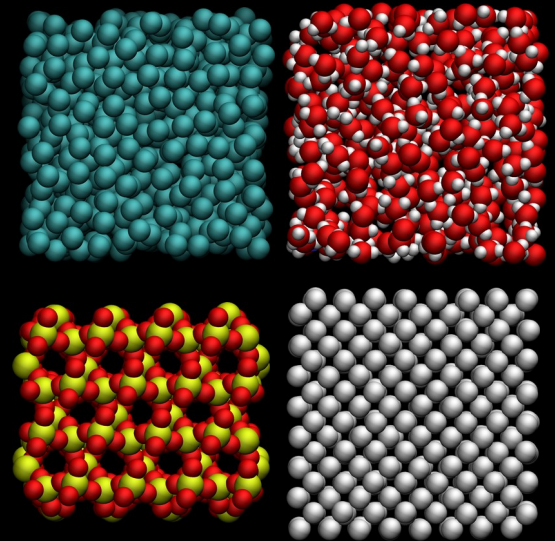
$$U_i = F_i(\rho_i) + \frac{1}{2} \sum_{j \neq i} \phi_{ij}(r_{ij}) \quad (\text{EAM})$$

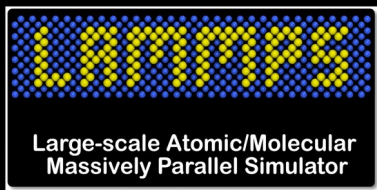
3) Solve Newton's equations of motion

4) Move atoms using Verlet algorithm $\mathbf{r}(t + \Delta t) = \mathbf{r}(t) + \dot{\mathbf{r}}(t)\Delta t + \frac{1}{2}\ddot{\mathbf{r}}(t)\Delta t^2$

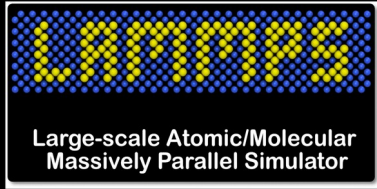
5) Optional : Apply thermostat or barostat $\rightarrow$ NVE, NVT, NPT ensemble

$\rightarrow$ Extract observables from trajectory (energies, structure, dynamics, etc.)



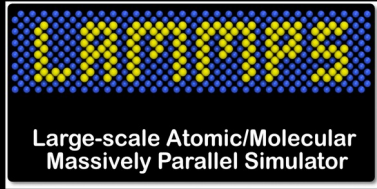


Why LAMMPS ?



Why LAMMPS ?

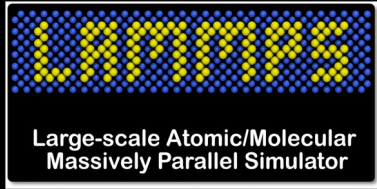
Open-source and written in C++



Why LAMMPS ?

Open-source and written in C++

Large and active community + good documentation

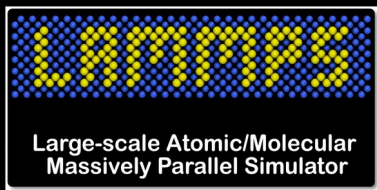


Why LAMMPS ?

Open-source and written in C++

Large and active community + good documentation

Scales from simple to large simulations

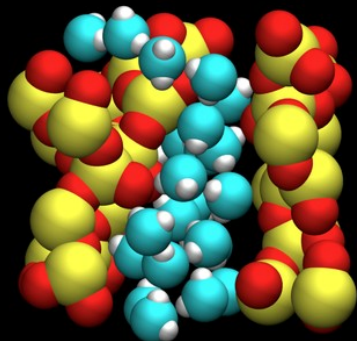
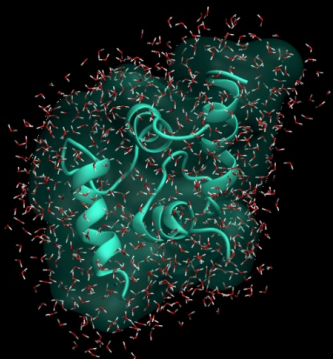


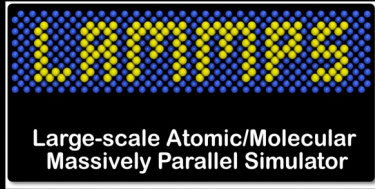
Why LAMMPS ?

Open-source and written in C++

Large and active community + good documentation

Scales from simple to large simulations





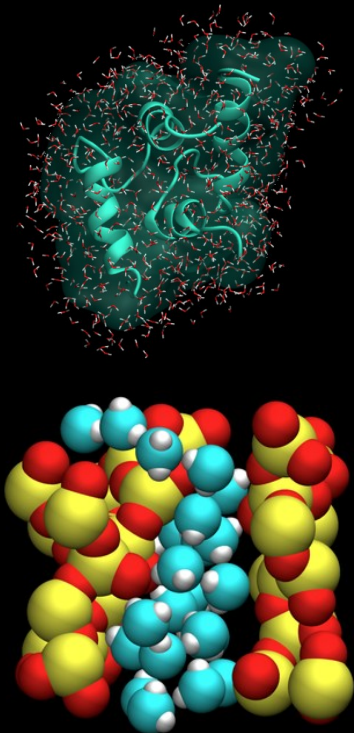
Why LAMMPS ?

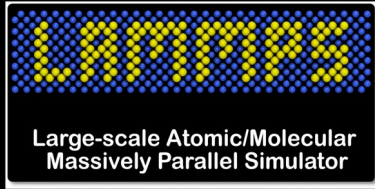
Open-source and written in C++

Large and active community + good documentation

Scales from simple to large simulations

Many implemented potential (LJ, EAM, MLPI, ...)





Why LAMMPS ?

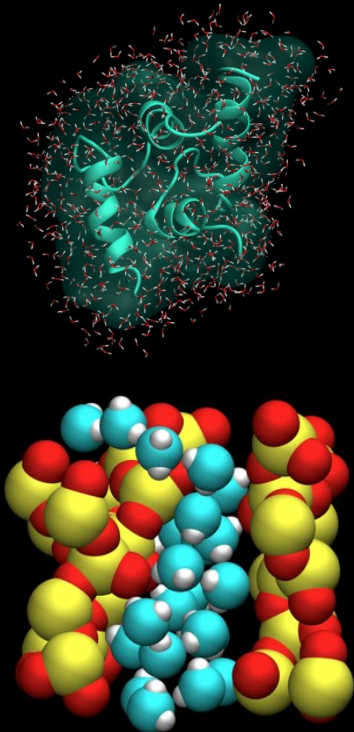
Open-source and written in C++

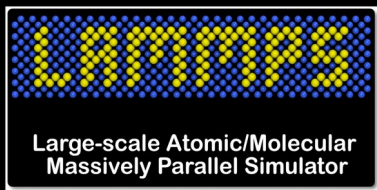
Large and active community + good documentation

Scales from simple to large simulations

Many implemented potential (LJ, EAM, MLPI, ...)

Graphical and visualization tools





Why LAMMPS ?

Open-source and written in C++

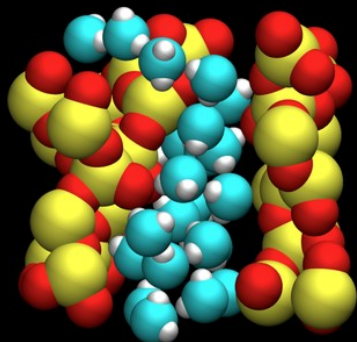
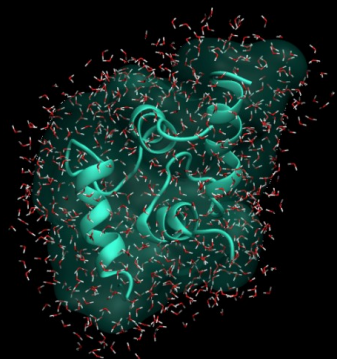
Large and active community + good documentation

Scales from simple to large simulations

Many implemented potential (LJ, EAM, MLPI, ...)

Graphical and visualization tools

Efficient parallelization



Interact with LAMMPS using input

```
# PART A - ENERGY MINIMIZATION
# 1) Initialization
units lj
dimension 3
atom_style atomic
boundary p p p

# 2) System definition
region simbox block -20 20 -20 20 -20 20
create_box 2 simbox
create_atoms 1 random 1500 34134 simbox overlap 0.3
create_atoms 2 random 100 12756 simbox overlap 0.3

# 3) Settings
mass 1 1.0
mass 2 5.0
pair_style lj/cut 4.0
pair_coeff 1 1 1.0 1.0
pair_coeff 2 2 0.5 3.0

# 4) Monitoring
thermo 10
thermo_style custom step etotal press

# 5) Run
minimize 1.0e-6 1.0e-6 1000 10000

# PART B - MOLECULAR DYNAMICS
# 4) Monitoring
thermo 50
thermo_style custom step temp etotal pe ke press
dump viz all image 100 myimage-*.ppm type type &
  size 800 800 zoom 1.452 shiny 0.7 fsaa yes &
  view 80 10 box yes 0.025 axes no 0.0 0.0 &
  center s 0.483725 0.510373 0.510373
dump_modify viz pad 9 boxcolor royalblue &
  bgcolor white adiam 1 1.6 adiam 2 4.8

# 5) Run
fix mynve all nve
fix mylqv all langevin 1.0 1.0 0.1 10917
timestep 0.005
run 15000
```

Interact with LAMMPS using input

```
# PART A - ENERGY MINIMIZATION
# 1) Initialization
units lj
dimension 3
atom_style atomic
boundary p p p

# 2) System definition
region simbox block -20 20 -20 20 -20 20
create_box 2 simbox
create_atoms 1 random 1500 34134 simbox overlap 0.3
create_atoms 2 random 100 12756 simbox overlap 0.3

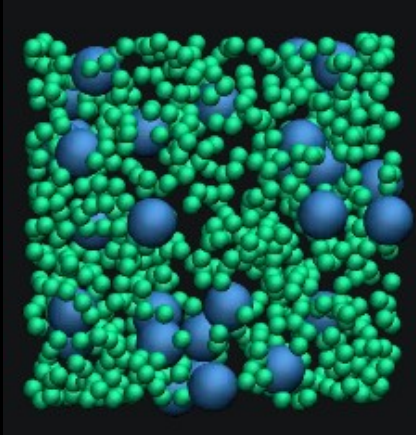
# 3) Settings
mass 1 1.0
mass 2 5.0
pair_style lj/cut 4.0
pair_coeff 1 1 1.0 1.0
pair_coeff 2 2 0.5 3.0

# 4) Monitoring
thermo 10
thermo_style custom step etotal press

# 5) Run
minimize 1.0e-6 1.0e-6 1000 10000

# PART B - MOLECULAR DYNAMICS
# 4) Monitoring
thermo 50
thermo_style custom step temp etotal pe ke press
dump viz all image 100 myimage-*.ppm type type &
  size 800 800 zoom 1.452 shiny 0.7 fsaa yes &
  view 80 10 box yes 0.025 axes no 0.0 0.0 &
  center s 0.483725 0.510373 0.510373
dump_modify viz pad 9 boxcolor royalblue &
  bgcolor white adiam 1 1.6 adiam 2 4.8

# 5) Run
fix mynve all nve
fix mylqv all langevin 1.0 1.0 0.1 10917
timestep 0.005
run 15000
```



Interact with LAMMPS using input

```
# PART A - ENERGY MINIMIZATION
# 1) Initialization
units lj
dimension 3
atom_style atomic
boundary p p p

# 2) System definition
region simbox block -20 20 -20 20 -20 20
create_box 2 simbox
create_atoms 1 random 1500 34134 simbox overlap 0.3
create_atoms 2 random 100 12756 simbox overlap 0.3

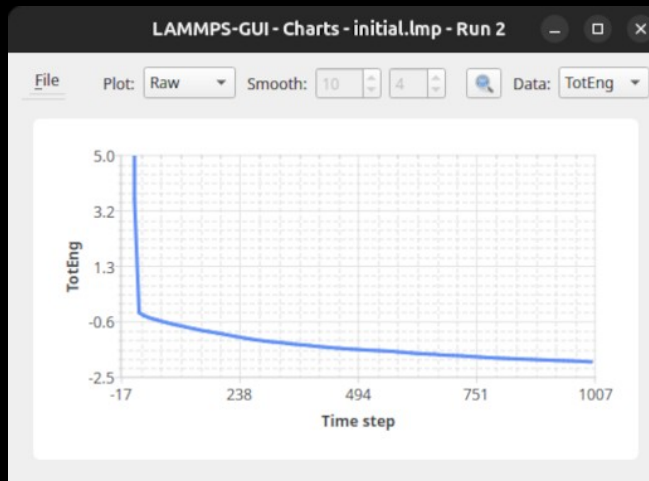
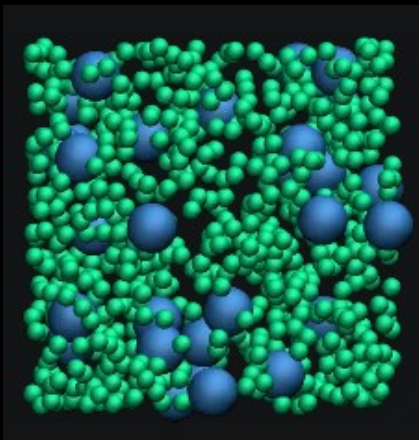
# 3) Settings
mass 1 1.0
mass 2 5.0
pair_style lj/cut 4.0
pair_coeff 1 1 1.0 1.0
pair_coeff 2 2 0.5 3.0

# 4) Monitoring
thermo 10
thermo_style custom step etotal press

# 5) Run
minimize 1.0e-6 1.0e-6 1000 10000

# PART B - MOLECULAR DYNAMICS
# 4) Monitoring
thermo 50
thermo_style custom step temp etotal pe ke press
dump viz all image 100 myimage-*.ppm type type &
  size 800 800 zoom 1.452 shiny 0.7 fsaa yes &
  view 80 10 box yes 0.025 axes no 0.0 0.0 &
  center s 0.483725 0.510373 0.510373
dump_modify viz pad 9 boxcolor royalblue &
  bgcolor white adiam 1 1.6 adiam 2 4.8

# 5) Run
fix mynve all nve
fix mylgv all langevin 1.0 1.0 0.1 10917
timestep 0.005
run 15000
```



LAMMPS-GUI - Output - initial.lmp - Run 2

Per MPI rank memory allocation (min/avg/max) = 21.03

Step	TotEng	Press
0	951929.29	95195.517
10	3.5701279	0.41939855
20	-0.29919041	-0.0039060286
30	-0.3941731	-0.0037206356
40	-0.46170501	-0.0035160833
50	-0.51118404	-0.0030873905
60	-0.55364542	-0.0023932664
70	-0.60105929	-0.0028490977
80	-0.63908384	-0.0024805522
90	-0.6799505	-0.0024682149
100	-0.71603181	-0.0024785996
110	-0.74686329	-0.0020756756
120	-0.78427168	-0.0022599663
130	-0.81793383	-0.0021402168
140	-0.85272023	-0.0019398829